第 16 章 推荐系统离线服务

在本书上册第2章中，我们介绍了推荐系统的整体架构，并分别讨论了召回、粗排、精排、重排等核心模块的设计与实现。同时，我们也对推荐系统的离线架构与在线架构进行了初步介绍。在线服务直接面向用户请求，负责实时完成候选生成、排序和推荐结果下发；离线服务则主要负责处理在线系统持续产生的各种数据，并通过特征生产、样本构造、模型训练和索引更新等环节，为在线推荐提供数据和模型基础。

从整个推荐系统的数据闭环来看，用户每一次浏览、点击、播放、点赞、关注以及其他行为，都会形成新的数据反馈。这些数据首先进入日志和消息队列系统，随后经过离线服务进行解析、清洗、关联和加工，最终形成模型训练样本、用户行为序列、统计特征、物品特征以及模型参数等数据。在线系统使用这些数据完成下一轮推荐，而新的推荐结果和用户反馈又会重新进入离线链路。因此，离线服务实际上承担着连接“在线推荐—用户反馈—数据生产—模型训练—在线更新”整个闭环的作用。

一个完整的工业级推荐系统离线架构通常包括样本流服务、用户行为日志解析服务、用户行为序列存储服务、统计特征生产服务、离线 Runner 服务、物品侧内容理解生产服务、物品索引特征服务、图存储服务以及离线训练服务等。除此之外，在强化学习推荐中，我们还需要构造包含后续状态的样本流，这样才能保证强化学习推荐模型的正常训练。

16.1 样本流服务

样本流服务是工业级推荐系统离线数据链路中最基础、也是最重要的服务之一。它负责将推荐请求、用户行为、物品信息以及上下文信息按照统一的数据协议进行组织，并最终形成供特征生产、模型训练、效果分析等下游任务使用的样本流。对于传统的监督学习推荐模型，一个训练样本通常可以表示为：

$$(user, item, context, label) \tag{16.1}$$

其中，$user$ 表示用户信息，$item$ 表示候选物品，$context$ 表示用户发起推荐请求时的上下文信息，$label$ 则表示用户在曝光之后产生的反馈，例如点击、播放、点赞、关注、分享以及消费时长等。

从这个角度来看，样本流服务实际上承担的是一个非常重要的数据关联工作。它需要将将发生在不同时间、甚至不同系统中的推荐请求与用户反馈重新关联起来。例如，一次推荐请求可能在时刻 t 产生，而用户的点击行为可能在数百毫秒之后发生，而观看时长则可能需要几十秒甚至更长时间才能确定。如果只保存单独的请求日志和行为日志，就无法直接知道某个 Label 对应的是哪个请求、哪个物品以及哪个版本的请求特征。因此，样本流服务通常需要依赖 Request ID、Item ID、User ID、设备 ID 以及时间戳等关键字段，通过消息队列、缓存以及流式处理等机制，将请求侧特征和反馈侧 Label 进行关联。

读到这里，读者朋友可能会问如下的问题：

笔记 既然推荐服务在用户请求时已经记录完整的特征，用户行为反馈也有独立的日志，那为什么不在训练的时候把两类日志做一次离线 Join 的操作，而要专门建设样本流服务来进行数据的拼接呢？

答案在于时效性与数据定义的统一。直接在训练前关联两类离线日志当然是可行的，这种方式通常适合天级或者小时级批处理的场景；但当模型更新周期缩短到小时级甚至分钟级时，系统链路需要持续接收陆续到达的反馈，并处理重复上报、延迟更新以及乱序事件等问题。在这种情况下，样本流服务的价值不是取代所有离线 Join，而是把这些规则放在同一条数据生产流程中，让不同的下游使用同一套样本定义，而不必各自重复清洗原始日志。

因此，样本流服务要做的事情如下：

笔记 样本流服务将属于同一次推荐请求的请求侧特征与反馈侧 Label 重新关联起来，既不漏掉应有数据，也不把先后顺序弄错，并尽快产出格式统一的训练样本流。

根据对实时性的要求不同，工业推荐系统中的样本流通常可以分为实时样本流和批量样本流两种形式。对于短视频、直播、信息流等用户反馈密集、模型更新频繁的业务，通常采用实时样本流，通过 Kafka 等消息队列

将推荐请求和用户行为实时发送给样本流服务，使下游能够尽快获得最新训练数据。对于用户反馈较为稀疏或者对模型更新实时性要求不高的业务，则可以采用小时级甚至天级的批量样本流。

实时样本流能够显著缩短“用户行为发生”到“样本进入训练系统”之间的时间差，使模型更加及时地学习用户兴趣变化，但同时也会遇到数据乱序、重复上报、延迟反馈、数据丢失以及服务不稳定等工程问题。因此，一个成熟的样本流系统需要保证数据不重复、不缺失、顺序正确，并能及时产出。

16.1.1 普通任务样本流服务

普通任务样本流主要服务于传统的监督学习模型。在线推荐系统在用户发起请求时记录用户特征、上下文特征、候选物品特征和模型分数，而客户端随后异步上报曝光、点击、播放、互动和消费时长等反馈行为。样本流需要把两条不同时间到达的数据重新关联为如下数据组织形式：

$$(user, item, context, pCTR, pWatchTime, \ldots, label_1, \ldots, label_n) \tag{16.2}$$

其中，请求侧字段描述模型作出判断时能够观察到的信息，$label_1, \ldots, label_n$ 表示候选物品展示后产生的多种不同反馈。二者必须通过稳定的 Request ID 和 Item ID 对齐，否则同一用户连续刷新时，一条行为很容易被拼到另一刷请求的特征上。

普通任务样本流处理的整体架构如图16.1所示。推荐系统首先将原始请求特征写入 Kafka；特征预处理消费者在请求侧完成过滤、裁剪以及可选的特征编码，输出体积更小的特征流；反馈标签消费者独立消费用户行为，并将用户反馈写入 Redis 等短期存储；样本拼接消费者消费预处理后的请求特征，同时读取 Redis 中对应的用户反馈，最终生成完整的训练样本，供下游训练服务消费。三个消费者通过 Kafka 消息队列进行实时衔接，用户反馈的写入也无需等待请求侧处理完成，训练服务也无需感知 Redis 缓存中的临时状态。由此，各服务按照职责分工通过消息队列和短期存储进行连接，形成一个弱耦合的普通任务样本流拼接框架。

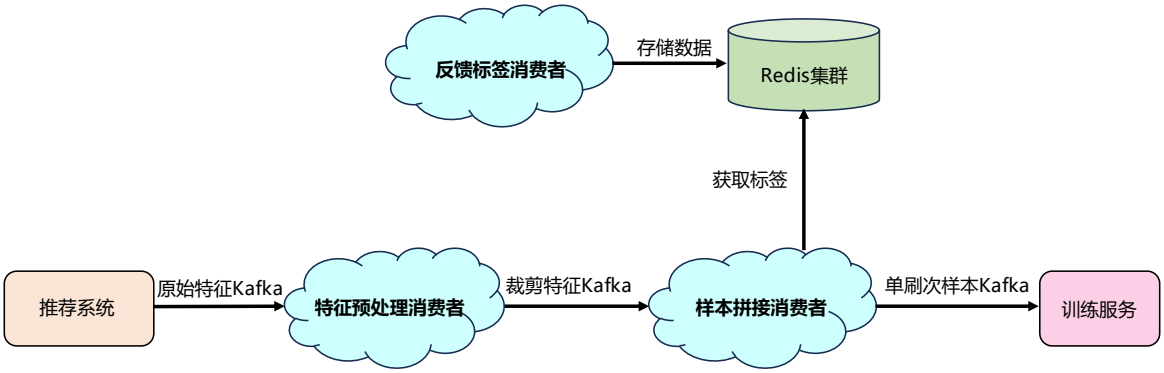


图 16.1: 普通任务样本流拼接的离线架构示意图。

图16.1中的特征预处理消费者、样本拼接消费者和反馈标签消费者分别负责请求侧数据处理、请求与反馈关联以及行为归档。需要注意的是，特征预处理并非所有样本流都必须经过的环节。当请求量较小，或原始请求特征已经符合训练数据格式时，样本拼接消费者可以直接消费 Kafka 中的原始请求数据；而在高吞吐推荐系统中，通常将特征预处理作为独立阶段，在数据进入下游之前完成过滤和裁剪，从而尽早降低数据传输带宽与后续处理的计算开销。

16.1.1.1 特征预处理消费者

特征预处理消费者位于请求日志与样本拼接消费者之间。它首先会校验 Request ID、请求场景、请求类型以及候选列表是否完整，过滤不参与训练的实验流量（比如长期 holdout 流量等）、异常请求以及重复请求；随后按照白名单来裁剪用户侧特征与物品侧特征，删除调试字段、大对象字段和训练不会使用到的冗余属性；最后可以把原始特征转换为训练任务底层 Embedding 层可以直接使用的哈希键值数据。

在工业级推荐模型训练中，许多类别型或离散化特征最终都会表示为 slotId 与 sign 的组合，其中 slotId 表示特征所属的槽位编号，而 sign 则是由特征取值、必要的命名空间以及哈希规则得到的长整型数字标识。在请求侧日志中如果仍然保留字符串特征名、原始字符串值、复杂的列表以及嵌套的对象结构，则单刷次的请求日志就会非常大，在实际中可能占用几个 G 甚至几十个 G 的带宽；如果在特征预处理阶段完成字段裁剪、槽位映射以及哈希编码等操作，Kafka 消息队列后续传输的只是一组紧凑的整数列表。这样下游的模型训练任务就可以直接读取 slotId/sign 对应的列表即可构造 Embedding 输入，而不必再次解析 JSON 或者 protobuf 等复杂的结构、也不需要查找特征字典或者重复执行哈希操作等。这种方式可以显著降低 Kafka 的网络带宽、下游训练任务反序列化处理成本以及训练任务的 CPU 开销。

代码16.1给出了特征预处理部分的代码骨架。前面的 FeaturePreprocessConsumer 是 Kafka 消费者，负责消费原始请求日志，并将处理后的结果写入特征精简后的 Kafka Topic；后面的 preprocess 方法负责具体的特征裁剪与编码。该方法按照训练配置分别处理用户侧和物品侧特征，同时保留 Request ID、设备 ID、Item ID、Session ID 和请求时间戳等用于后续样本关联的元数据。其中，字符串特征按照约定的哈希规则映射为整数，数值特征则保留原始数值。输出记录还需要携带特征规则版本，使训练侧与线上特征服务能够基于相同版本执行一致的字段筛选、特征映射和数值处理。

代码 16.1: 请求侧特征裁剪与哈希编码

```
// 对应文件：NormalSampleFlow.java
/** Kafka 消费者：消费原始请求特征日志（RawRequestWithFeatures），完成特征裁剪与哈希编码后输出精简 ↩
    ↪ Topic。 */
public class FeaturePreprocessConsumer {
    private final FeaturePreprocessService preprocessService;
    private final KafkaProducer<String, FeatureReadyRequest> producer;

    public void consume(RawRequestWithFeatures request) {
        FeatureReadyRequest result = preprocessService.preprocess(request);
        if (result != null) {
            // FeatureReadyRequest 保留 requestId/deviceId/sessionId/scene 等关键元数据；
            // 特征部分已编码为 slotId/sign 整数对，不再包含原始字符串字段。
            producer.send("sample_join_waiting", result);
        }
    }
}

// 对应文件：SampleFlowService.java
public FeatureReadyRequest preprocess(RawRequestWithFeatures request) {
    if (!requestValidator.isTrainable(request) || deduplicator.seen(request.getRequestId())) {
        metrics.count("drop_invalid_or_duplicate_request");
        return null;
    }

    FeatureSchema schema = featureSchemaProvider.forVersion(request.getFeatureSchemaVersion());
    FeatureReadyRequest.Builder output = FeatureReadyRequest.newBuilder()
            .setRequestId(request.getRequestId())
            .setRequestTimeMs(request.getRequestTimeMs())
            .setDeviceId(request.getDeviceId())
            .setSessionId(request.getSessionId())
            .setScene(request.getScene())
```

```
            .setFeatureSchemaVersion(schema.version());

    for (RawFeature feature : request.getCommonFeatures()) {
        EncodedFeature encoded = encode(feature, schema.commonRule(feature.name()));
        if (encoded != null) {
            output.addCommonFeature(encoded); // slotId + sign + optional numeric value
        }
    }

    for (RawItem item : request.getItems()) {
        EncodedItem.Builder encodedItem = output.addItemBuilder().setItemId(item.getItemId());
        for (RawFeature feature : item.getFeatures()) {
            EncodedFeature encoded = encode(feature, schema.itemRule(feature.name()));
            if (encoded != null) {
                encodedItem.addFeature(encoded);
            }
        }
    }

    return output.build();
}

private EncodedFeature encode(RawFeature feature, FeatureRule rule) {
    if (rule == null || !rule.isEnabled() || feature.isEmpty()) {
        return null;
    }

    long sign = rule.valueType() == ValueType.NUMERIC
            ? feature.numericValue()
            : stableHash(rule.namespace(), feature.stringValue(), rule.hashSeed());
    return EncodedFeature.of(rule.slotId(), sign, rule.valueType());
}
```

这里给出的代码主要用于说明特征预处理的基本流程。实际工业推荐系统中的特征处理通常更加复杂，但核心思想是一致的，即按照统一规则将原始特征转换为模型可以直接消费的表示。上述代码将原始字段编码为稳定的 slotID 和 sign 取值，处理后的特征可以被多个训练任务复用。这种前置特征抽取的方式适合请求 QPS 高、原始特征体积大，且多个下游任务需要共享相同编码规则的场景。在实际处理中，我们还需要关注哈希碰撞、缺失值、数值特征离散化以及离在线编码一致性等问题，否则可能引入特征噪声或者降低特征覆盖率。当特征的 Schema 发生变化时，我们也应通过版本字段或新旧版本并行的方式进行兼容，避免新映射规则直接解析历史日志，进而造成训练数据分布变化。采用修改替换而非新增特征字段取值的这种操作在工业界实践过程中非常危险，因为特征产出环节下游的训练任务可能非常多，如果强行在上游进行特征版本的更新，可能导致下游模型的数据分布变化从而产生输出值变化，在极端情况下甚至会造成线上故障。

当然，特征抽取过程也可以放在样本拼接之后。此时，样本拼接消费者先生成包含完整请求特征和 Label 的训练样本，再由独立的 Runner 或 Kafka 消费者完成字段裁剪、特征处理以及哈希编码。特征抽取过程前置和后置这两种方案最终都可以生成相同的训练 protobuf，它们主要的区别在于特征计算发生的阶段。前置处理能够降低后续 Kafka 传输、存储以及下游计算开销，适合高吞吐且多个任务共享特征的场景；后置处理只处理最终进入训练的数据，更适合数据过滤比例较高或特征规则迭代频繁的场景。

因此，特征抽取的前置和后置并不存在绝对的优劣，需要结合数据规模、资源压力和特征迭代频率进行选择。无论采用哪种方案，都应在输出记录中携带特征 Schema 版本和哈希规则版本，保证训练数据、离线回刷和在线特征服务使用一致的编码规则。

16.1.1.2 Label 更新规则

在实际系统中，请求日志与行为日志通常来自不同的数据链路。在推荐服务产生请求日志之后，样本流很快就可以从 Kafka 中收到对应的请求消息；而点击、停留时长等用户反馈则需要等用户完成相应操作之后才能上报，因此用户反馈行为可能与请求发起相隔数秒，甚至数分钟的时间间隔。为了解决这种“请求先到、反馈后到”的时序差异，反馈标签消费者通常不会直接修改训练样本，而是先将行为写入短期缓存中，后续样本拼接消费者在观察窗口结束后再读取这些反馈记录，并将其补充到对应的请求样本中。

对于点击、点赞、关注等只有“发生”与“未发生”这两种状态的用户反馈，可以采用 Bitmap 结构来进行高效的保存；而对于播放次数、观看时长、打赏金额等带有数值的用户反馈，则需要按具体业务含义采用计数器进行累计处理或者始终保存最新的标签状态。上述这两类记录都可以使用 Request ID 与 Item ID 建立索引，即：

$$(ReqID, ItemID) \rightarrow \{Action, ActionValue, ShowPosition\} \tag{16.3}$$

其中，$Action$ 表示已经发生的行为类型，$ActionValue$ 保存行为对应的数值，例如观看时长、播放次数或消费金额；$ShowPosition$ 则记录物品最终实际展示的位置。需要注意的是，展示位置不能简单地根据请求中的候选排序结果推断，因为推荐模型排序之后通常还会经过重排、插入策略以及客户端重排处理等环节，最终候选内容曝光的顺序可能发生变化。因此，将真实的展示位置一并写入样本，训练侧才能区分模型输出的排序位置与用户实际看到的位置，从而更准确地刻画曝光位置对行为标签的影响。

此外，用户反馈消息还可能存在重复上报或多次行为的问题。同一个 Label 可能由于客户端重试、网络延迟等原因被重复发送，也可能因为用户对同一个物品产生了多次真实行为而出现多条反馈。因此，样本流不能简单地将每条反馈消息都视为一次独立反馈，而需要先为不同 Label 写清更新规则。对于点击、点赞、关注等发生型 Label，只需要记录行为是否发生，重复消息再次置位即可；对于播放次数、互动次数等累计型指标，则应按照业务定义进行累加。观看时长还需要进一步区分上游发送的是“增量值”还是“截至当前时刻的累计值”：对于增量上报，每条时长都应累加；对于累计值上报，则应根据事件时间保留最新的有效记录，或者在业务明确允许的情况下保留更大的有效值。对于退出原因等不具有累计意义的状态信息，则更适合采用覆盖写入。

因此，反馈存储的关键并不是简单地“保存最后一条消息”，而是首先明确每种 Label 的更新规则。只有这样，样本流才能在反馈重复、延迟以及乱序到达的情况下尽可能保持 Label 的正确性。否则，重复消息可能导致次数类 Label 被虚增，而简单采用最近一次消息又可能丢失更早但更完整的行为信息。

下面通过一个短视频推荐中的具体案例说明这一问题。

笔记 用户在 Feed 流上下滑过程中，可能先按 $A \rightarrow B \rightarrow C \rightarrow D$ 的顺序浏览视频并完成正常观看，随后从 D 向上回滑重新查看前面的内容。第二次浏览时，A、B 和 C 都会再次播放，但用户可能在 B 和 C 上停留不到 1 秒就再次划走。此时，同一个 Request ID 和 Item ID 可能对应两次甚至多次播放行为。

假设第一次播放 B 视频时产生了完整的观看时长，而回滑时只产生了一个很短的播放时长。如果样本拼接阶段简单地按照“同一个 Request ID 下，同一个 Item ID 只保留最近一次反馈”的规则进行聚合，那么最终很可能只保留回滑产生的短时长，而丢失第一次完整观看对应的时长 Label。

代码16.2给出了反馈标签消费者的基本写入逻辑。消费者首先通过共享枚举识别行为类型，对于未知行为仅记录监控信息而不写入训练状态；对于已经确认的训练 Label，发生型反馈写入 Bitmap，数值型反馈则根据预先定义的更新模式执行累加或最新值更新。同时，系统记录反馈事件的发生时间，供后续样本拼接判断反馈是否仍处于观察窗口内，以及是否需要继续等待后续行为。需要注意的是，这里的“最新值”是按照业务事件时间确定的最新有效记录，而不是简单地按照消息到达顺序覆盖。对于存在多次行为的 Label，还需要在消费者阶段完成正确的聚合，而不能把所有问题都留给下游样本拼接阶段处理。换句话说，反馈消费者负责按上述规则保存 Label，样本拼接消费者则负责在合适的观察窗口结束后读取这一状态并生成训练样本。两者职责分开后，才能同时处理好反馈的重复、延迟、乱序以及同一物品多次真实行为等情况。

代码 16.2: 普通样本流的反馈暂存与更新

```
// 对应文件：NormalSampleFlow.java (FeedbackLabelConsumer + FeedbackStore)
/** Kafka 消费者：消费 ActionLog，通过 requestId + itemId 索引写入反馈存储。 */
public class FeedbackLabelConsumer {
    private final FeedbackStore feedbackStore;
    public void consume(ActionLog actionLog) {
        long reqId = actionLog.firstRequestId();
        if (reqId <= 0 || actionLog.itemIds.isEmpty()) return;
        for (int i = 0; i < actionLog.itemIds.size(); i++) {
            long itemId = actionLog.itemIds.get(i);
            long value = i < actionLog.actionValues.size()
                    ? actionLog.actionValues.get(i) : 1L;
            feedbackStore.record(new FeedbackEvent(
                    reqId, itemId, actionLog.actionName, value, actionLog.eventTimeMs));
        }
    }
}

public class FeedbackStore {
    private final MockRedis redis;

    public FeedbackStore(MockRedis redis) { this.redis = redis; }

    public void record(FeedbackEvent event) {
        if (event.action == null || event.action.isBlank()) return;

        redis.setOccur(event.requestId, event.itemId, event.action);

        if (isNumeric(event.action)) {
            redis.incrby(event.requestId, event.itemId, event.action, event.value);
        }

        redis.setTimestamp(event.requestId, event.eventTimeMs);
    }

    private boolean isNumeric(String action) { return action.equals("watch_time"); }
}
```

上述代码根据不同反馈动作的语义将用户行为分别写入 Redis，并记录最近一次反馈时间，供样本拼接消费者在等待窗口内读取。因此，Redis 的写入和读取必须遵循一致的行为枚举和数值更新规则。例如，写入方将观看时长作为增量持续累加，而读取方却将其理解为累计值，就会导致训练样本中的时长数据产生系统性偏差。此类问题通常难以通过单条请求直接发现，更适合结合 Label 分布、重复上报率以及线上日志与训练样本的抽样核对进行监控。换句话说，样本流不仅要保证反馈消息能够正确写入，还要保证这些反馈进入训练样本后，仍然符合原有的业务语义。

16.1.1.3 等待窗口与重复投递

在样本拼接消费者的处理过程中，请求日志到达并不意味着样本已经具备完整的反馈信息，因此样本通常不能在请求消息到达后立即完成拼接。如果等待窗口设置过短，部分延迟到达的用户反馈尚未进入短期存储就会被判定为缺失，导致慢反馈被系统性丢弃；如果等待窗口设置过长，则会增加样本从请求产生到最终进入训练数据的延迟，同时带来更大的缓存占用和消息队列压力。因此，实际系统通常不会为所有请求使用固定的等待时间，而是根据请求类型、反馈类型以及当前已经观察到的行为状态设置基础窗口和最大窗口。对于已经观察到关键反馈的请求，可以采用较短的窗口继续等待可能补充到达的行为；对于尚未观察到关键反馈的请求，则等待到最长的配置时间。如果在等待过程中持续收到新的用户行为，还可以根据最近一次行为时间适当延长活跃观察期，使观察窗口能够覆盖用户仍在持续消费的场景。

在流式处理系统中，消费者不能简单地阻塞当前消息并等待未来的反馈事件，否则会长期占用消费线程，降低整个消费链路的处理吞吐。因此，一种直接的实现方式是将暂时未满足完成条件的请求重新写入等待 Topic，在后续消费时再次检查。代码16.3展示了这种反复投递消息的实现方式。在重新投递时，Kafka 消息继续携带原始请求的 Request ID 等标识字段，并尽可能使用与首次写入时相同的分区键，使同一个请求在多次回到队列后仍尽量进入相同分区，从而降低跨分区状态管理的复杂度。需要注意的是，短暂休眠只能减缓消息被立即再次消费形成的循环，并不能决定请求实际等待多久。请求是否完成拼接，仍然需要根据事件时间、最近一次反馈时间、观察窗口以及最大重试次数等条件进行综合判断。

代码 16.3: 普通样本流的等待窗口与延迟再次投递

```
// 对应文件：NormalSampleFlow.java (SampleJoinConsumer)
public class SampleJoinConsumer {
    private final FeedbackStore feedbackStore;
    private final KafkaProducer<String, FeatureReadyRequest> producer;
    // 重投次数计数，工业落地时放在消息元数据里随消息流转
    private final Map<Long, Integer> replayCount = new HashMap<>();

    public void consume(FeatureReadyRequest request) {
        long nowMs = System.currentTimeMillis();
        int rc = replayCount.getOrDefault(request.getRequestId(), 0);

        if (feedbackStore.shouldWait(request, nowMs, rc)) {
            replayCount.put(request.getRequestId(), rc + 1);
            // 重新写入等待 Topic，使用相同分区键使同一请求始终进同一分区
            producer.send("sample_join_waiting", request.getRequestKey(), request);
            return;
        }

        joinAndEmit(request);
    }
}

// 对应文件：NormalSampleFlow.java (FeedbackStore.shouldWait)
public boolean shouldWait(FeatureReadyRequest request, long nowMs, int replayCount) {
    long firstTs = firstKeyFeedbackTime(request.getRequestId());
    long lastTs = lastFeedbackTime(request.getRequestId());

    long windowMs = firstTs > 0 ? BASE_WAIT_MS : MAX_WAIT_MS;
    long deadline = request.getRequestTimeMs() + windowMs;
```

```
    // 最近反馈后延长活跃观察窗口，覆盖用户仍在持续消费的场景
    long activeDeadline = lastTs > 0 ? lastTs + ACTIVE_EXT_MS : 0;
    return nowMs < Math.max(deadline, activeDeadline) && replayCount < 5; // MAX_REPLAY_COUNT = 5
}
```

上述代码在等待窗口未结束时把同一请求重新写入 Topic，因而把同步等待改造成了异步重试。这样的反复投递实现简单，但会放大消息量，并可能改变消息到达的相对顺序。请求等待得越久、再次投递的间隔设得越短，同一请求被重新写入 Topic 的次数就越多。因此，不能只靠缩短间隔来提高反馈检查频率；这样虽然能更早发现新到达的反馈，却会同时增加 Kafka 的读写压力和消费者负载。当流量上升时，等待中的请求会与新请求争抢消费能力；样本拼接消费者一旦出现 Lag，新反馈会更晚被处理，仍在等待的请求又会继续写回队列，形成不断放大的循环。此时 Redis 中保存反馈的临时记录也无法及时清理，写入量和存储占用会持续增长。若没有及时限流或停止接收部分请求，Kafka 积压、Redis 容量压力和样本拼接率下降会相互影响，最终造成训练样本缺失，甚至引发服务故障。

因此，生产环境需要同时监控再次投递消息的比例、单个请求被再次投递的次数、从请求产生到样本输出的耗时，以及 Kafka Topic 的消息积压量，并根据实际反馈延迟来调整等待的窗口与间隔。如果已经达到最大尝试次数却仍未满足完成条件的请求，服务必须将其标记为缺失、超时或者截断样本，并停止继续写回队列。这样既能留下可追踪的处理结果，也能避免少量异常请求长期循环，拖慢整个样本流拼接过程。

16.1.1.4 逐物品拼接、过滤与清理

在等待窗口结束后，样本拼接消费者才会开始把请求侧特征与反馈侧 Label 合并成为训练样本。它先一次性读取本次请求中全部候选物品的行为集合、数值反馈和真实展示位置，再按候选物品逐一构造样本项。请求级公共特征在整条 `TrainingBatch` 中只保存一份，而每个候选物品自身的特征、Label 和展示位置则写入对应的 `ItemSample`。代码16.4给出了这部分主流程。只有请求中至少存在一个候选获得有效反馈，消费者才会输出这条训练样本。

代码 16.4: 普通任务样本的逐物品拼接主流程

```
// 对应文件：NormalSampleFlow.java (SampleJoinConsumer.joinAndEmit)
private void joinAndEmit(FeatureReadyRequest request) {
    Map<Long, Map<String, Long>> allLabels = new HashMap<>();
    boolean hasValid = false;

    for (long itemId : request.getItemIds()) {
        Set<String> actions = feedbackStore.getActions(request.getRequestId(), itemId);
        Map<String, Long> labelMap = new HashMap<>();
        for (String action : actions) {
            labelMap.put(action, feedbackStore.getValue(request.getRequestId(), itemId, action));
            hasValid = true;
        }

        allLabels.put(itemId, labelMap);
    }

    if (!hasValid) {
        // drop_no_feedback
        return;
    }
```

```
    TrainingBatch batch = new TrainingBatch(
        request.getRequestId(),
        request.getScene(),
        request.getRequestTimeMs() * 1000L,
        allLabels
    );
    kafka.send("training_sample_stream", batch);
    feedbackStore.clear(request.getRequestId(), request.getItemIds());
}
```

上述代码从已写入的反馈记录恢复候选行为集合，并将其交给后续样本过滤与标签构造。行为集合为空通常说明请求尚未关联任何用户反馈；只有曝光占位则表示系统已确认物品被展示，但尚未观察到点击、观看等后续行为。这两种情况是否应丢弃，取决于推荐模型对正负样本的定义，而不能只凭 Redis 缓存中是否存在这个 Key 判断。

在完成基础的请求与反馈拼接之后，样本流还可以根据不同模型的训练目标补充一些额外特征。例如，当推荐模型需要了解用户与候选作者或内容实体之间的近期互动时，系统可以先在请求携带的行为序列中找到最近一次交互，再统计该时间点之后发生的观看、互动和消费行为。代码16.5给出了这部分特征构造的实现。代码通过二分查找确定回溯区间的起点，因此不需要为每个候选物品反复扫描整段历史序列。需要注意的是，这类历史特征只是对基础样本的补充，基础样本流仍然主要负责请求特征与用户反馈的拼接，而不是重点负责新特征的产出。即使某个模型后续增加了新的历史窗口，也不应该迫使所有样本消费者修改主流程。

代码 16.5: 按候选实体回溯历史反馈的可选增强层

```
// 对应文件：SampleFlowService.java (appendHistoryFeatures)
public void appendHistoryFeatures(ItemSample.Builder sample, long entityId, BehaviorSequence ↩
    ↪ history, long anchorTimeMs) {
    List<BehaviorEvent> events = history.eventsSortedByTime();
    int start = lowerBound(events, anchorTimeMs - historyWindowMs);

    Map<FeedbackKind, Long> amounts = new EnumMap<>(FeedbackKind.class);
    Set<FeedbackKind> occurred = EnumSet.noneOf(FeedbackKind.class);
    for (int i = start; i < events.size(); i++) {
        BehaviorEvent event = events.get(i);
        if (event.getEventTimeMs() > anchorTimeMs || event.getEntityId() != entityId) {
            continue;
        }

        occurred.add(event.getKind());
        if (event.getKind().valueMode() == ValueMode.SUM) {
            amounts.merge(event.getKind(), event.getValue(), Long::sum);
        }
    }

    occurred.forEach(kind -> sample.addFeature(IntFeature.of("history_" + kind.featureName(),1)));
    amounts.forEach((kind, value) -> sample.addFeature(
           LongFeature.of("history_" + kind.featureName() + "_value", value)));
}
```

上述代码按候选实体定位回溯区间并写入历史聚合值，得到的增强特征随后与基础样本一起交给训练任务。构造这些历史特征时，实体、时间和数值三类序列必须严格对应。例如，实体列表中的某次交互如果配上了错误的时间或数值，就可能生成看似正常、实际没有业务含义的特征。因此，这几类列表不仅需要有相同长度，还需要使用相同的排序方向。当消费者发现某个列表缺失，或者不同列表已经错位时，应跳过这部分额外特征并记录监控信息，不能让可选的增强逻辑阻断基础样本的产出。此外，回溯窗口大小、是否只统计推荐场景内的行为，以及数值如何聚合，都需要写入带版本的配置。这样重新处理历史数据时，训练任务才能还原当时使用的特征定义。用户行为序列存储服务负责维护标准的序列特征。

除了输出训练样本之外，普通样本流还需要持续记录请求在链路中的处理结果。例如，在请求级别，系统需要统计输入量、等待量、超时量、无反馈丢弃量和成功产出量；而在物品级别，则需要观察各类 Label 的出现次数、数值分布和展示位置分布。与此同时，链路级监控还应记录从请求产生到训练样本输出的完整延迟。当样本数量发生变化时，只有将这些指标继续按客户端、请求类型、实验组等维度拆开，工程师才能判断变化究竟来自用户行为和实验流量，还是某一段反馈链路出现了具体的异常。

16.1.1.5 基于 Flink 的普通样本流优化

在前面小节的代码实现中，样本拼接消费者通过 Kafka 把请求再次投递到队列中，以等待后续的用户反馈。这种方式可以直接复用既有的 Consumer 架构，因此在请求量较小，或者需要快速验证样本流逻辑时较容易落地。不过，请求等待反馈本质上仍是重复投递。假设等待窗口为 W，再次投递间隔为 I，一条请求在等待期间平均会带来约 W/I 次额外消费。当大量请求同时等待反馈时，这些额外消息还会与新请求争抢同一分区的消费能力；如果请求量继续上升，重复投递本身就可能拖慢样本产出。

Flink 框架提供了另一种处理方式。它将请求流和反馈流同时交给同一个有状态作业，由作业保存尚未完成的请求和已经到达的用户反馈，而不再把请求反复写回 Kafka。由于请求记录和反馈记录都按照 Request ID 分区，同一次请求的等待状态始终保存在同一个 Key 中。在采用 Flink 框架之后，普通样本流向下游输出的数据协议并不会发生变化。请求流仍然使用 `RequestEvent`，其中包含公共特征和候选物品；反馈流仍然使用 `FeedbackEvent`，其中记录 Request ID、Item ID、行为、数值和事件时间。两条数据流最终产出的仍然是上一节定义的 `TrainingBatch`。

基于 Flink 框架的样本拼接与前一小节的不同之处在于等待用户反馈的实现方式。当用户请求到达时，作业会将请求写入 `ValueState`；而当用户反馈到达时，作业会按照物品和行为将反馈累积到 `MapState`。等到事件时间定时器到期后，作业再读取这两部分状态并一次性组装训练样本。这样，等待反馈不会占用消费线程，同一条请求也不需要在 Kafka Topic 中反复流转。

代码16.6给出了 Flink 双流拼接的完整作业主干。作业从两个 Kafka topic 分别读取请求流和反馈流，通过 `addSource` 将两条流引入 `StreamExecutionEnvironment`，再各自配置有界乱序 Watermark 策略后，按 Request ID 做 `keyBy` 并通过 `connect` 汇入同一个 `SampleJoinFunction`（KeyedCoProcessFunction）。这里使用事件时间而不是处理时间，是因为请求日志和用户反馈可能经由不同的 Kafka 分区乱序到达。Watermark 策略负责判断系统还需要容忍多晚到达的事件，而业务等待窗口 `MAX_WAIT_MS` / `BASE_WAIT_MS` 则决定在反馈仍然不完整时，拼接算子愿意等待多久。这两个时间概念处理的问题并不相同。Watermark 用来处理事件乱序，等待窗口用来约束业务等待。如果把它们混作同一个参数，系统要么会过早关闭仍有机会补齐的训练样本，要么会让已经无须继续等待的状态长期停留在内存中。

代码 16.6: Flink 双流普通样本拼接主作业

```
// 对应文件：FlinkSampleJoiner.java（buildJob）
// 从两个 Kafka topic 读取请求流和反馈流，
// 按 requestId keyBy 后交给 SampleJoinFunction（KeyedCoProcessFunction）
// 完成等待与拼接，结果写回 training_samples topic。
public static void buildJob(String bootstrapServers) throws Exception {
    StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
```

```
// EXACTLY_ONCE Checkpoint: 外部化快照，任务取消后保留
env.enableCheckpointing(60_000L, CheckpointingMode.EXACTLY_ONCE);
env.getCheckpointConfig().setCheckpointTimeout(300_000L);
env.getCheckpointConfig().enableExternalizedCheckpoints(
    ExternalizedCheckpointCleanup.RETAIN_ON_CANCELLATION);
env.setMaxParallelism(1024);

Properties kafkaProps = new Properties();
kafkaProps.setProperty("bootstrap.servers", bootstrapServers);
kafkaProps.setProperty("group.id", "sample-joiner");

//   请求流: feature_ready_requests
// 特征预处理消费者将完整请求写入此 topic; 按 requestTimeMs 分配 Watermark
DataStream<RequestEvent> requestStream = env
    .addSource(new FlinkKafkaConsumer<>(
        "feature_ready_requests", new RequestEventDeserializer(), kafkaProps))
    .name("RequestSource")
    .assignTimestampsAndWatermarks(
        WatermarkStrategy.<RequestEvent>forBoundedOutOfOrderness(Duration.ofSeconds(5))
            .withTimestampAssigner((e, t) -> e.requestTimeMs)
            .withIdleness(Duration.ofSeconds(30)));

//   反馈流: user_feedback_events
// 行为日志服务将点击/播放/点赞等用户行为写入此 topic
DataStream<FeedbackEvent> feedbackStream = env
    .addSource(new FlinkKafkaConsumer<>(
        "user_feedback_events", new FeedbackEventDeserializer(), kafkaProps))
    .name("FeedbackSource")
    .assignTimestampsAndWatermarks(
        WatermarkStrategy.<FeedbackEvent>forBoundedOutOfOrderness(Duration.ofSeconds(5))
            .withTimestampAssigner((e, t) -> e.eventTimeMs)
            .withIdleness(Duration.ofSeconds(30)));

//   核心拼接: 双流按 requestId 分区后 connect
// SampleJoinFunction 在 Keyed State 中等待反馈，定时器触发后出样
DataStream<JoinedSample> output = requestStream
    .keyBy(e -> e.requestId)
    .connect(feedbackStream.keyBy(e -> e.requestId))
    .process(new SampleJoinFunction())
    .name("SampleJoinProcessor").uid("SampleJoinProcessor");

//   结果写回 training_samples: 事务型 Sink 与 Checkpoint 对齐
// 保证故障恢复后不重复输出; 若同时写 Redis 等外部系统须另行去重
output.sinkTo(KafkaSink.<JoinedSample>builder()
    .setBootstrapServers(bootstrapServers)
    .setRecordSerializer(KafkaRecordSerializationSchema.builder()
        .setTopic("training_samples")
        .setValueSerializationSchema(new JoinedSampleSerializer())
```

```
            .build())
        .setDeliveryGuarantee(DeliveryGuarantee.EXACTLY_ONCE)
        .build()).name("TrainingSampleSink");

    env.execute("SampleFlowJoiner");
}
```

代码16.7进一步展示了请求状态、用户反馈状态和定时器如何协同工作。对于每个 Request ID，Flink 作业都会保存尚未完成的请求、按照物品和行为聚合的反馈，以及当前的等待结束时间。当新的关键反馈到达时，作业会根据最近一次用户行为重新计算等待截止时间。此时不能直接注册新的定时器，而必须先删除旧定时器；否则旧回调仍会在原来的时间触发，并在新反馈尚未完成累积时提前输出样本，甚至清空仍然需要保留的状态。

代码 16.7: 请求与反馈在 Keyed State 中等待和汇合

```
// 对应文件：FlinkSampleJoiner.java (SampleJoinFunction)
public static class SampleJoinFunction {
    private static final long MAX_WAIT_MS = 120_000L; // 无关键反馈时最长等待
    private static final long BASE_WAIT_MS = 30_000L; // 有关键反馈时等待
    // [MOCK] 工业落地替换为：ValueState<RequestEvent> requestState;
    private final Map<Long, ValueState<RequestEvent>> requestStates = new HashMap<>();
    // [MOCK] 工业落地替换为：MapState<String, Long> feedbackState;
    private final Map<Long, MapState<String, Long>> feedbackStates = new HashMap<>();
    // [MOCK] 工业落地替换为：ValueState<Long> keyFeedbackTs;
    private final Map<Long, ValueState<Long>> keyFeedbackTsStates = new HashMap<>();
    private final Map<Long, TimerService> timerServices = new HashMap<>();
    private final Collector<JoinedSample> out = new Collector<>();
    public void processElement1(RequestEvent req) {
        long reqId = req.requestId;
        getRequestState(reqId).update(req);
        // 注册最长等待截止定时器（Event Time）
        getTimerService(reqId).registerEventTimeTimer(req.requestTimeMs + MAX_WAIT_MS);
    }

    public void processElement2(FeedbackEvent event) {
        long reqId = event.requestId;
        // 同一物品和行为的多次上报按该 Label 的更新规则合并
        String feedbackKey = event.itemId + ":" + event.action;
        MapState<String, Long> state = getFeedbackState(reqId);
        Long existing = state.get(feedbackKey);
        long merged = isNumeric(event.action)
            ? (existing == null ? 0L : existing) + event.value : 1L;
        state.put(feedbackKey, merged);
        if (event.keyFeedback) {
            ValueState<Long> kfTs = getKeyFeedbackTsState(reqId);
            if (kfTs.value() == null) {
                kfTs.update(event.eventTimeMs);
                // 关键反馈到达后延长观察时间，继续等待可能补充到达的行为
                RequestEvent req = getRequestState(reqId).value();
                if (req != null)
```

```
                getTimerService(reqId).registerEventTimeTimer(
                    event.eventTimeMs + BASE_WAIT_MS);
            }
        }
    }

    public void onTimer(long timestamp, long reqId) {
        // 样本已结束观察，强制出样并主动释放状态
        RequestEvent req = getRequestState(reqId).value();
        if (req == null) return;
        emitAndClear(reqId, out);
    }

    public void advanceWatermark(long watermarkMs) {
        List<long[]> toFire = new ArrayList<>();
        for (Map.Entry<Long, TimerService> e : timerServices.entrySet()) {
            Long ts;
            while ((ts = e.getValue().pollNextTimer()) != null && ts <= watermarkMs)
                toFire.add(new long[]{e.getKey(), ts});
        }

        for (long[] pair : toFire) onTimer(pair[1], pair[0]);
    }
}
```

上述代码将同一 Request ID 的状态和定时器限制在同一个 Flink Key 范围内，并随 Checkpoint 一起持久化，使任务故障恢复时能够直接恢复之前的等待状态，而不需要依赖 Kafka 重新投递消息来重新构建状态。对于 Redis、数据库等外部系统，Flink 任务需要避免同一请求被重复写入。例如，可以使用 Request ID 或事件 ID 作为唯一的标识，每次写入时覆盖相同请求的旧记录；如果写入失败，则保留对应的待处理记录，待任务恢复后再次处理，从而避免重试过程中产生重复数据。

与 Java Consumer 的“把消息重新放回队列”方案相比，Flink 更适合请求量较大、等待比例较高、用户反馈乱序明显，或者需要可靠故障恢复的样本流。它把等待状态交给 Flink 框架来托管，可以减少消息反复写回 Kafka 带来的读写压力；与此同时，样本拼接服务也需要承担 State Backend、Checkpoint 存储、Watermark 配置和作业运维带来的额外复杂度。对于短期验证或低吞吐业务，这些运维成本未必值得承担，Kafka 重新投递消息的方案仍然可以作为合理的起点。无论最终选择哪一种实现，请求与用户反馈的训练协议、缺失原因字段等指标都应该使用同一套定义。否则，仅仅因为切换了数据后端，模型就可能在不知情的情况下使用定义不同的训练样本。

当然，Flink 框架也不一定需要同时消费请求和用户反馈的两条 Kafka 样本流。在已经部署反馈标签消费者和 Redis 存储的系统中，用户反馈仍然可以先由反馈标签消费者写入 Redis，而 Flink 作业只消费请求特征的 Kafka 流，并按照 Request ID 保存等待中的请求状态。这里不能把 Redis 写入当作一个会自动触发 Flink 的事件，因为 Redis 不会推进 Flink 的 Watermark，也不会在用户反馈写入后主动唤醒作业。因此，在请求到达后，Flink 作业应该先登记请求并注册处理时间定时器。然后在每次定时器触发时，Flink 作业再根据 Request ID 和候选 Item ID 从 Redis 批量读取用户反馈的 Label。代码16.8给出了这一方案的核心流程。

代码 16.8: Flink 消费请求流并轮询 Redis Label

```
// 对应文件：FlinkSampleJoiner.java (RedisLabelJoinFunction)
public static class RedisLabelJoinFunction {
    /** [MOCK] 内存 Redis 替代真实 Jedis 连接 */
```

```
    private final Map<String, Long> redisStore;
    public RedisLabelJoinFunction(Map<String, Long> redisStore) {
        this.redisStore = redisStore;
    }

    /**
     * 对输出样本补充 Redis 中的反馈标签。
     * 工业落地时对应 RichMapFunction.map() 或 AsyncFunction.asyncInvoke()。
     * 一次读取本次请求所有候选的 Label，避免按 Item ID 逐条访问 Redis。
     */
    public JoinedSample enrich(JoinedSample sample) {
        Map<Long, Map<String, Long>> enriched = new HashMap<>(sample.labels);
        for (long itemId : enriched.keySet()) {
            // 读取发生型反馈（click / like / share）
            for (String action : List.of("click", "like", "share")) {
                String key = "fb_occ_" + sample.requestId + "_" + itemId + "_" + action;
                Long val = redisStore.get(key);
                if (val != null && val > 0) enriched.get(itemId).put(action, val);
            }

            // 读取数值型反馈（watch_time）
            for (String action : List.of("watch_time")) {
                String key = "fb_amt_" + sample.requestId + "_" + itemId + "_" + action;
                Long val = redisStore.get(key);
                if (val != null && val > 0)
                    enriched.get(itemId).merge(action, val, Long::sum);
            }
        }

        return new JoinedSample(
            sample.requestId, sample.sessionId, sample.scene, sample.requestTimeMs,
            enriched, sample.hasKeyFeedback);
    }
}
```

上述代码展示了 Flink 在处理请求样本时读取 Redis 中已有的用户反馈。代码按照 Request ID 和 Item ID 找到对应记录，并分别读取 click、like、share 等发生型反馈以及 watch_time 等数值型反馈，再将结果补充到样本的 Label 中。其中，发生型反馈可以直接使用 Redis 中的记录，数值型反馈则按照既定规则进行合并。

由于 Redis 不会主动通知 Flink 新反馈已经到达，Flink 需要在等待窗口内定期检查 Redis，或者额外接入反馈通知 Topic 来提前触发检查。同时，Redis 的 TTL 和 Flink 的等待状态需要相互匹配，确保任务重启或处理延迟时反馈仍然能够被读取。样本输出时也应根据 Request ID 和 Item ID 等信息避免同一请求被重复写入。

16.1.2 全链路级联样本流服务

普通任务样本流通常只保留最终曝光的物品，因为只有这些物品真正进入了用户视野，才能获得点击、播放、观看时长等真实反馈。但在级联推荐系统中，最终曝光并不是候选物品唯一值得记录的信息。一次请求中的候选物品通常会依次经过召回、粗排、精排和重排等多个阶段，大量物品虽然没有进入最终曝光列表，却已经参与过某一阶段的排序或在阶段出口处被过滤掉。如果训练任务需要分析召回覆盖率、建模粗排与精排完整的

候选数据分布，或者希望让推荐链路中不同排序阶段共享和对齐排序信息，那么就需要知道候选物品在推荐链路中的完整去向。

这类全链路级联样本流在普通样本流的基础上增加了各阶段的信息。它不重新实现用户反馈拼接，而是在上游推荐服务生成请求样本时，将部分未曝光候选及其阶段信息一并写入请求日志。这样，最终曝光物品继续沿用普通样本流的方式拼接真实用户反馈，未曝光的候选物品则携带其经过的阶段、排序位置、模型分数以及各个阶段去留等信息，供下游训练和分析任务使用。一个典型的级联推荐流程可以表示为：

$$Recall \rightarrow PreRank \rightarrow Rank \rightarrow ReRank \rightarrow Exposure \tag{16.4}$$

其中，最终曝光物品仍然按照普通样本流的逻辑读取真实用户反馈；而精排、粗排等阶段产生的未曝光候选并没有真正展示给用户，因此不能直接将点击、点赞或观看时长设置为 0。例如，一个物品在精排阶段被过滤，并不意味着用户看到该物品后没有点击，而是用户根本没有获得曝光机会。如果将这种样本直接作为点击负样本，模型学习到的实际上是历史排序策略造成的过滤偏差，而不是用户对物品本身的真实偏好。因此，未曝光候选应该记录阶段去留、排序位置和模型分数等链路信息，而用户行为 Label 仍然只针对实际曝光物品进行拼接。

需要特别说明的是，未曝光候选物品的分层采样并不是本服务负责的处理步骤，而通常在上游推荐服务中完成。一次请求的候选集合可能从数千甚至更多逐级缩减到最终曝光的数十个物品。如果将各阶段的全部候选写入请求日志，不仅会增加在线服务向 Kafka 输出的数据量，也会进一步扩大下游样本存储和训练数据的规模。因此，上游推荐服务通常会根据候选所处的推荐阶段进行分层采样，并将采样后的候选及相关元数据随请求日志一起输出。

例如，在精排出口可以完整保留 Top K 候选，因为这些候选是精排模型直接参与排序的主要对象；对于精排后未曝光的候选，可以额外随机采样少量物品，以补充排序靠后候选的训练数据。对于通过粗排但未进入精排的候选，也可以保留少量样本，以覆盖粗排阶段被过滤的候选。如果数据带宽以及存储资源充足，还可以继续从召回后未进入粗排的候选中进行少量采样。不过，召回阶段的候选规模通常最大，部分候选的特征也可能不完整，因此这一层通常作为可选的采样范围。

为了保证这种上游采样结果能够被下游正确使用，推荐在线链路在生成请求日志时需要同时记录候选所处的阶段、阶段内排序位置、阶段内对应分数以及采样原因等信息。例如，`stage`、`stageRank` 和 `stageScore` 分别表示候选所处阶段、阶段内排序位置和对应的模型分数；`sampleReason` 用于区分最终曝光、精排 Top K、精排尾部采样、粗排淘汰采样以及召回淘汰采样；`sampleProbability` 则记录该候选在对应分层中的采样概率。尤其是随机采样场景，如果只保留采样后的候选而不记录采样概率，下游只能看到经过采样策略改变后的候选分布，而无法在训练或离线分析时对原始候选分布进行修正。

代码16.9给出了上游推荐在线服务中的分层采样示例。这里的采样发生在推荐服务内部，而不是下游样本流服务中。推荐服务在各阶段已经获得对应的 Item ID 列表，只需要根据预先配置的采样 Quota，从未进入下一阶段的候选中抽取对应数量的候选物品，并将采样的结果伴随请求日志一同写入 Kafka Topic 即可。最终曝光物品的列表通常全量保留下来，精排 Top$K$ 的物品也可以全量保留，而精排尾部、粗排淘汰以及召回淘汰候选则分别按照配置的数量进行适当的采样。

代码 16.9: 推荐服务中保留级联各阶段候选的采样

```
// 对应文件：SampleFlowService.java (CascadeSampleService.sampleCascadeCandidates)
public static class CascadeSampleService {
    private final FeedbackStore feedbackStore;
    public CascadeSampleService(FeedbackStore feedbackStore) {
        this.feedbackStore = feedbackStore;
    }

    /**
     * 从完整候选列表中按阶段配额采样未曝光物品，
```

```
     * 同时全量保留所有最终曝光物品（可以获取真实用户反馈）。
     * quotaByStage 控制精排尾部、粗排淘汰、召回淘汰各阶段的采样上限。
     */
    public List<StageCandidate> sampleCascadeCandidates(
            List<StageCandidate> allCandidates, CascadeTrace trace,
            Map<String, Integer> quotaByStage) {
        Set<Long> exposedIds = trace.exposedItems().stream()
            .map(RankedItem::id).collect(Collectors.toSet());
        List<StageCandidate> result = new ArrayList<>();
        // 最终曝光物品全量保留
        for (StageCandidate c : allCandidates) {
            if (exposedIds.contains(c.itemId())) result.add(c);
        }

        // 未曝光候选按阶段配额随机采样
        Map<String, List<StageCandidate>> byStage = new LinkedHashMap<>();
        for (StageCandidate c : allCandidates) {
            if (!exposedIds.contains(c.itemId()))
                byStage.computeIfAbsent(c.stage(), k -> new ArrayList<>()).add(c);
        }

        Random rng = new Random(42);
        for (Map.Entry<String, Integer> entry : quotaByStage.entrySet()) {
            List<StageCandidate> pool = byStage.getOrDefault(entry.getKey(), List.of());
            int quota = Math.min(entry.getValue(), pool.size());
            Collections.shuffle(pool, rng);
            result.addAll(pool.subList(0, quota));
        }

        return result;
    }
}
```

上述代码在完成多阶段候选采样后，将候选 Item ID 与其阶段信息写入请求日志，作为后续阶段监督与真实反馈拼接的共同依据：

$$Candidate = (ItemID, Stage, StageRank, StageScore, SampleReason, SampleProbability) \tag{16.5}$$

其中，候选记录依次保存其所属阶段、阶段内排序位置与分数、采样原因和采样概率；最后一项供需要进行采样校正的训练任务使用。

下游保留级联候选的样本流不再重复进行候选采样，而是直接复用普通样本流的处理过程。在请求日志到达之后，首先根据其中的候选 Item ID 构造待拼接样本；行为反馈仍然由前述反馈消费者写入短期存储，并按照相同的等待窗口等待延迟反馈。在需要查询用户行为时，只针对最终曝光物品读取对应的反馈记录，而未曝光候选则不需要额外构造用户行为反馈 Label。

代码 16.10: 全链路级联候选的统一样本组装

```
// 对应文件：NormalSampleFlow.java（CascadeSampleJoinConsumer）
public class CascadeSampleJoinConsumer {
    private final FeedbackStore feedbackStore;
```

```
    private final MockKafka kafka;

    public void consume(FeatureReadyRequest request) {
        // 无级联候选时跳过，不干扰普通样本流的输出
        if (request.cascadeCandidates.isEmpty()) return;

        if (feedbackStore.shouldWait(request, nowMs, replayCount)) {
            kafka.send("sample_join_waiting", request);
            return;
        }

        joinAndEmit(request);
    }

    private void joinAndEmit(FeatureReadyRequest request) {
        //   曝光 item: 拼接真实用户反馈
        Map<Long, Map<String, Long>> exposedLabels = new HashMap<>();
        for (long itemId : request.getItemIds()) {
            Set<String> actions = feedbackStore.getActions(
                    request.getRequestId(), itemId);
            Map<String, Long> labelMap = new HashMap<>();
            for (String action : actions) {
                labelMap.put(action, feedbackStore.getValue(
                        request.getRequestId(), itemId, action));
            }

            exposedLabels.put(itemId, labelMap);
        }

        if (exposedLabels.values().stream().allMatch(Map::isEmpty)) return;

        //   未曝光候选: 只记录阶段信息，不读反馈
        List<CandidateRecord> candidates = new ArrayList<>();
        for (EncodedStagedItem c : request.cascadeCandidates) {
            candidates.add(new CandidateRecord(
                    c.getItemId(), c.getStage(), c.getRank(), c.getScore()));
        }

        kafka.send("cascade_training_sample_stream",
                new CascadeTrainingBatch(request.getRequestId(),
                        request.getScene(),
                        request.getRequestTimeMs() * 1000L,
                        exposedLabels, candidates));
        feedbackStore.clear(request.getRequestId(), request.getItemIds());
    }
}
```

上述代码同时写入候选物品所处的推荐阶段以及物品曝光后的用户反馈，使同一批样本可以支撑阶段分析与行为训练。这里必须区分“阶段监督”和“用户反馈”两个概念。例如，精排 Top$K$ 候选可以作为训练中优先保留

的样本，但这不表示用户点击了这些物品；同样，粗排淘汰的候选物品可以作为“未进入精排”的阶段负例，但也不能因此将其标记为用户点击的负例。是否进入下一阶段描述的是推荐系统自己的决策结果；而点击、观看时长等 Label 描述的则是用户在实际曝光后的行为，两者不能混为一谈。

在这种设计下，下游训练任务可以从同一份包含级联候选的样本中按需选择数据。例如，训练最终点击或时长模型时，只读取实际曝光的候选物品及其真实反馈；训练粗排与精排对齐模型时，可以使用精排 Top K 以及粗排阶段淘汰的候选构造阶段监督样本；分析召回覆盖率和各阶段淘汰情况时，则可以进一步使用召回阶段淘汰的候选。由于候选物品已经在上游推荐服务中完成采样，下游样本流无需获取完整候选池，也不需要再次进行随机采样。

因此，这类样本流并不改变普通样本流的反馈拼接方式，而是在请求产生时，由推荐服务额外保留一部分有价值的未曝光候选，并将其与最终曝光物品放在同一份请求记录中。曝光物品仍按照普通任务样本流等待并拼接 Label；未曝光物品则只记录阶段、位置、分数、去留结果和采样概率等信息。这样既可以控制样本规模和 Kafka 带宽，又能保留级联推荐过程中容易丢失的候选变化，为后续训练、排序策略分析和链路优化提供数据基础。关于这类优化的具体模型方案，将在后续第22章介绍。

16.1.3 Session 样本流服务

普通任务样本流产出的是单次请求维度的完整样本，每一刷都已经包含请求时特征、候选物品特征和对应的用户反馈 Label。对于点击率、观看时长等单刷次模型，这样的样本已经足够。但是序列模型、用户兴趣建模以及长周期效果分析还需要知道，同一个会话（Session）中的多次请求是如何连续发生的。Session 样本流服务因此在普通样本流之后消费已经完成拼接的请求样本，并按照设备 ID、用户 ID 或者业务 Session ID 将同一个 Session 内的多刷数据聚合起来。这样，训练样本不再是一组彼此独立的单刷记录，而是一段按时间顺序组织的会话序列。

通常来讲，一个 Session 可以表示为：

$$SessionID \rightarrow [(x_1, y_1), (x_2, y_2), \ldots, (x_T, y_T)] \tag{16.6}$$

其中，x_t 是第 t 刷已经完成特征拼接的请求数据，y_t 是该刷的完整 Label。Session 样本流不会重新计算点击、观看时长等用户反馈，也不重复普通样本流的等待逻辑；它只负责恢复同一 Session 内请求的顺序、去重重复请求，并在会话过长时进行截断。因此，普通样本流在输出单刷次样本时必须保留稳定的 Session ID、Request ID、请求时间和请求序号。如果缺少这些字段，即使用户反馈已经完整，后续也无法可靠地恢复会话顺序。

在具体的实现中，Session 样本流通常使用 Flink Session Window 或 Keyed State 来实现。Flink 作业首先会按照业务 Session ID 对完整请求样本分区，再根据 Session gap 来判断整个会话是否结束。这里的 Session gap 主要用于处理用户异常中断或者 Session ID 缺失等情况，不能简单地把设备 ID 加固定 gap 当作会话划分规则。否则，用户短暂退出后重新返回时，两段本来应该独立的会话可能会被错误地合并到一起。在窗口关闭之后，Flink 作业会按照事件时间和请求序号对本次会话中的请求重新排序，保留会话级公共特征以及每一刷的完整样本列表，最后写入 Session 训练的 Kafka topic。代码16.11给出了这一过程的简化实现，如下所示。

代码 16.11: 基于 Flink 的 Session 样本组装

```
// 对应文件：FlinkSampleJoiner.java (SessionAssemblyFunction / FlinkSessionJoinJob)

//   Session 组装函数：KeyedProcessFunction<String, JoinedSample, SessionSample>
public static class SessionAssemblyFunction {
    static final long SESSION_GAP_MS = 30 * 60 * 1_000L; // 30 分钟 gap
    static final int MAX_SESSION_LEN = 200;

    // [STATE] ListState<JoinedSample>：缓冲当前会话的所有已拼接样本
    private final Map<String, List<JoinedSample>> sessionBuffer = new HashMap<>();
    // [STATE] ValueState<Long>：当前活跃 gap timer 时间戳
```

```
    private final Map<String, Long> activeTimer = new HashMap<>();

    /**
     * 每条 JoinedSample 到达时追加到缓冲区，并滑动重置 gap 定时器。
     * 工业落地对应：processElement(JoinedSample value, Context ctx, Collector<SessionSample> out)
     */
    public void processElement(JoinedSample value, long eventTimeMs) {
        String sid = value.sessionId;
        List<JoinedSample> buf = sessionBuffer.computeIfAbsent(sid, k -> new ArrayList<>());
        if (buf.size() < MAX_SESSION_LEN) {
            buf.add(value);
        }

        // 删除旧 gap timer，注册新 gap timer（每条消息均滑动延迟）
        Long old = activeTimer.get(sid);
        if (old != null) timerService.deleteEventTimeTimer(old);
        long deadline = eventTimeMs + SESSION_GAP_MS;
        timerService.registerEventTimeTimer(deadline);
        activeTimer.put(sid, deadline);
    }

    /**
     * gap 定时器到期：对缓冲区排序、去重，输出 SessionSample 到 Kafka 输出 topic。
     * 工业落地对应：onTimer(long timestamp, OnTimerContext ctx, Collector<SessionSample> out)
     */
    public void onTimer(long timestamp, String sessionId) {
        if (!Long.valueOf(timestamp).equals(activeTimer.get(sessionId))) return;
        List<JoinedSample> buf = sessionBuffer.getOrDefault(sessionId, List.of());
        Map<Long, JoinedSample> dedup = new LinkedHashMap<>();
        buf.stream()
           .sorted(Comparator.comparingLong(s -> s.requestTimeMs))
           .forEach(s -> dedup.put(s.requestId, s));
        out.collect(new SessionSample(sessionId, new ArrayList<>(dedup.values())));
        sessionBuffer.remove(sessionId);
        activeTimer.remove(sessionId);
    }
}

//   Flink 作业入口：Kafka 输入流 → keyBy(sessionId) → 组装 → Kafka 输出流
// 工业落地等价（伪代码）：
//  env.fromSource(KafkaSource.<JoinedSample>builder()
//        .setTopics("joined_sample_stream").build(),
//      WatermarkStrategy.forBoundedOutOfOrderness(Duration.ofSeconds(5)),
//      "joined-sample-source")
//     .keyBy(s -> s.sessionId)
//     .process(new SessionAssemblyFunction())
//     .sinkTo(KafkaSink.<SessionSample>builder()
//        .setBootstrapServers(brokers)
//        .setRecordSerializer(...)
```

```
//        .build());
//  env.execute("session-assembly");
```

上述代码以 Session ID 为 Key，将同一 Session 内的请求样本暂存在 Flink 状态中，并通过 Session gap 定时器判断会话是否结束。每当新的请求到达时，都会重新计算 gap 时间。只有在连续超过设定时间没有新请求后，定时器才会触发，并对当前 Session 中的样本进行排序、去重后输出。因此，相比普通样本在反馈窗口结束后即可输出，Session 样本还需要等待最后一刷请求之后的 gap 时间，样本才会最终产出。当单个 Session 包含过多请求时，还需要设置最大 Session 长度，超过限制的部分进行截断，并记录相应的截断信息。

16.1.4 非实时样本流服务

非实时样本流服务面向时效性要求不高，但需要回溯重建或等待完整历史反馈的场景。例如，广告归因、长周期转化、LTV、冷启动历史回填以及电商下单等存在延迟的用户行为，可能在曝光数天后才能完整到达。对于这类任务，如果仍然在实时链路中设置很长的等待窗口，不仅会增加 Kafka 与 Redis 的状态压力，也无法保证最终拿到完整 Label。因此，这种场景更适合在离线 Hive 表中等待数据自然沉淀，然后再按照 Request ID 和 Item ID 将请求特征与用户行为关联起来。

典型的实现是在 Hive 中将请求特征表与行为 Label 表做 Inner Join，即：

$$FeatureTable \bowtie_{request_id,item_id} LabelTable \tag{16.7}$$

相比起编写 Java Kafka Consumer 或者 Flink 作业，Hive SQL 的样本拼接代码更短，按日期分区回刷、字段试验和全量重建也更直接。代码16.12给出了一个简化版样本拼接 SQL。代码先在 Label 表中按 Request ID 和 Item ID 选出事件时间最新的一条记录，再与请求特征表进行关联。此外，实际任务还需要明确日期分区、Label 去重规则，以及同一个 Request ID 出现多个特征版本或者 Label 上报时应当选择哪一个版本等细节问题；这样才能保证样本拼接之后得到正确且高质量的训练样本。

代码 16.12: 基于 Hive SQL 的非实时样本拼接

```sql
-- 对应文件：sample_join.sql
WITH latest_label AS (
    SELECT request_id, item_id, click, watch_time_ms, event_time_ms
    FROM (
        SELECT request_id, item_id, click, watch_time_ms, event_time_ms,
            ROW_NUMBER() OVER (
                PARTITION BY request_id, item_id
                ORDER BY event_time_ms DESC) AS rn
        FROM dwd_item_feedback
        WHERE dt BETWEEN '${start_dt}' AND '${end_dt}'
    ) t
    WHERE rn = 1
)
INSERT OVERWRITE TABLE dws_training_sample PARTITION (dt='${biz_dt}')
SELECT f.request_id, f.item_id, f.user_features, f.context_features,
    f.item_features, f.request_time_ms,
    l.click, l.watch_time_ms, l.event_time_ms AS label_time_ms
FROM dwd_request_features f
JOIN latest_label l
    ON f.request_id = l.request_id AND f.item_id = l.item_id
WHERE f.dt = '${biz_dt}';
```

上述代码将离线 SQL 的请求、物品与完整 Label 在截止时间前关联，产出的训练样本与实时链路共享同一套实体、标签和版本语义。实时链路服务分钟级或小时级模型更新；Hive 样本流适合需要完整 Label、可重处理历史数据和复杂离线统计的训练任务。无论样本来自哪条链路，二者都必须使用同一套 Request ID、Item ID、Label 定义、版本字段和时间截点。

16.2 强化学习样本流服务

传统推荐模型通常将一次推荐请求看作一个相对独立的监督学习样本，而强化学习（Reinforcement Learning, RL）推荐更加关注用户行为在时间维度上的连续性。监督学习模型也可以输入用户最近几十次甚至几百次的行为序列。但监督学习模型与强化学习模型的区别不在于是否使用行为序列，而在于学习目标的差异。监督学习即使输入完整历史，其学习目标仍然是当前这次曝光的反馈；而强化学习则显式建模的是当前动作对未来状态与未来收益的影响。这决定了强化学习模型的训练数据除了 (s_t, a_t, r_t) 之外，还必须恢复出下一状态 s_{t+1}。也正因如此，RL 样本流不能在普通样本流上修修补补，它需要专门的数据构造链路。

在监督学习中，我们通常构造：

$$(user, item, context, label) \tag{16.8}$$

形式的样本，模型根据这类样本来学习当前状态特征与用户反馈之间的关系。而在强化学习中，推荐系统还需要进一步考虑当前推荐动作不仅会产生即时反馈，还可能改变用户下一时刻的状态这一事实。因此，一个最基本的 RL 样本通常表示为：

$$(s_t, a_t, r_t, s_{t+1}) \tag{16.9}$$

其中 s_t 表示时间步 t 的状态，a_t 表示推荐系统采取的动作，r_t 表示用户在当前时间步产生的奖励，s_{t+1} 则表示用户接受该动作之后进入的下一状态。RL 样本流必须保存当前请求、该请求的反馈和下一次请求的状态，才能形成可训练的状态转移。只保存 (s_t, a_t, r_t) 时，训练数据不知道用户接受当前动作后进入了什么状态。

笔记 对于 RL 推荐而言，仅仅保存当前请求和当前 Label 是不够的。系统还需要进一步回答一个问题：

“当前这次推荐之后，用户下一次进入了什么状态？”

这使得 RL 样本流从传统的“请求—Label”的信息关联升级为“请求—反馈—下一次请求”的时序关联。

在工业级推荐系统中，强化学习离线训练通常涉及两类经典方法，即**时序差分学习**（Temporal Difference Learning, TD Learning）和**蒙特卡洛方法**（Monte Carlo Method）。其中，TD Learning 利用相邻状态之间的时序关系进行学习，只需要一步或有限步的未来信息即可更新价值函数，典型方法包括 SARSA、Q-learning 及其深度强化学习扩展。蒙特卡洛方法则利用完整交互轨迹中的实际回报进行学习，通常需要等待一个完整的 Episode 或行为序列结束后，才能获得较为完整的累计回报。虽然两类方法的训练方式不同，但都要求离线数据尽可能保留用户行为的时间结构。对于需要完整行为序列的蒙特卡洛方法，可以进一步利用前一节介绍的 Session 样本流服务，将同一会话中的多次请求组织成连续的训练序列，为后续的累计回报计算和模型训练提供数据基础。

在这一节中，我们主要以单步 TD Learning 为例来讲解强化学习样本流的构造。由于 TD Learning 只需要考虑下一个状态，我们只需要构造：

$$(s_t, a_t, r_t, s_{t+1}) \tag{16.10}$$

即可进行价值函数更新。在具体实现时，我们可以在传统样本流的基础上增加用户请求之间的时序关联。假设用户在时间 t 产生请求 ReqID_t，下一次请求为 ReqID_{t+1}，那么系统可以建立：

$$\text{ReqID}_t \rightarrow \text{ReqID}_{t+1} \tag{16.11}$$

的映射关系。在后续处理 ReqID_t 的样本时，我们就可以根据这个映射找到下一请求，并进一步获得下一状态 s_{t+1}，从而将传统的：

$$(s_t, a_t, r_t) \tag{16.12}$$

扩展为：

$$(s_t, a_t, r_t, s_{t+1}) \tag{16.13}$$

如果在合理的等待时间内没有找到下一请求，则可以根据业务规则将当前样本标记为终止样本，并构造：

$$(s_t, a_t, r_t, \text{terminal}) \tag{16.14}$$

需要注意的是，“没有找到下一请求”并不一定意味着用户在现实世界中真正终止了交互。它可能表示用户离开了推荐场景，也可能表示日志延迟、数据缺失或者当前数据处理窗口已经结束。因此，在实际系统中最好区分真实 Terminal 和观测窗口结束两种情况，避免把数据缺失错误地当作用户行为终止。

16.2.1 基于 Redis 的 RL 样本拼接

该 RL 样本流来自一个典型的短视频 Feed 流推荐场景。用户每次下拉刷新后，系统默认返回一刷短视频内容，同时推荐系统需要针对当前请求额外做一个二值决策，即**是否在这一刷中插入直播内容**。直播和短视频并非两套独立推荐系统产生的反馈，而是这个决策在不同动作下的奖励来源。动作空间可以表示为：

$$a_t \in \{\text{插入直播}, \text{不插入直播}\} \tag{16.15}$$

之所以需要针对这一决策单独构建 RL 模型，是因为插入直播与否产生的影响并不局限于当前一刷。例如，插入用户感兴趣的直播后，用户可能在直播间停留较长时间，随后继续浏览短视频；而插入不合适的直播，则可能导致用户快速划走，甚至提前结束当前会话。也就是说，当前动作会改变用户后续的行为轨迹和状态。这种跨请求的长期影响，很难仅通过传统监督学习中的 $(user, item, context, label)$ 样本进行刻画，因此需要进一步引入状态转移 s_{t+1} 和完整的交互轨迹。

业务场景决定了样本的奖励结构。当决策为插入直播，即 $a_t =$ 插入 时，当前请求中会包含直播内容，对应的奖励 r_t 主要来自直播侧反馈，例如是否进入直播间、是否长播播放、观看时长以及是否打赏等；而当决策为不插入直播，即 $a_t =$ 不插入 时，当前请求全部由短视频组成，奖励则主要来自短视频侧反馈，例如视频播放次数和视频播放时长。因此，同一条 RL 样本流中同时存在 live 和 photo 两类行为枚举以及对应的反馈 Label，是同一个 RL 决策在不同动作分支下产生不同奖励来源的自然结果。代码中的 `live` 和 `photo` 前缀，以及针对两类反馈进行的质量过滤，都由这一业务结构决定。

在此基础上，RL 样本拼接相较于传统监督学习样本流新增的核心工作，并不是简单增加几个字段，而是恢复不同请求之间的状态转移关系。一种直接的工程实现，是在已有 Kafka 与 Redis 样本流的基础上增加请求状态和曝光序列的维护。当前请求到达时，链路先保存经过裁剪的请求状态；当真实曝光行为到达之后，另一条支路按照设备维护最近的 Request ID 序列。后续样本消费者即可根据曝光序列定位当前请求的下一次真实曝光请求，并读取对应的状态。

假设设备的真实曝光序列中依次出现 ReqID_{t-1} 和 ReqID_t，那么相邻位置自然隐含了：

$$\text{ReqID}_{t-1} \rightarrow \text{ReqID}_t \tag{16.16}$$

的时序关系。系统不需要为每一对请求额外维护映射关系，只需定位当前 Request ID，并读取序列中的下一个元素即可。与此同时，当前请求的状态以

$$\text{ReqID}_t \rightarrow s_t \tag{16.17}$$

的形式暂存在 Redis 中。状态通常包含用户画像、请求上下文、历史行为以及候选物品特征等信息，数据量远大于普通的标量字段，因此写入 Redis 前通常需要进行字段裁剪、Protobuf 序列化以及格式压缩。在处理 ReqID_t 的基础样本时，Kafka 消费者首先在真实曝光序列中查找 ReqID_{t+1}，再读取对应的状态 s_{t+1}，最终构造：

$$(s_t, a_t, r_t, s_{t+1}) \tag{16.18}$$

的样本。如果下一次真实曝光请求或对应状态尚未到达，则当前样本进入等待流程。在超过配置的等待窗口之

后如果仍然无法完成拼接，则系统会输出 `has_join_next=0` 的截断样本。需要注意的是，这里“没有下一刷”并不一定意味着用户主动结束了会话，也可能是由于日志延迟、状态过期或链路数据丢失导致的。因此，实际系统通常还需要记录具体的截断原因，以便进行后续的训练数据质量分析。

由于 RL 状态通常包含大量特征，直接将原始 Protobuf 或其他序列化数据写入 Redis 会产生较大的内存压力，因此工程上通常会在写入 Redis 前进行压缩。常见方案包括 Snappy、LZ4、Gzip 和 Zstandard。其中，Snappy 和 LZ4 更强调压缩与解压速度，适合高频实时数据链路；Gzip 通常具有更高的压缩率，但 CPU 开销相对较大；Zstandard 则可以通过不同压缩级别在压缩率和计算开销之间进行权衡。在推荐系统的实时样本拼接场景中，压缩算法不能只比较最终压缩率，还需要综合考虑 CPU 消耗、压缩与解压延迟、内存带宽以及 Redis 存储成本。对于需要频繁读写的中间状态，通常更倾向于选择 Snappy 或 LZ4 等低延迟方案。

由于前面的内容已经介绍了 Redis 方案的基本机制，图16.2则进一步汇总了服务之间的关系、Kafka 数据流以及 Redis 中间状态。Redis 在整个 RL 样本拼接链路中主要承担如下两类职责：

- 一类是暂存 live 和 photo 用户行为，由反馈标签消费者写入，再由 Label 拼接消费者读取。
- 另一类是保存经过裁剪的请求状态、会话信息以及真实曝光请求序列。

围绕这两类状态，整个 RL 样本流形成了特征预处理、Label 拼接和状态转移拼接的三级流水线，依次完成请求日志规范化、(s_t, a_t, r_t) 基础样本构造以及 s_{t+1} 状态拼接。该方案可以复用已有的 Kafka 消费者和 Redis 缓存设施，但状态过期、读写数据是否对应以及异常重试等问题仍需要在代码实现中进行明确处理。

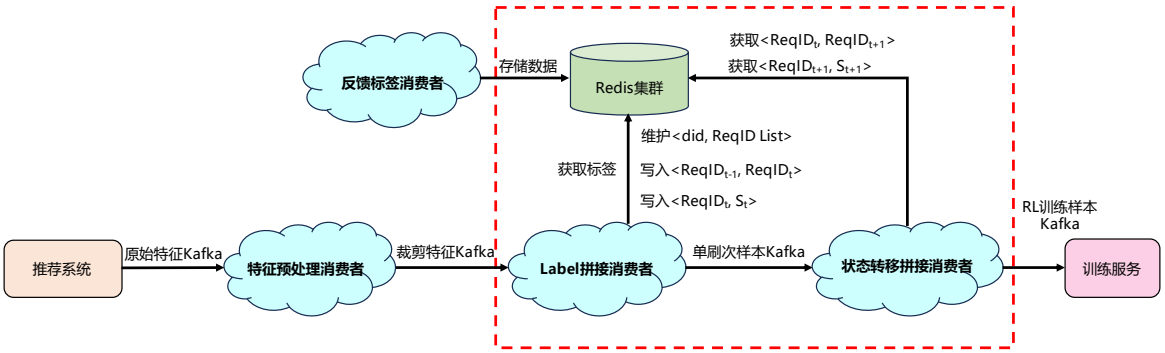


图 16.2: 基于 TD Learning 的工业界强化学习推荐离线架构。

具体来说，特征预处理消费者首先按照流量类型以及请求类型过滤无效请求或者不参与训练的流量，再根据设备 ID 与 Request ID 进行去重，并通过白名单裁剪用户侧特征以及物品侧特征。同时，预处理阶段还会记录 Session ID、请求时间、候选物品 ID 以及决策动作等元数据，为后续的反馈归因和会话校验提供基础信息。Label 拼接消费者则负责等待用户反馈，从 Redis 中读取当前请求对应的 live 和 photo 行为，构造 (s_t, a_t, r_t) 基础样本，并将裁剪后的请求状态写入状态存储。随后，状态转移拼接消费者继续处理基础样本，根据设备的真实曝光序列来定位下一刷的请求，读取对应状态并将 s_{t+1} 拼入当前样本。与此同时，反馈标签消费者作为独立的写入链路会持续接收客户端和服务端反馈，并将其写入 Redis，供后续 Label 拼接使用。

16.2.1.1 用户反馈的存储与读取

用户反馈行为与请求特征通常来自两条相互独立的日志链路，因此样本拼接首先需要解决的就是如何将反馈准确关联到对应请求。最直接的方案是将请求日志和行为日志同时写入 Kafka，再由同一个作业进行流式拼接。但在实际系统中，行为来源往往同时包括客户端、服务端以及多个业务方，行为类型、数据格式和到达时间也会持续变化。如果将这些变化全部暴露给样本拼接作业，那么任意一侧的协议调整都可能迫使样本流同步修改，增加系统维护成本。

这里使用对已有链路影响更小的处理方式。反馈标签消费者在用户行为发生后，以 Request ID 和 Item ID 作为索引，将不同来源的行为写入 Redis。而 Label 拼接消费者在进入等待窗口之后，再按照相同的索引读取反馈。在这个过程中，Redis 处于请求日志与行为日志之间，写入方将异构的用户行为转换为统一的反馈状态，读取方

则按照统一协议读取并解释这些状态。每当新增一种反馈类型时，通常只需要在反馈枚举和写入逻辑中增加相应定义，而不需要修改整套流式拼接链路。

在用户反馈写入 Redis 时，不同类型的行为通常需要根据自身语义选择合适的存储方式。点击、进入直播间、快速划走等发生型行为只需要记录是否发生，可以使用 Bitmap 将不同反馈映射到固定的比特位，一次读取即可获得同一物品的多种布尔型反馈，同时减少 Redis Key 的数量。观看时长、播放次数等数值型反馈需要保留累计结果，则更适合使用 INCRBY 计数器；退出方式等状态型反馈则可以直接覆盖写入。此外，还需要单独记录最近一次反馈事件的发生时间，为后续判断等待窗口提供依据。这些反馈 Key 通常设置较短的 TTL，并在样本拼接完成后由消费者主动删除。

除了前面介绍的反馈写入方式外，读取用户反馈时还需要关注 Redis 的访问开销。一次请求通常包含多个候选物品，如果逐个执行 GET 或 Bitmap 操作，就需要对每个候选分别访问 Redis，容易造成大量 Redis 请求。因此，Label 拼接消费者通常使用 Pipeline 批量读取同一请求下多个物品的反馈状态，将多个独立请求合并处理，从而减少 Redis 的访问次数。代码16.13给出了反馈标签消费者的简化实现，其中保留了枚举校验、Bitmap 置位、计数器累加和行为时间戳更新等关键逻辑，与前面介绍的不同类型反馈存储方式相对应。

代码 16.13: 反馈标签消费者写入逻辑骨架

```
// 对应文件：RLSampleFlowService.java (FeedbackLabelConsumer)
/** Kafka 消费者：消费用户行为 Action Log，将 Label 写入 Redis 中转区。 */
public class FeedbackLabelConsumer {
    private final FeedbackSinkService sinkService;
    public FeedbackLabelConsumer(FeedbackSinkService sinkService) {
        this.sinkService = sinkService;
    }

    public void consume(ActionLog actionLog) {
        long reqId = firstRequestId(actionLog.requestIds);
        if (reqId <= 0 || actionLog.itemIds.isEmpty()) return;
        for (int i = 0; i < actionLog.itemIds.size(); i++) {
            long itemId = actionLog.itemIds.get(i);
            if (FeedbackKind.isNumericAction(actionLog.actionName)) {
                long value = i < actionLog.actionValues.size()
                        ? actionLog.actionValues.get(i) : 0L;
                sinkService.recordAmountLabel(reqId, itemId, actionLog.actionName, value);
            } else {
                sinkService.recordOccurLabel(reqId, itemId, actionLog.actionName);
            }
        }
    }

    private long firstRequestId(List<String> ids) {
        if (ids == null || ids.isEmpty()) return 0L;
        try { return Long.parseLong(ids.get(0)); } catch (NumberFormatException e) { return 0L; }
    }
}

public static class FeedbackSinkService {
    private static final int LABEL_TTL_SECONDS = 7200;
```

```
    public void recordOccurLabel(long reqId, long itemId, String actionName) {
        FeedbackKind action = FeedbackKind.fromName(actionName);
        // 未知行为直接丢弃并打点，防止客户端新版本新行为枚举冲垮老样本流
        if (action == null || !action.isTrainingLabel()) {
            metrics.count("sink_unknown_action", actionName);
            return;
        }

        String key = String.format("fb_occ_%d_%d", reqId, itemId);
        redis.pipeline(pipe -> {
            pipe.setbit(key, action.bitIndex(), true); // 行为 -> 比特位
            pipe.expire(key, LABEL_TTL_SECONDS);
        });
    }

    // 数值型行为：观看时长、播放次数等，计数器累加
    public void recordAmountLabel(long reqId, long itemId, String actionName, long delta) {
        FeedbackKind action = FeedbackKind.fromName(actionName);
        if (action == null) return;
        String key = String.format("fb_amt_%d_%d_%s", reqId, itemId, actionName);
        redis.pipeline(pipe -> {
            pipe.incrby(key, delta);
            pipe.expire(key, LABEL_TTL_SECONDS);
        });
        // 每次写入刷新行为时间戳，自适应等待窗口的锚点输入
        String tsKey = String.format("fb_ts_%d", reqId);
        redis.setex(tsKey, LABEL_TTL_SECONDS, String.valueOf(System.currentTimeMillis()));
    }
}
```

上述代码在将反馈写入 Redis 前先校验动作枚举，并同步更新该请求最近一次行为的发生时间，供后续判断等待窗口是否结束。对于未知动作，需要记录并直接丢弃，避免客户端新增行为类型后，旧版本消费者无法识别而持续处理失败。同时，行为时间也不能遗漏，否则可能导致样本过早结束等待，或者因为无法判断最新反馈时间而持续等待。

代码16.14给出了从 Redis 批量读取用户反馈的具体实现。读取时通过 Pipeline 一次获取同一请求下所有候选物品的 Bitmap，再根据反馈枚举解析其中的比特位，将对应的反馈还原为行为名称列表。数值型 Label 则以 `(reqId, itemId, actionName)` 作为 Key 进行批量读取，整体处理方式与 Bitmap 类似。

代码 16.14: 用户反馈行为的批量读取与 Bitmap 解码

```
// 对应文件：RLSampleFlowService.java (BitmapLabelReader)
public class FeedbackQueryService {

    /**
     * 批量读取一次请求下所有候选物品的行为集合.
     * 返回：itemId -> 该物品发生的所有行为名列表
     */
    public Map<Long, List<String>> queryOccurLabels(long reqId, List<Long> itemIds) {
        Map<Long, List<String>> result = new HashMap<>();
```

```
    if (reqId <= 0 || itemIds.isEmpty()) {
        return result; // 非法请求不读取，直接返回空，后续按"Label缺失"处理
    }

    // Pipeline: 一次网络往返读取全部物品的 Bitmap
    Map<Long, byte[]> bitmapMap = redis.binaryPipeline(itemIds, (pipe, itemId) ->
            pipe.get(String.format("fb_occ_%d_%d", reqId, itemId).getBytes())
    );
    if (bitmapMap.isEmpty()) {
        return result; // 全部 Key 不存在：无任何行为
    }

    // Bitmap -> 行为枚举：遍历枚举而不是比特位，枚举新增行为时老样本自动被忽略
    for (Map.Entry<Long, byte[]> entry : bitmapMap.entrySet()) {
        long itemId = entry.getKey();
        byte[] bytes = entry.getValue();
        if (bytes == null) {
            continue;
        }

        BitSet bits = fromRedisBitmap(bytes);
        if (bits.isEmpty()) {
            continue;
        }

        for (FeedbackKind action : FeedbackKind.values()) {
            if (bits.get(action.bitIndex())) {
                result.computeIfAbsent(itemId, k -> new ArrayList<>()).add(action.featureName());
            }
        }
    }

    return result;
}

/** 数值型 Label 批量读取：与 Bitmap 同理，Key 维度为 (reqId, itemId, actionName) */
public Map<Long, Map<String, Long>> queryAmountLabels(long reqId, Map<Long, List<String>> ↩
    ↪ itemActionMap) {
    Map<Long, Map<String, Long>> result = new HashMap<>();
    // 展开为 (itemId, actionName) 对列表，Pipeline 批量 GET
    List<long[]> pairs = new ArrayList<>();
    List<String> actionNames = new ArrayList<>();
    itemActionMap.forEach((itemId, actions) ->
            actions.forEach(action -> {
                pairs.add(new long[]{itemId});
                actionNames.add(action);
            }));
    for (int idx = 0; idx < pairs.size(); idx++) {
        long itemId = pairs.get(idx)[0];
```

```
            String action = actionNames.get(idx);
            String key = String.format("fb_amt_%d_%d_%s", reqId, itemId, action);
            String value = redis.get(key);
            if (value != null) {
                result.computeIfAbsent(itemId, k -> new HashMap<>())
                        .put(action, Long.parseLong(value));
            }
        }

        return result;
    }

    /** Redis SETBIT 为大端比特序，与 JDK BitSet 的内存布局相反，必须显式反转 */
    private BitSet fromRedisBitmap(byte[] bytes) {
        BitSet bits = new BitSet();
        for (int i = 0; i < bytes.length * 8; i++) {
            if ((bytes[i / 8] & (1 << (7 - (i % 8)))) != 0) {
                bits.set(i);
            }
        }

        return bits;
    }
}
```

上述代码通过 Pipeline 批量读取候选物品的 Bitmap，并将 Redis 中的位信息转换为 Java 的位集合，最终得到用户的行为集合并用于 Label 拼接。一次请求通常包含数个甚至几十个候选物品，如果逐个 Key 执行 GET，就会产生大量 Redis 访问，因此示例代码使用 `binaryPipeline` 一次读取整刷候选物品的 Bitmap。

在读取字节数据后，服务还需要保证行为枚举与历史数据中的比特位始终保持一致。代码按照行为枚举逐一检查对应比特位，使新增行为可以正常兼容历史数据，同时避免枚举顺序变化后将已有行为错误地解释为其他动作。由于 Redis 的 SETBIT 与 Java `BitSet.valueOf` 在字节序上的处理方式不同，示例还显式进行了位序转换，否则可能出现“进入”被解析为“退出”等 Label 错误。这类问题通常不会直接导致程序报错，而是表现为部分 Label 的分布长期异常，因此需要结合行为分布监控及时发现。

在行为集合读取完成之后，Label 拼接消费者还需要把每个候选物品的反馈写入样本 Protobuf 中对应的 label 字段，使一条样本同时包含请求时的状态特征与曝光后的行为结果。代码16.15详细展示了这一过程。

代码 16.15: Label 拼入样本：逐物品写入 label 名与 label 值

```
// 对应文件：RLSampleFlowService.java (LabelJoinService.joinLabel)
/** 将 Redis 行为中转区读回的行为写入每个候选物品的样本结构 */
private void appendFeedbackLabels(TrainRecord.Builder builder, long reqId, List<Long> itemIds) {
    // 1. 批量读取是否发生与数值两类行为
    Map<Long, List<String>> actionMap = feedbackService.queryOccurLabels(reqId, itemIds);
    Map<Long, Map<String, Long>> amountMap = feedbackService.queryAmountLabels(reqId, actionMap);
    boolean hasAnyLabel = false;
    // 2. 逐物品写入：label 名与 label 值按下标对齐写入样本协议
    for (int i = 0; i < builder.getEntryCount(); i++) {
        long itemId = itemIds.get(i);
        List<String> actions = actionMap.getOrDefault(itemId, Collections.emptyList());
```

```
        for (String action : actions) {
            EntryBuilder entry = builder.getEntryBuilder(i);
            entry.addLabelName(action);       // label 名，如 item_click
            // 观看时长等数值型行为读计数器；只表示是否发生的行为默认值为 1
            long value = amountMap.getOrDefault(itemId, Collections.emptyMap())
                    .getOrDefault(action, 1L);
            // 时长类 Label 夹紧到物理合理区间，防御客户端脏数据
                if (action.endsWith("duration") || action.endsWith("watch_time")) {
                value = Math.max(0, Math.min(value, MAX_DURATION_MS));
            }

            entry.addLabelValue(value);
            hasAnyLabel = true;
        }
    }

    // 3. 请求级标记：该请求是否拼到过任何 label，供数据质量门控使用
    builder.addCommonAttr(toAttr("has_feedback", hasAnyLabel ? 1 : 0));
}
```

上述代码将每个候选物品对应的反馈行为写入样本中的 Label 字段，并同时保存 Label 名称和对应取值。由于样本协议中的 Label 名称与 Label 值按相同顺序对应存储，因此写入时必须保持两者的一致，否则就可能出现行为名称与数值错位的问题。这类错误通常不会在编译阶段暴露，而会表现为某些 Label 的数值分布异常。

而对于观看时长等数值型反馈，在写入样本前通常还需要限制合理的取值范围，避免客户端时间异常或重复累加产生极端值，影响训练数据的分布。代码16.15最后在公共属性中写入 `has_feedback` 字段，用于标记当前请求是否包含任何有效反馈。这样，下游既可以根据具体 Label 判断用户发生了哪些行为，也可以通过该字段快速判断当前样本是否存在反馈，后续样本过滤则可以结合这一标记与决策动作共同完成。

16.2.1.2 自适应等待窗口

在 Label 拼接消费者中，等待窗口决定了 Label 完整率与样本时效性之间的平衡。固定窗口虽然实现简单，但难以适应不同用户的反馈节奏。当等待窗口过短时，延迟到达的行为可能被误判为无反馈；而当等待窗口过长时，没有后续行为的请求又会长期占用中间状态。由于不同反馈的到达延迟差异较大，对所有用户请求都采用统一的固定等待窗口通常并不合适。

在实际的 Label 拼接消费者服务中，可以采用**自适应等待窗口**的机制。具体来说，以请求时间作为初始等待时间，当 Label 拼接消费者收到新的有效反馈后，就将最近一次行为时间更新为当前记录的时间：

$$T_{\text{last}} \leftarrow \max(T_{\text{last}}, T_{\text{action}}) \tag{16.19}$$

在此基础上，根据不同用户行为的特点设置不同的延长等待时间。例如，进入直播间、开始长时间播放等行为通常需要更长的观察时间，而后续的停留、退出或滑动等行为到达后，还可以继续延长等待时间。这样，反馈持续产生的请求可以获得更充分的观察时间，而始终没有新反馈的请求则可以在达到最大等待时间后及时结束。代码16.16给出了等待窗口判定的简化实现，如下所示。

代码 16.16: 自适应等待窗口判定

```
// 对应文件：RLSampleFlowService.java (AdaptiveWaitWindow)
/**
* 判断请求是否仍处于 Label 拼接等待窗口内。
* 根据请求时间和已产生的用户行为动态调整等待时间。
```

```
 */
private boolean isInWaitWindow(long requestId, long requestTimeMs) {
    long nowMs = System.currentTimeMillis();

    // 如果已经产生关键行为，则从该行为发生时间开始计算等待时间；
    // 否则仍以请求时间作为计算起点。
    long keyActionTs = feedbackService.getActionTimestamp(requestId, "key_action");
    long baseTs = keyActionTs > 0 ? keyActionTs : requestTimeMs;
    // 未产生关键行为时使用较长的最大等待时间；
    // 产生关键行为后使用基础等待时间。
    long windowMs = seconds(keyActionTs > 0 ? BASE_DELAY : BASE_MAX_DELAY);
    if (nowMs - baseTs < windowMs) {
        return true; // 仍在窗口内，继续等待
    }

    // 若关键行为之后产生了间内行为(停留/滑动)，继续延长窗口
    long innerActionTs = feedbackService.getActionTimestamp(requestId, "inner_action");
    if (innerActionTs > 0 && nowMs - innerActionTs < seconds(ACTION_EXTEND_DELAY)) {
        return true;
    }

    return false;    // 超出所有窗口，允许出样
}
```

上述代码根据请求时间和关键行为时间判断是否需要继续等待，为每个请求设置一个有限的观察期限。即使关键行为一直没有出现，也不能让请求无限期等待，而需要在达到最大等待时间后结束处理，避免没有反馈的请求长期留在链路中。在实际应用中，基础等待时间、最大等待时间以及不同反馈对应的延长时间都可以动态配置，并根据 Label 的实际到达情况持续调整。

代码16.16中的窗口判定只负责判断当前请求是否需要继续等待，而 Label 拼接消费者本身不能一直占用处理线程等待新的反馈。因此，当等待窗口尚未结束时，可以将当前消息重新写回输入 Topic，等下一次消费时再次检查反馈状态。当等待时间结束，或者已经获得所需的反馈后，再继续完成样本拼接。代码16.17展示了这种延迟再次投递的基本实现。

代码 16.17: Label 拼接消费者的延迟再次投递骨架

```
// 对应文件：RLSampleFlowService.java (LabelJoinConsumer)
public static class LabelJoinConsumer {
    private final WaitWindowService waitWindowService;
    private final FeedbackQueryService feedbackService;
    private final MockKafka kafka;
    public void consume(TrainRecord message) {
        long reqId = message.getRequestId();
        long requestMs = message.getTimestamp();
        if (waitWindowService.isInWaitWindow(reqId, requestMs)) {
            // 仍在等待窗口内：记录再次投递原因，然后写回自身 topic
            message.setReplayFlag("wait_window");
            kafka.send(kafka.selfTopic(), message);
            sleepUninterruptibly(100, TimeUnit.MILLISECONDS); // 轻微限流
            return;
```

```
        }

        // 超出窗口：从行为中转区读取行为并写入样本
        TrainRecordBuilder builder = new TrainRecordBuilder().copyFrom(message);
        appendFeedbackLabels(builder, reqId, message.getItemIds(), feedbackService);
        // 拼接完成后主动清理该请求的行为 Key；异常遗留数据由 TTL 到期后清理
        feedbackService.clearOccurLabels(reqId, message.getItemIds());
        kafka.send("rl_joint_sample", builder.build());
    }
}
```

上述代码在等待窗口内将请求重新写回 Topic，等待后续反馈到达并再次检查。在实际生产环境中，拼接服务还需要限制重新投递的频率和次数，避免同一请求反复进入队列，影响正常消息的处理。每次重新投递时可以记录相应的原因和次数；当达到重投上限后，则根据当前已有的反馈状态结束等待，避免异常请求长期循环。

16.2.1.3 状态转移拼接

在 Label 拼接完成后，样本已经具备 (s_t, a_t, r_t)，状态转移拼接消费者接下来只需要补全 s_{t+1}。当前实现依赖如下的三类 Redis 数据：

- **真实曝光序列**：$deviceId \rightarrow [ReqID_1, ReqID_2, \ldots, ReqID_n]$，反馈标签消费者会在消费真实曝光的用户行为时往序列中追加 Request ID；
- **请求状态**：$ReqID \rightarrow s_t$，保存完成 Label 拼接且经过字段裁剪的状态数据；
- **会话与处理状态**：$ReqID \rightarrow SessionID/Status$，分别用于检查请求是否属于同一会话和选择等待窗口。

其中，真实曝光序列既是事实记录，也是下一刷 Request ID 的寻址索引。状态转移拼接消费者在列表中定位当前 Request ID，如果后一项存在便把它作为下一刷；反之，如果当前 ID 不在列表、列表为空或当前项已经位于末尾，则可以认为下一刷不存在。代码16.18给出了这一过程的具体实现。

代码 16.18: 真实曝光请求序列与下一刷定位

```
// 对应文件：RLSampleFlowService.java (RequestSequenceIndex)
public void appendRealShow(String deviceId, long reqId) {
    String flagKey = String.format("real_show_%s_%d", deviceId, reqId);
    if (redis.exists(flagKey)) {
        return;                    // 同一请求只登记一次
    }

    redis.setex(flagKey, ACTION_TTL_SECONDS, "1");

    String listKey = String.format("real_show_list_%s", deviceId);
    List<Long> ids = parseIds(redis.get(listKey));
    ids.remove(reqId);             // 先删后加，避免检查与写入分开时重复
    ids.add(reqId);
    if (ids.size() > TIMELINE_MAX_LEN) {
        ids = ids.subList(ids.size() - TIMELINE_MAX_LEN, ids.size());
    }

    redis.setex(listKey, TIMELINE_TTL_SECONDS, joinIds(ids));
}
```

```
public LookupResult findNextRequest(String deviceId, long reqId) {
    List<Long> ids = parseIds(redis.get("real_show_list_" + deviceId));
    int pos = ids.indexOf(reqId);
    if (pos < 0) {
        return LookupResult.missing("req_id_not_found");
    }

    if (pos + 1 == ids.size()) {
        return LookupResult.missing("next_req_id_not_found");
    }

    return LookupResult.found(ids.get(pos + 1));
}
```

上述代码以设备维度维护有序的请求列表，并据此定位下一刷请求。其中，列表长度和 TTL 会直接影响后续消费者能否找到对应状态，因此列表需要设置合理的长度上限。过短可能导致长会话中的早期请求在完成拼接前被截掉，而过长则会增加 Redis 存储和序列化开销。TTL 则应覆盖反馈等待、下一刷请求到达以及故障恢复所需的时间，避免活跃度较低的设备提前丢失请求索引。

需要注意的是，请求列表的顺序应当来自真实曝光事件的写入顺序，而不能在读取时简单地按照事件时间重新排序。如果行为日志存在跨分区乱序，较早发生的曝光可能晚于后续事件到达，从而被错误地追加到列表末尾。因此，上游应该尽量按照设备 ID 来稳定分区，或者使用支持按事件时间有序查询的数据结构。同时，对于同一设备的并发写入，还需要避免不同请求相互覆盖。

在 Label 拼接过程中，已经完成 Label 拼接的请求还需要将必要状态写入 Redis，供后续请求定位和读取。代码16.19给出了请求状态的压缩写入与结果标记过程。

代码 16.19: 请求状态的压缩写入与结果标记

```
// 对应文件：RLSampleFlowService.java (StateWriter.compress)
public void saveRequestState(long reqId, TrainRecord state) {
    String dataKey = String.format("state_pb_%d", reqId);
    String flagKey = String.format("state_flag_%d", reqId);
    final byte[] compressed;
    try {
        compressed = Snappy.compress(state.toByteArray());
    } catch (IOException e) {
        redis.setex(flagKey, FLAG_TTL_SECONDS, "write_fail");
        metrics.count("save_state_error", e.getClass().getSimpleName());
        return;
    }

    redis.setNxEx(dataKey.getBytes(), compressed, STATE_TTL_SECONDS);
    redis.setex(flagKey, FLAG_TTL_SECONDS, "write_success");
}
```

由于上游 Label 拼接消费者会将每次请求的状态 protobuf 数据提前写入 Redis，因此状态转移拼接消费者在消费请求 ReqID_t 时，首先会在真实曝光序列中定位 ReqID_{t+1}，然后读取并解压该请求对应的状态。如果下一刷 ReqID_{t+1} 与当前刷次 ReqID_t 属于不同的 Session，则不应将其状态拼接到当前样本中。反之，状态转移拼接消费者才会执行后续的状态转移拼接操作。代码16.20展示了请求状态读取与解析的具体实现过程。

代码 16.20: 状态转移消费者读取下一刷请求状态

```
// 对应文件：RLSampleFlowService.java (StateReader.read)
public TrainRecord loadRequestState(long reqId) {
    if (reqId <= 0) {
        return null;
    }

    byte[] compressed = redis.binaryGet(String.format("state_pb_%d", reqId).getBytes());
    if (compressed == null) {
        return null;
    }

    try {
        return TrainRecord.parseFrom(Snappy.uncompress(compressed));
    } catch (Exception e) {
        metrics.count("load_state_error", e.getClass().getSimpleName());
        return null;
    }
}
```

上述代码从 Redis 中读取下一刷请求的状态，并将其中的公共特征、物品特征和 Label 加上 `nxt_` 前缀后拼入当前样本，避免与当前请求的字段发生冲突。在下一刷状态成功拼接后，样本中会记录 `has_join_next=1` 字段用于标记状态转移拼接的状态。在完成前面提到的这些处理操作之后，服务会删除 Redis 中 ReqID_{t+1} 对应的状态，并将清理结果标记为 `clear_success`，既避免同一状态被重复使用，也可以及时释放 Redis 存储空间。代码16.21展示了这一状态清理过程。

代码 16.21: 拼接后的状态清理与标记

```
// 对应文件：RLSampleFlowService.java (StateManager.clear)
public void evictRequestState(long reqId) {
    if (reqId <= 0) {
        return;
    }

    String dataKey = String.format("state_pb_%d", reqId);
    String flagKey = String.format("state_flag_%d", reqId);
    if (redis.exists(dataKey)) {
        redis.delete(dataKey);
        redis.setex(flagKey, FLAG_TTL_SECONDS, "clear_success");
    }
}
```

上述辅助函数由 `TransitionJoinConsumer` 统一组织，对外只提供 `consume` 一个入口。代码16.22展示了完整的类骨架和主流程：构造器中引入了三个核心依赖，其中 `seqIndexService` 用于定位下一刷请求，`stateStore` 负责状态的读取与清理，`kafka` 用于等待中的消息重新投递以及最终样本输出。`consume` 函数按照“先查状态、再校验 Session、最后判断等待窗口”的顺序组织整个处理过程，并依次调用对应方法，使各个环节职责清晰、相互独立。

代码 16.22: TransitionJoinConsumer 骨架与主流程

```
// 对应文件：RLSampleFlowService.java (TransitionJoinConsumer)
public static class TransitionJoinConsumer {
    private final SeqIndexService seqIndexService;
    private final RequestStateStore stateStore;
    private final MockKafka     kafka;

    public TransitionJoinConsumer(SeqIndexService seqIndex,
                                  RequestStateStore stateStore,
                                  MockKafka kafka) {
        this.seqIndexService = seqIndex;
        this.stateStore    = stateStore;
        this.kafka         = kafka;
    }

    public void consume(TrainRecord message) {
        long reqId = message.getRequestId();
        String deviceId = message.getDeviceId();
        String sessionId = message.getSessionId();
        LookupResult lookup = seqIndexService.findNextRequest(deviceId, reqId);
        long nextReqId = lookup.requestId;
        TrainRecord nextState = stateStore.loadRequestState(nextReqId);
        if (nextState != null) {
            String nextSession = stateStore.lookupSession(nextReqId);
            if (equals(sessionId, nextSession) && !isBlank(sessionId)) {
                emitJoined(message, nextState);  // has_join_next=1
            } else {
                emitTerminal(message, "session_mismatch");
            }

            stateStore.evictRequestState(nextReqId); // 已处理的下一刷状态清理
            return;
        }

        String stateFlag = stateStore.queryStateFlag(nextReqId);
        if (isInJoinWindow(message.getTimestamp(), stateFlag)) {
            kafka.send(kafka.selfTopic(), message);
            sleepUninterruptibly(100, TimeUnit.MILLISECONDS);
            return;
        }

        emitTerminal(message, nextReqId > 0 ? stateFlag : lookup.status);
    }

    private boolean isInJoinWindow(long requestMs, String stateFlag) {
        long windowSec = JOIN_WINDOW_SECONDS;
        if ("replay_data".equals(stateFlag)) windowSec = REPLAY_JOIN_WINDOW_SECONDS;
        else if ("missing_data".equals(stateFlag)) windowSec = MAX_JOIN_WINDOW_SECONDS;
        return System.currentTimeMillis() - requestMs < TimeUnit.SECONDS.toMillis(windowSec);
```

```
    }

    /** 完成拼接：标记 has_join_next=1 并发往下游 Topic */
    private void emitJoined(TrainRecord current, TrainRecord next) { ... }

    /** 终止路径：标记 has_join_next=0 并写入终止原因 */
    private void emitTerminal(TrainRecord record, String reason) { ... }
}
```

16.2.1.4 样本的过滤与产出

在状态转移拼接完成后，还需要根据反馈完整性和行为归因判断样本是否可以进入训练集。代码16.22中的不同 `return` 分支对应“继续等待、正常产出和丢弃”三种处理路径。对于已经获得明确反馈的请求，可以提前结束等待。例如，整刷候选均已快速划走时，用户的行为反馈通常已经比较明确，可以直接完成下一刷的样本拼接。相反，如果请求已经曝光且等待窗口结束，却始终没有合理反馈，则不应直接视为负样本，因为其中可能存在行为丢失或上报失败，将缺失反馈当作负反馈容易引入数据偏差。

反馈完整性还需要结合当前决策动作判断。在短视频 Feed 流插入直播的场景中，插入直播时主要需要检查直播点击、自动播放、观看时长和划走等反馈；而在未插入直播时，则需要检查短视频播放次数和播放时长的 label 是否完整。只有动作对应的曝光真实发生，并获得相匹配的反馈，样本才适合进行 Reward 归因。最终样本除了 (s_t, a_t, r_t, s_{t+1}) 等核心字段外，还应保留 `has_join_next`、处理结果和终止原因等信息，便于下游区分正常结束、窗口截断和数据缺失，并持续监控样本拼接质量。

上述方案可以复用已有监督学习样本流，通过 Kafka、Redis 和 Java Consumer 快速构造 TD Learning 所需的状态转移，通常比较适合作为 RL 推荐早期的工程方案。但随着规模增长，每个请求都需要在 Redis 中保存状态，并持续维护请求序列、反馈和下一状态，存储与吞吐压力会不断增加。同时，乱序、延迟、状态过期和异常恢复等问题也需要由应用层处理，使消费者承担越来越多的状态管理逻辑。因此，当 RL 推荐规模进一步扩大后，可以将状态管理和时序关联逐步迁移到更标准的流式处理架构中，提升样本构造的稳定性和扩展能力。

16.2.2 基于 Flink 的 RL 数据流优化

上一节基于 Redis 的 RL 样本拼接方案，需要由 Java Consumer 服务自行维护请求寻址、等待、状态清理以及异常恢复。为了让样本“等得到反馈、找得到下一刷、失败后还能恢复”，Kafka 消费者需要额外维护真实曝光请求序列、请求状态、处理标记和消息再次投递机制。该方案可以在已有 Kafka Consumer 体系中快速落地。但随着状态类型和流量增加，消费者会逐渐承担越来越多本应由流处理框架处理的工作。

因此，我们同样可以引入 Flink 框架来进行 RL 样本流优化，即将状态管理和时序关联的逻辑交给流处理框架处理，从而减轻消费者的负担，提高系统的稳定性和扩展能力。图16.3展示了这种框架。已有样本拼接消费者继续负责请求规范化和 Label 拼接，输出统一的 (s_t, a_t, r_t) 基础样本。而 Flink 框架主要处理跨请求的时序关联，根据真实曝光顺序恢复下一状态 s_{t+1}，最终生成完整的状态转移样本。这样既可以复用已经验证过的反馈定义，又可以把等待、乱序和状态恢复交给标准的流式作业处理。整个数据流如下所示：

$$\text{Request/Behavior Logs} \rightarrow \text{Sample Join} \rightarrow (s_t, a_t, r_t) \rightarrow \text{Flink} \rightarrow (s_t, a_t, r_t, s_{t+1}) \tag{16.20}$$

需要注意的是，Flink 框架接收到的不是未经整理的原始日志，而是已经覆盖 Request ID、设备 ID、Session ID、事件时间、状态、动作、奖励等信息的基础样本。如果输入协议中缺少这些信息，或者不同上游对 Reward 的定义不同，流处理框架本身无法替业务补齐这些信息。

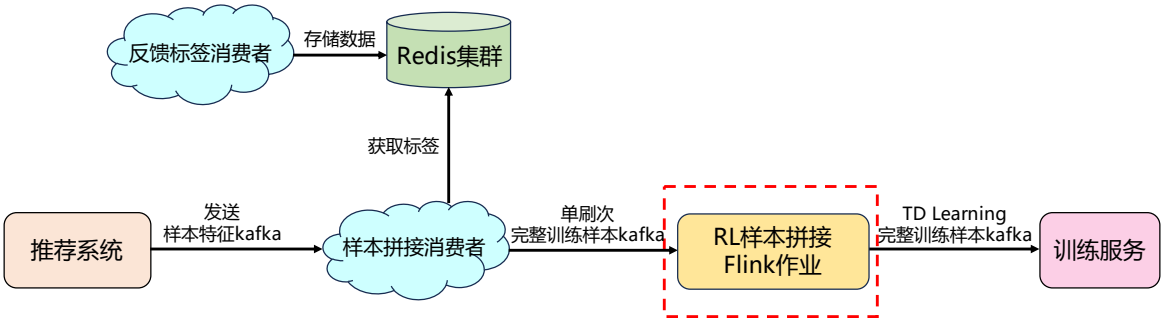


图 16.3: 基于 TD Learning 的工业界强化学习推荐离线架构优化。

16.2.2.1 从手工状态到框架托管

表16.1对比了 Redis 方案中的手工职责、Flink 可以接管的运行时能力以及仍需由业务定义的部分。这里的“Flink 负责”主要指状态存储、恢复与定时触发等基础设施能力，并不意味着框架能够自动理解业务中的下一刷关系、Reward 定义或样本质量等信息。

表 16.1: Redis 手工职责与 Flink 运行时机制的对应关系。

Redis 方案中的职责	Flink 可以托管的部分	业务仍需明确的部分
设备请求列表、状态 Key 与分片	`keyBy`、Keyed State、Key Group 以及扩缩容时的状态重分配	Key 的选择、热点用户、请求去重和事件顺序
消息回写 Topic、休眠与再次投递次数	Event-time Timer、Window Trigger 以及定时器恢复	等待截止时间、乱序容忍度和超迟数据处理
状态数据、处理标记与主动删除	ValueState、MapState、ListState、State TTL 和 `clear()`	状态结构、TTL 长度、缺失原因和业务终止条件
Kafka 位点、内存状态与故障恢复	Checkpoint 同时保存输入位置和算子状态	Checkpoint 周期、存储位置、失败策略和恢复目标
人工状态搬迁与重新分片	Key Group 在恢复和扩缩容时重分配	最大并行度、数据倾斜和状态规模控制

我们首先来看请求寻址。前一章节中的 Redis 方案需要维护设备到 Request ID 列表的映射，并由应用层代码负责列表追加、截断、TTL 以及并发更新的过程；状态转移消费者随后读取列表并查找当前请求的后继。Flink 框架通过 `keyBy` 将相同 Key 的数据路由到同一逻辑状态空间，业务可以直接使用 Keyed State 保存当前设备或 Session 的请求序列，而不再需要通过 Redis Key 在不同消费者之间共享这些中间状态。在任务扩缩容或故障恢复时，Flink 框架也会根据 Key Group 重新分配状态，无需业务额外编写状态搬迁逻辑。

但 `keyBy` 只解决了“数据属于哪份状态”的问题，这个操作并不自动保证事件的顺序，也不会判断两个 Request ID 是否属于同一个 Session。业务层面仍然需要选择稳定的 Key，并根据 Session ID 或时间间隔划分交互会话。例如，对于高活跃设备形成的热点 Key，也需要单独进行数据倾斜控制，因为单个 Key 的状态和计算仍可能集中在同一个并行实例上。

表16.1中的等待机制是两种方案差异较大的地方。Redis 方案需要将尚未完成的消息重新写回 Topic，再通过休眠、再次投递次数和最大等待时间维持等待状态。Flink 则可以直接使用 Event-time Timer，即业务为当前请求注册未来某个事件时间的定时器，当 Watermark 推进到该时间后，框架触发 `onTimer` 函数。定时器会随 Checkpoint 一起保存和恢复，因此无需靠反复写回 Kafka 消息来维持等待，也能减少额外 Topic 流量。但定时器不会替业务决定等待多久。业务还是要根据反馈延迟、请求间隔和 Session 规则定义来等待截止时间，并确定如何处理太晚到达的事件。相比起依赖机器时钟的 Processing-time Timer，这里更适合使用 Event-time Timer，使等待逻辑使用与请求和行为相同的事件时间。当上游 Kafka 数据乱序时，还要结合 Watermark 和允许的乱序时间决定何时结束一个时间窗口。

除此之外，状态的保存和清理从应用自行管理转变为框架托管。Redis 方案通常需要分别维护状态数据、处

理标记和 TTL，Kafka 消费者还要主动执行读取、删除与异常恢复。Flink 框架则通过 ValueState、MapState 和 ListState 等类型化状态保存中间数据，由 State Backend 负责状态的序列化、存储和快照。State TTL 可以限制状态保留时间，使过期数据不再参与计算。需要注意的是，State TTL 更适合处理异常遗留的数据，而不是正常流程中的主要清理手段。状态过期后，底层存储空间不一定立即释放；因此已经完成拼接的请求，业务仍应主动调用 `clear()` 清理状态。TTL 的配置也必须覆盖正常的反馈等待、下一刷等待以及故障恢复所需时间，否则框架可能按配置删除业务仍需的数据。

在 Flink 框架中，故障恢复主要由 Checkpoint 提供。Checkpoint 会保存 Kafka Source 的读取位置以及算子的状态和定时器，使任务失败后能够从最近一次完成的快照恢复，而不需要应用自行判断哪些 Redis 状态属于故障前的数据。在任务恢复或扩缩容时，Flink 框架会重新分配 Key Group，使原有状态继续对应正确的 Key。需要区分的是，Checkpoint 只能恢复 Flink 作业自己的状态，不代表整条链路每条消息都只会写入一次。如果输出使用支持事务的 Kafka Sink，可以把输出提交与 Checkpoint 一起完成；而 Redis、RPC 或普通 Kafka Producer 的写入不受 Checkpoint 的事务控制。对于这些外部系统，通常需要按具体业务选择事务、补偿或去重，不能把 Checkpoint 当成整条链路的事务保证。

最后我们讨论 Session 划分这个对比项。Flink 框架的 Session Window 可以根据一段时间内没有新事件来关闭窗口，并由框架来维护窗口状态和清理逻辑，从而减少应用层手工维护 Session 映射的工作。但窗口本身仍然只是时间规则，并不能判断业务上的 Session ID 是否正确。实际系统可以将设备 ID 与业务 Session ID 结合使用，在同一个 Session 内按照事件时间维护请求关系，并以 Session gap 处理 Session ID 缺失或异常中断的情况。这样既能利用业务 Session 信息划分会话，又能避免用户离开推荐场景后再次进入时产生跨会话的状态转移。

总体而言，Redis 方案解决的是“在现有 Kafka Consumer 体系中快速实现 RL 状态转移拼接”的功能，而 Flink 方案进一步解决的是“如何以标准流处理框架管理大规模、有状态、具有时间约束的状态转移样本构造”。前者强调快速复用已有链路，后者则通过框架托管状态、定时器和恢复机制，降低应用层复杂度，为后续规模化扩展提供更稳定的基础。

16.2.2.2 Event Time 与 Watermark

当状态管理交给 Flink 框架托管之后，应用代码仍需要明确时间究竟以什么为准。Processing Time 表示数据进入 Flink 的时间，而 Event Time 则表示用户行为实际发生的时间。在工业级推荐系统中，网络延迟、Kafka 分区处理速度的差异以及上游重试都有可能造成事件乱序。例如，用户先产生请求 A，再产生请求 B，满足 $T(A) < T(B)$，但由于传输和处理延迟，Flink 可能先收到 B。如果按照到达顺序恢复状态转移，就可能错误地构造出 $B \to A$，直接破坏 RL 中 $(s_t, a_t) \to s_{t+1}$ 的时序关系。

Redis 方案主要依靠请求序列恢复业务顺序，但对于迟到事件缺少统一的时间规则。Flink 框架则可以使用 Event Time 配合 Watermark 来管理乱序数据。Watermark 可以理解为系统对“某个时间点之前的事件已经基本到齐”的判断，它不是对未来数据的绝对保证，而是告诉窗口和定时器可以处理到什么时间。对于推荐日志，通常采用有界乱序策略，即：

$$W(t) = \max_{u \le t} \big(\text{EventTime}(u) - \Delta\big) \tag{16.21}$$

其中，$W(t)$ 表示当前 Watermark，Δ 表示允许的最大乱序时间。Watermark 会随着已经观察到的最大事件时间向前推进，并保留 Δ 的迟到空间。Δ 越大，能够容纳的迟到事件越多，但样本产出也会更晚；Δ 越小，实时性更好，但更多迟到事件可能无法参与正常拼接。在实际配置时，我们可以根据线上事件时间与到达时间之间的延迟分布确定 Δ 的取值，例如参考 $p_{99.9}$ 延迟并结合业务的样本时效要求预留一定余量。

需要注意的是，使用 Event Time 的前提是整条链路统一时间戳来源。例如，用户请求、反馈行为和状态转移都应使用约定的业务事件时间，不能在部分环节切换为 Kafka 写入时间或 Consumer 接收时间，否则不同阶段对时间的理解会产生偏差。与此同时，Watermark 的推进还会受到并行输入的影响。一个算子的 Watermark 通常取各个非空闲输入中最小的值，如果某个 Kafka 分区长时间没有数据或消费严重滞后，就可能拖慢整个算子的时间进度。因此，生产作业需要配置 `withIdleness` 等空闲分区检测机制，并持续监控各分区的 Watermark 差值、事件延迟和定时器触发延迟。

对于需要历史回放、问题排查和训练样本复现的 RL 数据流，Event Time 更适合描述用户行为发生的真实时间。不过，统一时间规则只能保证事件按照一致的时间逻辑处理。如果特征配置、业务规则或数据处理逻辑在不同时间发生变化，即使使用相同的 Event Time，重新处理得到的样本也可能与当时的线上结果不同。因此，实际系统还需要记录相关配置和处理逻辑的版本，才能更完整地复现历史数据处理结果。

16.2.2.3 Flink RL 样本拼接的实现

代码16.23给出了完整的 RL 样本拼接主作业。它从两个 Kafka topic 分别读取基础样本流和真实曝光流，配置 Watermark、Checkpoint、业务分区 Key 以及事务型 Kafka Sink，使前文讨论的时间处理、状态管理与故障恢复机制落实到具体实现。实际项目中的连接参数、序列化方式以及部分 API 会随 Flink 版本变化，但 Watermark、Checkpoint、业务分区、迟到数据处理和事务型 Sink 的配置不能省略。

代码 16.23: Flink 相邻刷次样本拼接主作业

```
// 对应文件：FlinkRLJoiner.java (buildJob)
// 从两个 Kafka topic 读取基础样本流和真实曝光流，
// 按 deviceId keyBy 后交给 RLStateJoinFunction（KeyedCoProcessFunction）
// 完成状态转移拼接，结果写回 rl_transition_samples topic。
public static void buildJob(String bootstrapServers) throws Exception {
    StreamExecutionEnvironment env =
        StreamExecutionEnvironment.getExecutionEnvironment();

    // EXACTLY_ONCE Checkpoint；外部化快照，任务取消后保留
    env.enableCheckpointing(60_000L, CheckpointingMode.EXACTLY_ONCE);
    env.getCheckpointConfig().setCheckpointTimeout(300_000L);
    env.getCheckpointConfig().enableExternalizedCheckpoints(
        ExternalizedCheckpointCleanup.RETAIN_ON_CANCELLATION);
    env.setMaxParallelism(4096);

    Properties kafkaProps = new Properties();
    kafkaProps.setProperty("bootstrap.servers", bootstrapServers);
    kafkaProps.setProperty("group.id", "rl-sample-joiner");

    //   基础样本流：rl_base_samples
    // 上游 Label 拼接消费者输出 (s_t, a_t, r_t)，按 requestTimeMs 分配 Watermark
    DataStream<BaseRLSample> sampleStream = env
        .addSource(new FlinkKafkaConsumer<>(
            "rl_base_samples", new BaseRLSampleDeserializer(), kafkaProps))
        .name("BaseRLSampleSource")
        .assignTimestampsAndWatermarks(
            WatermarkStrategy.<BaseRLSample>forBoundedOutOfOrderness(Duration.ofSeconds(5))
                .withTimestampAssigner((s, t) -> s.requestTimeMs)
                .withIdleness(Duration.ofSeconds(30)));

    //   真实曝光流：real_show_events
    // 客户端回调确认后写入，用于恢复请求顺序
    DataStream<RealShowEvent> showStream = env
        .addSource(new FlinkKafkaConsumer<>(
            "real_show_events", new RealShowEventDeserializer(), kafkaProps))
```

```
        .name("RealShowEventSource")
        .assignTimestampsAndWatermarks(
            WatermarkStrategy.<RealShowEvent>forBoundedOutOfOrderness(Duration.ofSeconds(5))
                .withTimestampAssigner((e, t) -> e.showTimeMs)
                .withIdleness(Duration.ofSeconds(30)));

    //   核心拼接：双流按 deviceId 分区后 connect
    // RLStateJoinFunction 维护曝光序列和等待样本，
    // 定时器超时后输出 terminal 样本
    DataStream<RLTransitionSample> output = sampleStream
        .keyBy(s -> s.deviceId)
        .connect(showStream.keyBy(e -> e.deviceId))
        .process(new RLStateJoinFunction())
        .name("RLStateJoinProcessor").uid("RLStateJoinProcessor");

    //   结果写回 rl_transition_samples：事务型 Sink 与 Checkpoint 对齐
    // 若同时写 Redis 等外部系统须针对该存储另行设计幂等或去重
    output.sinkTo(KafkaSink.<RLTransitionSample>builder()
        .setBootstrapServers(bootstrapServers)
        .setRecordSerializer(KafkaRecordSerializationSchema.builder()
            .setTopic("rl_transition_samples")
            .setValueSerializationSchema(new RLSampleSerializer())
            .build())
        .setDeliveryGuarantee(DeliveryGuarantee.EXACTLY_ONCE)
        .build()).name("RLTransitionSampleSink");

    env.execute("RLSampleJoiner");
}
```

上述代码通过 Watermark 策略设置 5 秒的乱序容忍时间，并在 Event-time Timer 到期后由 `onTimer` 输出超时的终止样本。`withIdleness` 用于避免空闲 Kafka 分区影响整个算子的 Watermark 推进。Checkpoint 则与事务型 Kafka Sink 配合，使状态快照与已提交的输出保持一致，任务恢复后不会重复输出。

在具体实现中，Flink 可以采用窗口式或者增量式两种拼接方式。窗口式方案将同一设备或 Session 的请求暂存在窗口内，待窗口结束后统一进行去重、排序和相邻请求拼接。代码**??**以 Request ID 保留有效请求，再按照事件时间和 Request ID 进行稳定排序，最终生成相邻请求之间的状态转移。

代码 16.24: 窗口函数：去重、排序与相邻拼接

```
// 对应文件：FlinkRLJoiner.java (SessionWindowFunction)
/**
 * Session Window 中处理完整会话片段，按事件时间排序后构建状态转移对。
 * 工业落地时对应：
 *   ProcessWindowFunction<BaseRLSample, RLTransitionSample, String, TimeWindow>.process()
 */
public static class SessionWindowFunction {
    public List<RLTransitionSample> process(String sessionKey,
            Iterable<BaseRLSample> records) {
        Map<Long, BaseRLSample> latestByRequest = new HashMap<>();
        for (BaseRLSample record : records) {
```

```
            BaseRLSample prev = latestByRequest.get(record.requestId);
            if (prev == null || record.requestTimeMs > prev.requestTimeMs)
                latestByRequest.put(record.requestId, record);
        }

        List<BaseRLSample> ordered = new ArrayList<>(latestByRequest.values());
        ordered.sort(Comparator.comparingLong(s -> s.requestTimeMs));
        List<RLTransitionSample> output = new ArrayList<>();
        for (int i = 0; i + 1 < ordered.size(); i++) {
            output.add(RLTransitionSample.joined(ordered.get(i), ordered.get(i + 1)));
        }

        if (!ordered.isEmpty()) {
            output.add(RLTransitionSample.terminal(
                ordered.get(ordered.size() - 1), "session_window_closed"));
        }

        return output;
    }
}
```

上述代码在 Session 窗口结束后对请求进行排序，生成相邻请求之间的状态转移，并补充最后一条请求。因此，样本的产出时间同时受到窗口结束和 Watermark 推进的影响。窗口式方案的主要问题是产出时间相对固定，例如 30 分钟的 Session Window 还需要等待 Watermark 的乱序处理时间，Session 中最后一条请求及其相关状态转移都要等窗口结束后才能输出。对于离线训练而言，这种延迟通常可以接受。但如果希望在下一刷到达后尽快生成状态转移样本，则更适合采用增量式处理。

增量方案则不需要等待整个 Session 结束，而是在请求陆续到达时持续构造相邻的状态转移。对于暂时无法确定顺序的请求，算子会先将其保存在 `pendingByRequest` 中，等到 Watermark 越过对应的事件时间后再处理。这样既可以在允许的乱序范围内接收迟到请求，也能保证已经输出的请求顺序不会被后续数据改变。当一批请求的顺序确定后，算子只保留其中最后一条请求作为当前状态，存入 `retainedTail`，等待下一刷到达后继续生成状态转移。如果长时间没有新的请求，则由 `terminalTimer` 判断当前 Session 是否结束。代码16.25给出了这一过程的具体实现。

代码 16.25: 基于 Keyed State 与 Timer 的增量式拼接

```
// 对应文件：FlinkRLJoiner.java (RLStateJoinFunction)
/**
 * 增量状态转移拼接函数：实时判断下一刷是否已就绪，避免攒整个会话窗口。
 * Key 为 deviceId，状态保存真实曝光序列和等待中的样本。
 * 工业落地时对应：
 *   KeyedProcessFunction<String, BaseRLSample, RLTransitionSample>
 */
public static class RLStateJoinFunction {
    private static final long JOIN_WINDOW_MS = 300_000L;
    // [MOCK] 工业落地替换为：ListState<Long> realShowList;
    private final Map<String, ListState<Long>> realShowLists = new HashMap<>();
    // [MOCK] 工业落地替换为：MapState<Long, BaseRLSample> sampleState;
    private final Map<String, MapState<Long, BaseRLSample>> sampleStates = new HashMap<>();
    private final Map<String, TimerService> timerServices = new HashMap<>();
```

```
    private final Collector<RLTransitionSample> out = new Collector<>();
    public void processElement1(BaseRLSample sample) {
        String deviceId = sample.deviceId;
        getSampleState(deviceId).put(sample.requestId, sample);
        if (!tryJoin(deviceId, sample.requestId, sample))
            getTimerService(deviceId).registerEventTimeTimer(
                    sample.requestTimeMs + JOIN_WINDOW_MS);
    }

    public void processElement2(RealShowEvent event) {
        // 追加到真实曝光序列，再触发所有等待样本的拼接
        ListState<Long> showList = getRealShowList(event.deviceId);
        List<Long> ids = new ArrayList<>();
        for (Long id : showList.get()) ids.add(id);
        if (!ids.contains(event.requestId)) { ids.add(event.requestId); showList.update(ids); }
        for (Map.Entry<Long, BaseRLSample> e : getSampleState(event.deviceId).entries()) {
            tryJoin(event.deviceId, e.getKey(), e.getValue());
        }
    }

    private boolean tryJoin(String deviceId, long reqId, BaseRLSample sample) {
        Long nextReqId = findNextRequest(deviceId, reqId);
        if (nextReqId == null) return false;
        BaseRLSample next = getSampleState(deviceId).get(nextReqId);
        if (next == null) return false;
        if (sample.sessionId.equals(next.sessionId))
            out.collect(RLTransitionSample.joined(sample, next));
        else
            out.collect(RLTransitionSample.terminal(sample, "session_mismatch"));
        getSampleState(deviceId).remove(reqId);
        return true;
    }
}
```

与窗口式方案相比，增量式拼接不需要等待整个 Session 结束，而是随着 Watermark 推进持续释放已经确定顺序的请求，因此能够更快生成 RL 状态转移样本。其代价是需要显式维护样本缓冲、曝光序列和定时器，状态管理也更加复杂。

16.2.3 长序列模型的 RL 数据流扩展

前面两节中介绍的基于 Redis 和 Flink 的方案主要面向 TD Learning 范式的强化学习建模，最终产出的是单步状态转移的样本，即 (s_t, a_t, r_t, s_{t+1})。而以 Decision Transformer（DT）为代表的长序列强化学习模型通常不只关注当前一步，而是直接从一段历史交互序列中学习动作决策。因此，数据流需要进一步从“拼接相邻请求”扩展到“恢复用户长期行为序列”。在工业推荐场景中，一条 DT 的样本通常不是简单地保存过去若干个 (s, a, r)，而是同时维护多条长度一致的序列。例如，对于最近 1000 次请求，可以保存对应的 Request ID、状态特征、推荐物品以及动作和反馈：

$$\begin{aligned}\mathbf{Q}_t &= [q_{t-999}, \ldots, q_t] \\ \mathbf{S}_t &= [s_{t-999}, \ldots, s_t] \\ \mathbf{I}_t &= [i_{t-999}, \ldots, i_t] \\ \mathbf{A}_t &= [a_{t-999}, \ldots, a_t]\end{aligned} \tag{16.22}$$

其中，q 表示 Request ID，s 表示对应请求时的状态，i 表示推荐物品，a 表示该请求下采取的决策。例如，在每次请求最多插入一个直播的场景中，可以额外维护长度为 1000 的直播 ID 序列 $\mathbf{L}_t$，其中每个位置与对应的 Request ID 相对应。对于用户观看、点赞、关注等反馈，也可以分别构造同样长度的行为序列：

$$\mathbf{Y}_t^{(m)} = [y_{t-999}^{(m)}, \ldots, y_t^{(m)}] \tag{16.23}$$

其中 m 表示具体的反馈类型。这里最关键的问题并不是把这些 List 简单保存下来，而是**保证不同序列在时间维度上严格对齐**。例如，第 k 个 Request ID 必须对应第 k 个状态、第 k 个推荐物品、第 k 个动作以及第 k 个 Reward。Reward 或其他 Label 通常并不是在请求发生时立即得到，而是由后验行为日志产生，

因此，我们需要通过 Request ID 和 Item ID 将行为反馈重新映射回对应的序列位置。假设当前请求序列为：

$$[q_1, q_2, \ldots, q_{1000}] \tag{16.24}$$

而后验行为流中记录了若干 $(q_j, i_j, \text{feedback})$，则数据流需要根据 (q_j, i_j) 找到其在请求序列中的位置 j，再将对应的观看时长、点赞、进入直播间等 Label 填入第 j 个位置，最终形成与请求序列严格对齐的 Reward 或 Label List。对于没有发生对应行为的位置，应使用明确的缺失标记或者 Mask，而不能简单地将所有缺失都解释成真实的负反馈。这样最终可以得到一组直接送入序列模型的数据，即：

$$[(s_1, a_1, r_1), (s_2, a_2, r_2), \ldots, (s_{1000}, a_{1000}, r_{1000})] \tag{16.25}$$

然后再根据训练时的上下文长度 K 从中截取不同的子序列，用于构造 Decision Transformer 所需的训练样本。由于这里保存的是完整的历史序列，因此训练阶段可以灵活调整上下文长度、滑动窗口以及序列重叠方式，而不需要重新构造原始行为数据。

与单步 TD 样本相比，长序列样本首先会面临数据规模问题。如果将完整的 1000 步 Request ID、State、Action、Item 和各类 Label 列表直接写入 Kafka，单条消息就可能非常大。再考虑到 Kafka 的副本复制、网络传输、序列化以及多级 Topic 的重复读取，长序列会进一步增加整个数据链路的存储以及带宽压力，因此工业系统通常不会让 Kafka 频繁传输完整的长序列。

一种方式是在在线推荐系统中直接维护用户的长期行为序列。当请求发生时，推荐系统已经能够获得最近一段时间的 Request ID、状态和行为历史，可以直接构造固定长度的序列，并将必要的数据或序列引用传入后续数据流。另一种方式是在数据流中根据用户或设备 ID 查询历史序列，补齐最近 N 次请求的 Request ID、Item ID 和状态，再与后续反馈进行 List 级别的 Join，构造 Reward List。无论采用哪种方式，都必须保证序列截取点位于当前请求之前，避免未来行为或状态进入训练样本。

为了控制长序列的数据量，实际保存的通常是经过压缩和裁剪后的序列字段，而大量实时上下文特征通常不会完整地保留下来。这样虽然可以让 DT 模型从数百甚至上千次历史交互中学习长期行为模式，但也可能削弱模型对当前请求复杂上下文和细粒度物品信息的表达能力，因此长序列样本和单步样本在数据内容上各有侧重。

在工业级 RL 推荐中，更实际的做法是让两类样本并存，而不是用长序列样本完全替代 TD 样本。TD 样本以单步 (s_t, a_t, r_t, s_{t+1}) 为核心，可以保留更加丰富的状态和业务特征，适合学习局部状态转移。而 DT 样本则保存更长的历史行为，重点提供跨请求的长期依赖信息。两类样本可以共享底层的请求、行为和状态数据，分别服务 TD Learning、长序列建模，也可以进一步用于混合建模。

从数据流角度看，长序列样本并不是简单地将单步样本串联起来，而是需要围绕 Request ID 建立稳定的时间索引，将状态、动作、物品和反馈组织成长度一致的序列，同时控制 Kafka 和存储系统中的数据量。这样既可以复用已有的单步样本体系，又能够为长序列 RL 模型提供连续的历史交互信息。

16.3 用户行为日志解析服务

客户端在曝光、播放、停留、点赞、关注或分享等行为发生后，会将相应的行为日志上报到服务端。行为反馈消费者负责读取这些事件，并将能够关联到曝光请求的反馈补充到请求样本中。除此之外，行为日志还需要按照用户或设备维度进行整理，用于更新兴趣序列、统计计数和实时索引。这样，在线请求、训练样本和特征生产可以基于同一套行为数据，避免不同服务重复处理。

一条行为事件至少应包含用户或设备 ID、Request ID、物品 ID、作者 ID、行为类型、行为值、业务场景和事件时间。Request ID 用于关联行为发生时对应的曝光和排序上下文；用户或设备 ID 用于组织行为序列；物品和作者 ID 则分别用于更新物品侧和作者侧的数据。原始行为日志写入后应尽量保持不变，后续的计数、序列和索引都可以基于这些原始事件重新计算。

如图16.4所示，行为解析完成后，还需要进一步更新兴趣序列、统计计数、实时索引和图存储等数据。这些任务在处理速度和资源需求上存在差异，因此通常不会放在同一条同步调用链中，而是将解析结果分别写入对应的 Topic 或任务队列，再由各个服务独立消费。这样某一路服务出现延迟或重试时，不会影响其他数据的正常更新。

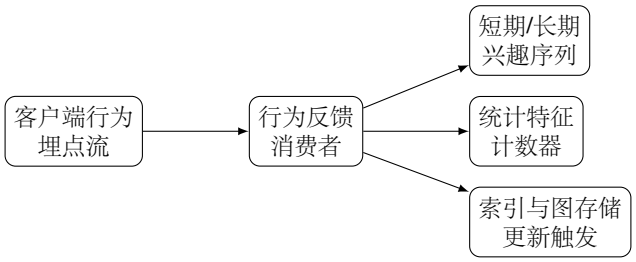


图 16.4: 行为反馈流产生的后续数据服务

16.3.1 短期兴趣序列服务

短期兴趣序列并不是行为日志的简单缓存，而是面向在线推荐请求保存的近期交互历史。排序模型可以从中获取用户最近访问的物品、作者、行为值以及行为时间；召回模块可以利用有效观看、点赞和关注等行为提取兴趣 Trigger；推荐策略也可以据此过滤用户近期负反馈的内容，或者限制同一作者在短时间内重复曝光等。因此，用户行为序列记录除了物品 ID 之外，还需要保留 Request ID、作者 ID、行为值等必要的上下文信息。

从实现上看，Kafka 消费者负责读取和解析行为日志，将一条日志拆分为多个单物品行为，再交给序列服务写入在线键值存储中的 action list。这样，消费者只负责日志解析和任务调度，序列服务负责记录校验、序列化和存储，两部分可以独立处理异常，也不会因为存储问题影响 Kafka 消费。

代码16.26给出了消费者的主要处理流程。`KeySerialExecutor` 保证同一账号或设备的行为按顺序处理，同时允许不同用户并行执行；`consume` 负责确定行为主体和串行键，`appendActions` 则完成 Request ID、并行数组等字段校验，并将每个物品转换为一次序列写入。

代码 16.26: Kafka 行为消费者骨架

```
// 对应文件：BehaviorLogService.java (ActionLogConsumer)
public static class ActionLog {
    public final long userId;
    public final String deviceId;
    public final List<String> requestIds;
    public final List<Long> itemIds;
    public final List<Long> actionValues;
    public final List<Long> authorIds;
```

```
    public final List<Map<String, String>> itemAttributes;
    public final String actionName;
    public final String requestType;
    public final long eventTimeMs;
    public ActionLog(long userId, String deviceId, List<String> requestIds,
                     List<Long> itemIds, List<Long> actionValues, List<Long> authorIds,
                     List<Map<String, String>> itemAttributes, String actionName,
                     String requestType, long eventTimeMs) {
        this.userId = userId;   this.deviceId = deviceId;
        this.requestIds = requestIds != null ? requestIds : List.of();
        this.itemIds = itemIds != null ? itemIds : List.of();
        this.actionValues = actionValues != null ? actionValues : List.of();
        this.authorIds = authorIds != null ? authorIds : List.of();
        this.itemAttributes = itemAttributes != null ? itemAttributes : List.of();
        this.actionName = actionName;
        this.requestType = requestType != null ? requestType : "";
        this.eventTimeMs = eventTimeMs;
    }
}

public static class ActionLogConsumer {
    private final SingleThreadSerialExecutor serialExecutor;
    private final ShortSequenceService sequenceService;
    private final InMemoryItemUpdateBuffer itemUpdateBuffer;
    private final Set<String> ignoredForItemUpdate;
    public ActionLogConsumer(SingleThreadSerialExecutor serialExecutor,
                             ShortSequenceService sequenceService,
                             InMemoryItemUpdateBuffer itemUpdateBuffer,
                             Set<String> ignoredForItemUpdate) {
        this.serialExecutor = serialExecutor;
        this.sequenceService = sequenceService;
        this.itemUpdateBuffer = itemUpdateBuffer;
        this.ignoredForItemUpdate = new HashSet<>(ignoredForItemUpdate);
    }

    public void consume(ActionLog actionLog) {
        String subjectKey = subjectKeyOf(actionLog);
        if (subjectKey == null) return;
        // 串行执行保证同一主体的状态更新不会并发竞争
        serialExecutor.execute(subjectKey, () -> appendActions(actionLog));
    }

    private void appendActions(ActionLog actionLog) {
        long requestId = firstRequestId(actionLog.requestIds);
        if (requestId <= 0 || actionLog.itemIds.isEmpty()) return;
        boolean hasAlignedValue = actionLog.actionValues.size() == actionLog.itemIds.size();
        boolean hasAlignedAuthor = actionLog.authorIds.size() == actionLog.itemIds.size();
        for (int i = 0; i < actionLog.itemIds.size(); i++) {
            long itemId = actionLog.itemIds.get(i);
```

```
            if (itemId <= 0) continue;
            long actionValue = hasAlignedValue ? actionLog.actionValues.get(i) : 1L;
            long authorId  = hasAlignedAuthor ? actionLog.authorIds.get(i) : 0L;
            sequenceService.appendAction(requestId, actionLog.userId, actionLog.deviceId,
                    itemId, authorId, actionLog.actionName, actionValue,
                    actionLog.requestType, sourceTypeAt(actionLog.itemAttributes, i),
                    actionLog.eventTimeMs);
            if (!ignoredForItemUpdate.contains(actionLog.actionName))
                itemUpdateBuffer.add(itemId);
        }
    }
}
```

上述代码最终将校验后的日志字段转换为 `appendAction` 调用，写入 Request ID、账号或设备 ID、物品与作者 ID、行为类型及行为值、来源和事件时间等信息。如果账号和设备标识均缺失，则不再调用序列服务；如果某个物品 ID 不合法，也只跳过当前物品，不影响同一条日志中其他行为的处理。同时，并行数组校验可以保证物品、作者和行为值之间保持正确对应，避免上游数据异常导致字段错位。

序列服务负责维护类似 Redis 的在线键值存储，每个行为都会追加到一个有限长度的 action list 中，列表元素保存经过紧凑编码的行为记录。当列表超过设定长度时，最早的记录会被移除，并同步刷新列表的过期时间。这里以设备 ID 作为 action list 的索引，在线推荐请求时可以直接按照设备 ID 获取近期行为。

代码16.27中的 `ActionListStore` 是一个用于说明存储逻辑的本地内存实现，并不承担生产环境中的持久化职责。这里的两个 `HashMap` 只存在于当前 Java 进程中，进程重启后 action list 和过期信息都会随之消失；如果服务部署了多个实例，不同实例之间也无法共享各自写入的记录。因此，这段代码更适合用来展示追加、截断以及 TTL 刷新的先后关系。

代码 16.27: 短期序列的核心追加逻辑

```
// 对应文件：BehaviorLogService.java (ShortSequenceStore.append)
public static class ActionRecord {
    public final long requestId;
    public final long itemId;
    public final long authorId;
    public final String actionName;
    public final long actionValue;
    public final String requestType;
    public final String sourceType;
    public final long eventTimeMs;
    public ActionRecord(long requestId, long itemId, long authorId, String actionName,
                        long actionValue, String requestType, String sourceType, long eventTimeMs) {
        this.requestId = requestId; this.itemId = itemId;
        this.authorId = authorId; this.actionName = actionName;
        this.actionValue = actionValue; this.requestType = requestType;
        this.sourceType = sourceType; this.eventTimeMs = eventTimeMs;
    }
}

public static class ShortSequenceService {
    private static final int MAX_ACTIONS_PER_LIST = 200;
    private static final java.time.Duration ACTION_LIST_TTL = java.time.Duration.ofDays(14);
```

```
    private final ActionListStore actionListStore;
    public ShortSequenceService(ActionListStore actionListStore) {
        this.actionListStore = actionListStore;
    }

    public void appendAction(long requestId, long userId, String deviceId, long itemId,
                             long authorId, String actionName, long actionValue,
                             String requestType, String sourceType, long eventTimeMs) {
        if (requestId <= 0 || itemId <= 0 || actionName == null || actionName.isBlank()) return;
        if (userId <= 0 && (deviceId == null || deviceId.isBlank())) return;
        if (deviceId == null || deviceId.isBlank()) return;
        ActionRecord record = new ActionRecord(requestId, itemId, authorId, actionName,
                actionValue, requestType == null ? "" : requestType,
                sourceType == null ? "" : sourceType, eventTimeMs);
        // [MOCK] 工业落地时替换为 Redis LPUSH / LTRIM / EXPIRE 原子操作
        actionListStore.append(deviceKey(deviceId, actionName),
                record.toBytes(), MAX_ACTIONS_PER_LIST, ACTION_LIST_TTL);
    }

    public List<ActionRecord> getRecentActions(String deviceId, String actionName) {
        return actionListStore.get(deviceKey(deviceId, actionName));
    }

    private String deviceKey(String deviceId, String actionName) {
        return "recent-action:" + deviceId + ":" + actionName;
    }
}
```

上述 ActionListStore 的代码仍然只是为了说明核心处理逻辑。在生产环境中，append 通常需要替换成 Redis 等外部在线存储的原子操作，使记录追加、超长列表截断以及 TTL 刷新能够在一次操作中完成。如果这三个步骤被拆成多次远程调用，那么服务在中途失败时可能留下未截断的列表或者未刷新的过期时间；而继续采用进程内 Map，则会导致多实例之间看到不同的用户历史。ShortSequenceService 本身则负责将已经校验的字段组织成 ActionRecord，生成设备 action list 的键，并调用底层存储。代码中的 toBytes 使用 UTF-8 文本只是为了便于展示字段含义，实际生产实现可以替换成 Protobuf 等更加紧凑的二进制编码，而键、列表长度和 TTL 等核心规则保持不变。

总的来说，短期兴趣序列服务实际上完成了一次清晰的数据转换：把 Kafka 中以日志处理为中心的多物品行为，整理成在线推荐可以快速读取、按照行为类型组织，并具有明确长度和保留周期的 action list。

16.3.2 长期兴趣序列服务

长期兴趣序列保存更长时间范围内的用户行为，用于描述相对稳定的兴趣偏好，也可以作为长期序列模型的输入，为训练样本提供可回放的历史轨迹。与短期序列一样，长期序列也可以由 Kafka 消费者实时写入，并通过封装后的 RPC 接口供上层服务读取。两者的主要区别在于数据规模和保留时间。短期序列主要服务在线请求，需要控制长度和访问开销；长期序列则需要保存更丰富的历史，并具备更好的持久化和恢复能力。

如果推荐系统已经使用 Redis 承担短期序列和缓存服务，长期序列可以继续采用相同的数据模型和访问方式，但最好使用独立的 Redis Cluster 或资源池，避免长期历史与在线缓存相互影响。长期序列可以同时开启 AOF 和 RDB 持久化，其中 AOF 记录写操作，RDB 保存数据快照，两者分别提供不同的数据恢复能力。AOF 使用 appendfsync everysec 可以在写入性能和数据安全之间取得平衡；如果对数据丢失更加敏感，也可以使

用 `appendfsync always`，但会增加写入开销。长期运行时还需要关注 AOF 重写、磁盘空间以及持久化过程带来的额外资源消耗。

在存储结构上，长期序列可以按照设备或账号进行组织。如果需要保证同一主体的数据尽量落在相同分片，可以将设备或账号标识作为 Redis Cluster 的 hash key。长期历史更适合使用有序集合（sorted set）保存，以 `eventTimeMs` 作为 score，以包含 `eventId`、`itemId`、`authorId`、行为值和来源等信息的紧凑记录作为 member。这样可以直接按照时间范围读取指定数量的历史记录，而不需要将整个集合取出后再由应用层排序和过滤。`ZADD` 的时间复杂度为 $O(\log N)$，适合持续追加行为并按时间查询。

长期保存也需要控制数据规模，不能让单个用户的历史无限增长。通常需要同时设置最大保留时间和最大记录数，超过限制后删除最早的数据。对于行为非常频繁的用户或设备，还可以按照月份拆分多个 sorted set，查询时只读取覆盖目标时间范围的少量数据，从而控制单个 Key 的大小和访问开销。具体采用多大的时间粒度，可以根据单 Key 内存、写入 QPS 和读取 P99 等指标进行调整。

短期与长期序列的存储差异如表16.2所示。短期序列更强调在线访问效率，长期序列则需要同时考虑历史保留、持久化和容量控制，因此两者虽然可以采用相同的存储系统，但在资源隔离和数据管理策略上通常需要区别对待。

表 16.2: 短期与长期兴趣序列的存储差异

维度	短期兴趣序列	长期兴趣序列
时效与延迟	毫秒级追加和毫秒级按键读取	毫秒级追加；经 RPC 按用户或设备、动作和时间窗读取
数据组织	主体、动作对应的有限列表	主体、动作对应的 Redis 有序集合；按事件时间排序，超大键再按月分桶
存储选型	在线 Redis 列表或等价结构	独立的持久化 Redis Cluster，配置 AOF、RDB、复制和不淘汰历史的容量策略
保留与容量	数天至数周，自动过期并截断	数月以上，按 score 清理历史、长 TTL 和最大成员数共同约束
主要用途	在线序列特征、召回种子和业务策略	长期序列特征、用户画像、长期模型与训练回放

为了避免上层业务直接依赖 Redis 的存储细节，调用方不需要维护 Key 名、时间分桶或 Lua 脚本。长期序列服务可以通过 RPC 提供 `AppendHistory(event)` 和 `GetHistory(subjectId, actions, startTimeMs, endTimeMs, limit)` 等接口，分别负责历史写入和按时间范围读取。读取时从当前时间桶向前查询，达到 `limit` 条记录后即可停止，从而避免将全部历史拉回应用层再过滤。RPC 层还可以统一处理动作类型、时间范围、内容下线和用户删除等规则，并通过读写延迟、成员数量和副本延迟等指标监控服务状态。

当长期历史规模超过 Redis 更适合承载的范围时，可以将低热度时间桶或完整历史逐步迁移到 HBase、Cassandra 等宽列存储，而 Redis 继续承担高频在线查询。本节采用 Redis，主要是因为它能够同时满足实时写入和低延迟查询，并通过持久化、复制和资源隔离提供相应的数据保护能力。

在训练侧读取长期历史时，还需要以样本时间作为查询上界，只使用该时间之前已经发生的行为，避免未来信息进入训练样本。这一规则既可以通过同一套 RPC 实现，也可以通过按日期生成的列式快照实现，但都应以 `eventTimeMs` 作为时间筛选依据。这样实时链路负责持续写入新行为，长期历史则同时服务在线查询和离线回放，形成统一的行为历史数据通道。

16.4 统计特征生产服务

统计特征生产服务负责消费用户行为反馈的 Action Log，并按照用户、物品和作者等维度更新行为统计信息。服务根据不同的行为类型和时间窗口，持续累加曝光、点击、观看、点赞、关注以及观看时长等统计信息。

这些统计结果可以进一步提供给在线特征服务等，用于构造用户侧、物品侧和作者侧的统计特征以及经验转化率（empirical XTR）。

在一条 Action Log 进入服务后，首先会先检查行为类型和相关字段是否有效，再按照日志中的物品逐条更新统计。用户侧统计特征主要按照用户 ID 进行累加；对于未登录用户，也可以使用设备 ID 进行统计，并进一步记录用户与作者之间的交叉信息。而物品侧统计特征则根据物品 ID 更新对应的行为计数，同时将相关统计信息同步到作者侧。对于观看时长等数值型行为，统计值可以直接累加。而对于点击、点赞等发生型行为，则按照行为次数进行累加。

为了支持不同时间范围的统计，服务会将行为按照事件时间划分到不同的时间桶中。计数 Key 中包含统计对象、行为类型、时间窗口以及业务场景等信息。例如，`item:9001:click:5m:home` 表示物品 `9001` 在首页场景下，`v3` 版本的五分钟点击统计。查询一个时间窗口时，只需要将覆盖该时间范围的时间桶进行累加即可：

$$C_W(e,a) = \sum_{b \in \mathcal{B}(W,a)} C_b(e,a) \tag{16.26}$$

其中，e 表示统计对象，a 表示行为类型，C_b 表示某个时间桶中的统计值。较短的统计窗口通常使用更细的时间桶，较长的窗口则可以使用更粗的时间桶，从而在统计精度和存储开销之间取得平衡。

代码16.28给出了用户侧、物品侧和作者侧共用的事件消费骨架。`consume` 首先读取 Action Log 中的行为和物品信息，然后分别调用用户侧和物品侧的统计逻辑。用户侧按照用户 ID 累加行为，物品侧按照物品 ID 更新统计，并进一步同步作者侧数据。对于一条日志中的多个物品，代码会逐个处理对应的行为值和作者 ID。这样，一条 Action Log 就可以同时更新用户、物品和作者三个维度的统计，为后续在线特征构造与使用提供基础数据。

代码 16.28: 用户侧与物品侧共用的事件消费骨架

```
// 对应文件：StatFeatureService.java (CounterEventConsumer)
/** Kafka 消费者：消费用户 Action Log，实时更新用户侧和物品侧窗口计数。 */
public static class CounterEventConsumer {
    private final TimeBucketCounter counter;
    private final UserCounterWriter userWriter;
    private final ItemCounterWriter itemWriter;
    public CounterEventConsumer(TimeBucketCounter counter) {
        this.counter = counter;
        this.userWriter = new UserCounterWriter(counter);
        this.itemWriter = new ItemCounterWriter(counter);
    }

    public void consume(ActionLog actionLog) {
        if (actionLog == null || actionLog.actionName == null) return;
        long eventTimeMs = actionLog.eventTimeMs;
        for (int i = 0; i < actionLog.itemIds.size(); i++) {
            long itemId = actionLog.itemIds.get(i);
            long reqId  = firstRequestId(actionLog.requestIds);
            long delta  = i < actionLog.actionValues.size()
                    ? actionLog.actionValues.get(i) : 1L;
            long authorId = i < actionLog.authorIds.size()
                    ? actionLog.authorIds.get(i) : -1L;
            // 用户侧计数（按 userId，不按 requestId 去重）
            if (actionLog.userId > 0) {
                userWriter.process(new CounterEvent(
                        actionLog.userId, "user", actionLog.actionName, delta, eventTimeMs));
```

```
            }

            // 物品侧计数（按 requestId 去重，同一次刷新只计一次；同步写入作者侧）
            itemWriter.process(new CounterEvent(
                    itemId, "item", actionLog.actionName, delta, eventTimeMs), reqId, authorId);
        }
    }

    private long firstRequestId(List<String> ids) {
        if (ids == null || ids.isEmpty()) return -1L;
        try { return Long.parseLong(ids.get(0)); } catch (NumberFormatException e) { return -1L; }
    }
}

public static class CounterEvent {
    public final long subjectId;
    /** "user" or "item" */
    public final String subjectType;
    /** 行为类型："click" / "show" / "like" */
    public final String actionName;
    /** 行为数值（发生型为 1，数值型为实际值） */
    public final long delta;
    public final long eventTimeMs;
    public CounterEvent(long subjectId, String subjectType, String actionName,
                        long delta, long eventTimeMs) {
        this.subjectId = subjectId; this.subjectType = subjectType;
        this.actionName = actionName; this.delta = delta;
        this.eventTimeMs = eventTimeMs;
    }
}
```

16.4.1 用户侧计数特征产出

代码16.29给出用户侧写入器的核心逻辑。它接收共同消费者已校验的 CounterEvent，以用户 ID 为主体键调用底层 counter.increment 原子累加到对应时间桶。需要注意的是，用户侧不按 requestId 去重，同一用户对同一物品的多次行为均分别计入窗口计数。

代码 16.29: 用户侧窗口计数写入器

```
// 对应文件：StatFeatureService.java (UserCounterWriter)
public static class UserCounterWriter {
    private final TimeBucketCounter counter;
    public UserCounterWriter(TimeBucketCounter counter) {
        this.counter = counter;
    }

    /** 处理用户侧行为事件，按时间窗口累加计数；用户侧不按 requestId 去重。 */
    public void process(CounterEvent event) {
        if (event == null || !"user".equals(event.subjectType)) return;
```

```
        if (event.delta <= 0) return;
        counter.increment(event.subjectId, event.actionName,
                event.delta, -1L /* 不去重 */, event.eventTimeMs);
    }
}
```

上述代码按照用户 ID 将行为写入对应的时间桶。用户侧不按 `requestId` 去重，因此同一用户的多次行为都会分别计入统计；其中有效观看等数值型行为累加实际值，而其他行为则累加次数。在线特征 RPC 可以按照用户或设备 ID 批量读取不同时间窗口的统计结果，用于生成近期曝光、点击、观看和互动等特征。在读取时也应该以当前请求时间作为上界，避免将请求之后产生的行为混入到当前的统计特征中。

16.4.2 物品侧计数特征产出

物品侧主要统计物品和作者维度的行为数据。代码16.30会同时更新物品和作者的窗口计数，并按照 `requestId` 进行去重，保证同一次刷新中同一物品的重复上报不会被重复计入，作者侧统计也采用相同的规则。为了区分两个统计维度，作者侧的动作名称统一增加 `author_` 前缀。对于有效观看等包含数值的行为，则将观看时长作为增量进行累加。

代码 16.30: 物品侧窗口计数写入器

```
// 对应文件：StatFeatureService.java (ItemCounterWriter)
public static class ItemCounterWriter {
    private final TimeBucketCounter counter;
    public ItemCounterWriter(TimeBucketCounter counter) {
        this.counter = counter;
    }

    /**
     * 处理物品侧行为事件，分别为物品和作者写入窗口计数。
     * requestId 用于去重：同一次刷新对同一物品只计一次。
     * 作者侧复用相同的 requestId 去重标记，动作名前缀 "author_" 以区分命名空间。
     */
    public void process(CounterEvent event, long requestId, long authorId) {
        if (event == null || !"item".equals(event.subjectType)) return;
        if (event.delta <= 0) return;
        counter.increment(event.subjectId, event.actionName,
                event.delta, requestId /* 按 requestId 去重 */,
                event.eventTimeMs);
        if (authorId > 0) {
            counter.increment(authorId, "author_" + event.actionName,
                    event.delta, requestId, event.eventTimeMs);
        }
    }
}
```

上述处理完成后，索引服务可以根据物品的窗口计数更新对应的物品数据。经验转化率不需要在行为到达时单独维护，而是根据已经统计的曝光、点击和有效观看等数据计算得到。这样可以复用已有的窗口计数，同时支持不同时间范围下的统计特征。代码16.31按 Item ID 批量读取多个不同的反馈行为统计特征，从而计算当

前 Item 的经验转换率。以物品 i 为例，平滑的点击率和有效观看率分别为：

$$\widehat{CTR}_W(i) = \frac{click_W(i) + \alpha_c}{show_W(i) + \alpha_c + \beta_c}, \qquad \widehat{VTR}_W(i) = \frac{valid_watch_W(i) + \alpha_v}{click_W(i) + \alpha_v + \beta_v}. \tag{16.27}$$

其中 `show` 是经验 CTR 的分母，`click` 是经验 VTR 的分母。需要注意的是，新物品在没有曝光和互动时，平滑后的比率仍等于先验均值。此外，当计数服务读取失败时，应返回明确错误并执行预先定义的失败处理流程，而不能将读取失败当做取值为零的 count 计数。只有这样，我们才能将其与新物品的正常零计数区分开来。

代码 16.31: 物品计数读取与经验转化率构造

```
// 对应文件：StatFeatureService.java (CounterSnapshotService)
public static class EmpiricalXtrService {
    private final double ctrPrior;
    private final double vtrPrior;
    public EmpiricalXtrService(double ctrPrior, double vtrPrior) {
        this.ctrPrior = ctrPrior;
        this.vtrPrior = vtrPrior;
    }

    /** 贝叶斯平滑 CTR：(click + alpha) / (show + beta)，避免冷启动物品 CTR 为 0 */
    public double smoothedCtr(long shows, long clicks) {
        double beta = 10.0;
        double alpha = ctrPrior * beta;
        return (clicks + alpha) / (shows + beta);
    }

    public double smoothedVtr(long shows, long views) {
        double beta = 10.0;
        double alpha = vtrPrior * beta;
        return (views + alpha) / (shows + beta);
    }

    public Map<Long, ItemStatistics> load(Collection<Long> itemIds,
                                          TimeBucketCounter counter) {
        Map<Long, ItemStatistics> result = new HashMap<>();
        for (long itemId : itemIds) {
            Map<String, Long> shows = counter.getAll(itemId, "show");
            Map<String, Long> clicks = counter.getAll(itemId, "click");
            Map<String, Long> views = counter.getAll(itemId, "view");
            long show24h = shows.getOrDefault("24h", 0L);
            long click24h = clicks.getOrDefault("24h", 0L);
            long view24h = views.getOrDefault("24h", 0L);
            result.put(itemId, new ItemStatistics(itemId, shows, clicks,
                    smoothedCtr(show24h, click24h), smoothedVtr(show24h, view24h)));
        }

        return result;
    }
}
```

```
public static class ItemCounterSnapshotService {
    private final TimeBucketCounter counter;
    private final EmpiricalXtrService xtrService;
    public ItemCounterSnapshotService(TimeBucketCounter counter,
                                      double ctrPrior, double vtrPrior) {
        this.counter  = counter;
        this.xtrService = new EmpiricalXtrService(ctrPrior, vtrPrior);
    }

    /** 批量拉取物品计数快照，并用贝叶斯平滑计算经验 CTR/VTR。 */
    public Map<Long, ItemStatistics> load(Collection<Long> itemIds) {
        return xtrService.load(itemIds, counter);
    }
}
```

16.4.3 计数变更触发的索引更新

当物品侧的计数更新完成后，生产端会向 Kafka 发送一条计数变更消息，通知下游更新对应的物品索引。消息以物品 ID 作为 Key，使同一物品的消息尽量进入同一个分区；消息内容只包含物品 ID、触发原因和产生时间，不直接携带当前的计数值。下游收到消息后，再根据物品 ID 读取最新的统计结果并更新索引。这样，即使同一物品收到多次触发消息，也只是重复执行更新，不会因为消息中的计数过旧而覆盖最新结果。代码16.32给出了这一过程的实现。

代码 16.32: Kafka 发送计数变更触发消息

```
// 对应文件：StatFeatureService.java (CounterIndexTriggerProducer)
public static class CounterIndexTrigger {
    /** [MOCK] 工业落地时替换为 KafkaProducer<String, IndexUpdateTrigger> */
    private final List<String> kafkaMessages;
    public CounterIndexTrigger(List<String> kafkaMessages) {
        this.kafkaMessages = kafkaMessages;
    }

    /**
     * 计数更新后向 Kafka 发布触发消息，通知下游重建物品索引。
     * 消息体只包含 itemId、触发原因和产生时间，不携带具体计数快照。
     * 工业落地时对应 KafkaTemplate.send(triggerTopic, indexUpdateTrigger)。
     */
    public void trigger(long itemId, String actionName) {
        String msg = "item_counter_update:" + itemId + ":" + actionName
               + ":ts=" + System.currentTimeMillis();
        kafkaMessages.add(msg);
    }
}
```

上述代码将物品 ID 和行为类型组成触发消息，并记录当前产生时间。在实际接入 Kafka 后，CounterIndexTrigger 只负责发送通知，具体的计数读取和索引更新由下游服务完成。综上，统计特征生产服务通过离线计算得到用户侧、物品侧以及作者侧的计数类特征。在具体线上使用时，用户侧计数可以由在线特征 RPC 接口直接获取。而

物品侧计数、作者侧计数以及物品侧经验 XTR 通常都会存储到索引服务中，在使用时通过请求正排在线服务即可获取这些统计字段。

16.5 物品侧内容理解生产服务

行为反馈反映的是物品被消费后的结果，而对于尚无反馈的新物品，索引和模型还需要从内容本身获取特征。因此，物品侧内容理解服务通常以图片、视频帧、标题、OCR 文本和语音转写等原始内容为输入，先检查内容完整性、重复性和处理条件，再根据输入模态完成下载、解码和归一化，最后调用视觉、文本或多模态基础模型进行内容理解。模型可以输出类目、主题、实体、安全和质量等标签，也可以生成连续的内容向量。标签主要用于解释、过滤和构造稀疏特征，而 Embedding 则用于语义召回、相似物品检索和深度模型输入，二者面向不同的下游用途。

在实际系统中，内容理解通常采用两层服务架构。靠近数据流的消费者负责接收新内容，完成轻量筛选和预处理，并维护内容标识与处理结果的对应关系。常驻的模型服务负责加载基础模型、合并推理请求、执行模型推理并返回结构化结果。这样可以让消费者随数据流横向扩展，而模型权重只需在有限数量的 GPU 进程中加载。模型输出的标签可以写入内容标注存储，向量则写入特征日志或向量索引，进一步进入索引构建、召回、排序和训练样本链路。

下面以直播切片的实时内容理解为例。在直播开始后，切片采集服务按固定的时间间隔从直播画面截取图片，并附带作者 ID、截取时间以及图片地址。内容向量消费者需要筛选满足时限和实验条件的图片，完成预处理后调用多模态模型服务，并将返回的 Embedding 写入下游的日志中。模型服务同时可以利用 Qwen2.5-VL 等多模态大模型为图片生成垂类、置信度以及主题标签，并写入独立的标注存储。代码16.33展示了消费者如何组织一批切片并保持输入与向量的对应关系；代码16.34展示了后端模型服务如何加载模型、校验请求、生成标签并返回向量。

16.5.1 内容向量消费者

内容向量消费者位于直播切片数据流和模型服务之间，接收带有作者 ID、截取时间以及图片地址的切片元数据，并输出带有相同标识的内容向量日志。作者 ID 与截取时间共同标识输出记录，使索引构建和训练样本服务能够将向量关联回对应的直播 Item。因此，消费者并不只是一个图片下载任务，而是将新开播的实时内容转换为可供检索以及模型使用的特征。

如下所示，代码16.33将这一过程划分为切片筛选、图片预处理和向量推理三个环节。`LiveSlice`、`ImageInput` 和 `ImageEmbeddingResponse` 使用 Java 16 引入的 `record` 表示不可变数据载体，分别对应切片元数据、预处理后的图片和模型响应。图片下载器、向量服务客户端和日志写入器则分别负责图片下载与归一化、远程推理和结果输出。`consume` 方法则串起主流程。`consume` 方法首先筛选满足时限和实验条件的切片，再并行下载和预处理图片，随后将有效图片拼接成批量输入，调用向量服务并校验响应，最后将返回的扁平数组拆分为逐图片向量写入日志。

代码 16.33: 直播切片内容向量消费者

```
// 对应文件：ContentEmbeddingService.java (LiveSliceEmbeddingConsumer)
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Future;

record LiveSlice(long authorId, long capturedAtMs, String imageUrl) { }
```

```
record ImageInput(long authorId, long capturedAtMs, List<Double> pixels) { }

record ImageEmbeddingResponse(
       List<Double> values,
       int embeddingDimension,
       int batchSize) { }

interface ImageDownloader {
   List<Double> downloadResizeAndNormalize(String imageUrl) throws Exception;
}

interface ImageEmbeddingClient {
   ImageEmbeddingResponse embed(
          List<Double> imageValues,
          int imageDimension,
          int batchSize) throws Exception;
}

interface ContentVectorLog {
   void append(long authorId, long capturedAtMs, List<Double> embedding);
}

interface ExperimentBucketSelector {
   boolean isEnabled(long authorId);
}

public final class LiveSliceEmbeddingConsumer {
   private static final long RECENT_SLICE_MILLIS = 5 * 60 * 1000L;
   private static final int IMAGE_DIMENSION = 3 * 448 * 448;

   private final ExecutorService downloadExecutor;
   private final ImageDownloader downloader;
   private final ImageEmbeddingClient embeddingClient;
   private final ContentVectorLog outputLog;
   private final ExperimentBucketSelector bucketSelector;

   public LiveSliceEmbeddingConsumer(
          ExecutorService downloadExecutor,
          ImageDownloader downloader,
          ImageEmbeddingClient embeddingClient,
          ContentVectorLog outputLog,
          ExperimentBucketSelector bucketSelector) {
      this.downloadExecutor = downloadExecutor;
      this.downloader = downloader;
      this.embeddingClient = embeddingClient;
      this.outputLog = outputLog;
      this.bucketSelector = bucketSelector;
   }
```

```
public void consume(List<LiveSlice> slices, long nowMs) {
    List<Future<ImageInput>> futures = new ArrayList<>();
    for (LiveSlice slice : slices) {
        if (!isRecent(slice, nowMs) || !bucketSelector.isEnabled(slice.authorId())) {
            continue;
        }

        futures.add(downloadExecutor.submit(download(slice)));
    }

    List<ImageInput> inputs = collectInputs(futures);
    if (inputs.isEmpty()) {
        return;
    }

    List<Double> imageValues = new ArrayList<>(inputs.size() * IMAGE_DIMENSION);
    for (ImageInput input : inputs) {
        imageValues.addAll(input.pixels());
    }

    ImageEmbeddingResponse response;
    try {
        response = embeddingClient.embed(
                imageValues,
                IMAGE_DIMENSION,
                inputs.size());
    } catch (Exception error) {
        recordFailure("embedding_rpc", error);
        return;
    }

    if (!isComplete(response, inputs.size())) {
        recordInvalidResponse(response);
        return;
    }

    writeEmbeddings(inputs, response);
}

private Callable<ImageInput> download(LiveSlice slice) {
    return () -> new ImageInput(
            slice.authorId(),
            slice.capturedAtMs(),
            downloader.downloadResizeAndNormalize(slice.imageUrl()));
}

private List<ImageInput> collectInputs(List<Future<ImageInput>> futures) {
    List<ImageInput> inputs = new ArrayList<>();
    for (Future<ImageInput> future : futures) {
```

```
            try {
                inputs.add(future.get());
            } catch (Exception error) {
                recordFailure("image_download_or_preprocess", error);
            }
        }

        return inputs;
    }

    private boolean isRecent(LiveSlice slice, long nowMs) {
        return slice.authorId() > 0
                && slice.capturedAtMs() <= nowMs
                && nowMs - slice.capturedAtMs() <= RECENT_SLICE_MILLIS;
    }

    private boolean isComplete(ImageEmbeddingResponse response, int inputCount) {
        return response.batchSize() == inputCount && response.embeddingDimension() > 0
                && response.values().size() == inputCount * response.embeddingDimension();
    }

    private void writeEmbeddings(List<ImageInput> inputs, ImageEmbeddingResponse response) {
        int dimension = response.embeddingDimension();
        for (int index = 0; index < inputs.size(); index++) {
            int from = index * dimension;
            int to = from + dimension;
            ImageInput input = inputs.get(index);
            outputLog.append(input.authorId(), input.capturedAtMs(), response.values().subList(from,↩
                ↪ to));
        }
    }

    private void recordFailure(String stage, Exception error) {
        System.err.println("[ContentEmbedding] " + stage + ": " + error.getMessage());
    }

    private void recordInvalidResponse(ImageEmbeddingResponse response) {
        System.err.println("[ContentEmbedding] incomplete embedding response: " + response);
    }
}
```

上述代码将切片筛选、图片预处理、批量推理和结果写入串成一条完整的数据链路。这里有一个重要的工程取舍，即图片级失败采用局部跳过，批量推理结果异常则可以整体丢弃。前者避免单张坏图片阻塞整个直播流，后者则保证批量向量与原始切片之间的对应关系不被破坏。

16.5.2 多模态模型推理 RPC 服务

上一小节的消费者将归一化图片整理成批量输入，并通过远程过程调用（Remote Procedure Call, RPC）发送给常驻的模型服务。如果每个消费者进程都自行加载大模型，不仅会重复占用显存、增加启动开销，也难以在

单个进程内积累足够的请求形成有效批处理。因此，模型加载和推理统一由独立的 RPC 服务负责，消费者只需提交图片张量，而 RPC 服务则完成请求校验、模型推理并返回向量。

一次请求携带单张图片的大小维度以及 batch size 信息，多模态模型推理 RPC 服务据此校验输入数据是否完整。在推理完成后，服务按照输入顺序返回向量，同时将垂类、置信度和主题标签写入独立的内容标注存储。这样，向量和标签可以分别服务于索引、训练以及内容审核等下游链路，后续调整标签体系时也无需修改向量调用方的接口。

代码16.34给出了一个基于 gRPC 和 Qwen2.5-VL 的实现。`VisionEngine` 负责模型加载、图片恢复和推理，`ImageEmbeddingService` 负责请求校验、调用模型和组织响应，标注仓库则负责持久化模型生成的结构化标签。多模态大模型在服务启动时加载，后续请求复用同一模型实例，并通过批量推理提高 GPU 利用率。

多模态模型推理 RPC 服务接收 448×448 图片的 CHW 展平数组，并通过 $(x+1) \times 127.5$ 将归一化像素恢复为 RGB 取值。模型推理会同时产生两类结果：一条路径根据提示词生成垂类、置信度和主题标签；而另一条路径提取视觉 token，并根据每张图片的网格长度切分后进行均值池化，最终得到固定长度的图片向量。

代码 16.34: Qwen2.5-VL 图片向量 RPC 服务

```
# 对应文件：qwen_vl_rpc_service.py (ImageEmbeddingService)
from __future__ import annotations

import json
import os
import threading
from concurrent import futures
from dataclasses import asdict, dataclass
from pathlib import Path

import grpc
import numpy as np
import torch
from PIL import Image
from transformers import AutoProcessor, Qwen2_5_VLForConditionalGeneration

import image_embedding_service_pb2 as image_api
import image_embedding_service_pb2_grpc as image_rpc


IMAGE_SIDE = 448
MAX_BATCH_SIZE = 8
PROMPT = """仅依据图片可见画面判断垂类。从[游戏、才艺、户外、体育、知识、购物、美食、其他]中选择 ↩
    ↪ category，给出 confidence 和不超过三个 tags。只输出 JSON。"""


@dataclass(frozen=True)
class Annotation:
    category: str
    confidence: float
    tags: list[str]
```

```
class JsonlAnnotationRepository:
    def __init__(self, output_path: Path) -> None:
        self.output_path = output_path
        self.lock = threading.Lock()

    def write_batch(self, rows: list[Annotation]) -> None:
        payload = "".join(json.dumps(asdict(row), ensure_ascii=False) + "\n" for row in rows)
        with self.lock, self.output_path.open("a", encoding="utf-8") as output:
            output.write(payload)
            output.flush()
            os.fsync(output.fileno())


class VisionEngine:
    def __init__(self, model_id: str) -> None:
        dtype = torch.bfloat16 if torch.cuda.is_available() else torch.float32
        self.processor = AutoProcessor.from_pretrained(model_id)
        self.model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
            model_id,
            torch_dtype=dtype,
            device_map="auto" if torch.cuda.is_available() else None,
        ).eval()

    def decode(
        self,
        values: list[float],
        image_dim: int,
        batch_size: int,
    ) -> list[Image.Image]:
        expected = 3 * IMAGE_SIDE * IMAGE_SIDE
        if image_dim != expected or len(values) != image_dim * batch_size:
            raise ValueError("inconsistent image dimensions")
        array = np.asarray(values, dtype=np.float32).reshape(batch_size, 3, IMAGE_SIDE, IMAGE_SIDE)
        pixels = np.clip((array + 1.0) * 127.5, 0, 255).astype(np.uint8)
        return [
            Image.fromarray(row.transpose(1, 2, 0), "RGB")
            for row in pixels
        ]

    @torch.inference_mode()
    def infer(self, images: list[Image.Image]) -> tuple[list[Annotation], list[list[float]]]:
        messages = [
            {
                "role": "user",
                "content": [
                    {"type": "image", "image": image},
                    {"type": "text", "text": PROMPT},
                ],
            }
```

```
            for image in images
        ]
        prompts = [
            self.processor.apply_chat_template([message], tokenize=False, add_generation_prompt=True)
            for message in messages
        ]
        inputs = self.processor(
            text=prompts,
            images=images,
            padding=True,
            return_tensors="pt",
        ).to(self.model.device)

        tokens = self.model.generate(**inputs, max_new_tokens=128, do_sample=False)
        answers = self.processor.batch_decode(tokens[:, inputs.input_ids.shape[1]:], skip_special_tokens=True)

        visual = self.model.visual(inputs.pixel_values, grid_thw=inputs.image_grid_thw)
        lengths = (inputs.image_grid_thw[:, 0] * inputs.image_grid_thw[:, 1] * inputs.image_grid_thw[:, 2]).tolist()
        vectors = [
            part.float().mean(dim=0).cpu().tolist()
            for part in torch.split(visual, lengths, dim=0)
        ]
        return [self._parse(answer) for answer in answers], vectors

    @staticmethod
    def _parse(text: str) -> Annotation:
        try:
            data = json.loads(text[text.find("{"):text.rfind("}") + 1])
            return Annotation(
                category=str(data["category"]),
                confidence=float(data["confidence"]),
                tags=[str(tag) for tag in data.get("tags", [])[:3]],
            )
        except (KeyError, TypeError, ValueError, json.JSONDecodeError):
            return Annotation("其他", 0.0, [])


class ImageEmbeddingService(image_rpc.ImageEmbeddingServiceServicer):
    def __init__(self, engine: VisionEngine, annotations: JsonlAnnotationRepository) -> None:
        self.engine = engine
        self.annotations = annotations

    def Get(self, request, context):
        if not 1 <= request.batch_size <= MAX_BATCH_SIZE:
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, "invalid batch_size")
```

```
            return None

        try:
            images = self.engine.decode(list(request.image_list), request.image_dim, request.↩
                ↪ batch_size)
            labels, vectors = self.engine.infer(images)
        except (ValueError, OSError, RuntimeError) as error:
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, str(error))
            return None

        self.annotations.write_batch(labels)
        return image_api.ImageEmbeddingResponse(
            image_embedding=[value for vector in vectors for value in vector],
            emb_dim=len(vectors[0]) if vectors else 0,
            batch_size=len(vectors),
        )


def serve() -> None:
    server = grpc.server(
        futures.ThreadPoolExecutor(max_workers=16),
        options=[
            ("grpc.max_receive_message_length", 64 * 1024 * 1024),
            ("grpc.max_send_message_length", 64 * 1024 * 1024),
        ],
    )
    service = ImageEmbeddingService(
        VisionEngine("Qwen/Qwen2.5-VL-7B-Instruct"),
        JsonlAnnotationRepository(Path("image_annotations.jsonl")),
    )
    image_rpc.add_ImageEmbeddingServiceServicer_to_server(service, server)
    server.add_insecure_port("[::]:50051")
    server.start()
    server.wait_for_termination()
```

上述代码实现的关键在于保持批量输入、模型输出以及原始图片之间的顺序对应。服务首先检查 batch size 和图片数组长度，校验通过后再恢复图片并执行推理；在推理成功后，标注和向量才会写入或返回。视觉编码产生的 token 数由图片网格决定，因此代码根据 `image_grid_thw` 计算每张图片对应的 token 数，再逐段池化，保证返回的第 i 个向量仍对应请求中的第 i 张图片。

当标签解析失败时，服务返回默认的“其他”、零置信度和空标签，但不会影响向量生成。而当输入维度错误、图片恢复失败或模型执行异常时，服务则会直接终止整个 RPC 请求，不写入标注也不返回部分向量。这样可以避免不完整的 batch 结果进入下游，同时保留单条标签解析异常的容错空间。

16.6 物品索引特征服务

物品索引特征服务将内容、状态、统计和向量等分散结果组织为在线可读取的物品文档与候选入口。它要解决的并非单个字段如何存储，而是让召回和排序能在有限时延内获得一致的物品视图。如果在线请求逐个扫描全部物品，再分别调用内容、状态和统计服务计算特征，就无法在规定时延内完成召回和排序。因此，需要预

先将这些结果写入可按键读取的索引。

读到这里，读者可能会问：

笔记 在线推荐已经有内容、状态和统计服务，为什么还要额外维护一份索引？

答案在于在线请求没有足够的时间逐个扫描全部物品，再分别调用各类服务计算特征。召回、粗排和排序都需要在极短时间内从大量候选中完成筛选，因此必须提前将分散的数据组织成可按 key-value 快速读取的索引结构。索引的时效性同样重要。当索引过于陈旧时，新物品可能无法及时进入召回，而已经下线的物品仍可能出现在候选集中，排序模型也可能读取到缺失或过期的特征。

因此，索引构建不仅需要整合多个特征 RPC，还需要处理 Redis 中按物品、作者和时间窗口维护的键值数据。在构建过程中，下游服务可能出现超时、部分返回或缓存值损坏，这些情况需要与正常的冷启动和字段缺失区分开来。尤其是损坏值不能简单地当作空值写入索引，否则一次错误的构建或发布就可能将异常数据扩散到大量线上请求。

以一场直播为例，正排索引（forward index）可写成“直播 ID → 标题、类目、作者、开播状态、内容向量和互动计数”的映射。当排序服务已经拿到候选直播 ID 时，便可据此一次读出完整的特征信息。倒排索引（inverted index）则反向组织数据，例如“类目 = 美食且地域 = 北京 → [直播 18、直播 42、直播 91]”或“标签 = 露营 → [直播 7、直播 63、直播 84]”。线上召回根据条件从这样的倒排拉链取得一批候选 ID，再用正排索引补齐属性并交给后续模型。正排索引回答“这个物品是什么”，倒排索引回答“满足某个条件的物品有哪些”；二者将内容、状态、统计和缓存中的离线数据转为在线服务可直接使用的输入。

本章节将详细介绍物品索引特征服务的构建与维护，包括离线 Runner、批量正排更新、批量倒排更新以及实时更新等服务的职责与协作方式。其中，通用离线 Runner 负责统一处理任务调度、数据分片、进度管理和失败重试。在此基础上，批量正排更新服务以独立版本重建正排索引，批量倒排更新服务再根据正排索引生成有序的候选入口，而实时更新服务则消费索引更新触发消息，仅重建发生变化的少量物品。索引全量更新与实时更新二者相互配合，共同维护一致且持续更新的在线物品字段信息。

16.6.1 离线 Runner 服务

离线 Runner 为索引、图和训练等业务提供统一的定时批处理执行框架。以索引全量构建为例，调度器在计划时间创建一次运行，并确定本次运行统一使用的输入截止时间，再将构建任务拆分成多个可以独立处理的分片。工作线程领取分片后调用对应的业务处理器，处理结果先写入本次运行的暂存版本；只有处理成功后，分片才会被标记为完成，发生异常则记录失败原因并等待后续重试。

代码16.35中的运行记录保存运行 ID、输入截止时间和输出版本，用于标识一次完整的离线构建。一次运行可以包含多个独立分片，每个分片处理完成后再记录对应的状态和重试次数。`execute` 是一次运行的主要入口，负责不断获取待处理的分片并执行对应的业务逻辑；所有分片成功完成后，才生成本次运行的输出版本。统一的输入截止时间则保证同一次运行中的所有分片使用相同的数据范围，避免不同分片因为处理时间不同而读取到不一致的数据。

代码 16.35: 离线 Runner 的通用执行主干

```
// 对应文件：ItemIndexService.java（OfflineRunner / OfflineRun）
public static class OfflineRun {
    public final String id;
    public final java.time.Instant inputCutoff;
    public final String outputVersion;
    public OfflineRun(String id, java.time.Instant inputCutoff, String outputVersion) {
        this.id          = id;
        this.inputCutoff = inputCutoff;
        this.outputVersion = outputVersion;
    }
```

```
}

public static class OfflineRunner {
    private final DualBufferIndex forwardIndex;
    private final OrderedInvertedIndex invertedIndex;
    /** 离线全量索引构建入口：从快照读取物品特征，填充双 Buffer，然后原子切换。 */
    public OfflineRun run(List<ItemFeature> snapshot) {
        String runId = "run-" + System.currentTimeMillis();
        java.time.Instant cutoff = java.time.Instant.now();
        // 1. 全量写入写 Buffer
        forwardIndex.reload(snapshot);
        // 2. 重建倒排索引
        for (ItemFeature f : snapshot) {
            Set<String> tags = deriveTags(f.score, f.attributes);
            invertedIndex.upsert(f.itemId, f.score, tags);
        }

        String version = "v" + cutoff.getEpochSecond();
        System.out.printf("[OfflineRunner] run=%s version=%s items=%d%n",
                runId, version, snapshot.size());
        return new OfflineRun(runId, cutoff, version);
    }

    private Set<String> deriveTags(double score, Map<String, String> attributes) {
        Set<String> tags = new HashSet<>();
        String cat = attributes.getOrDefault("category", "");
        if (!cat.isBlank()) tags.add("cat:" + cat);
        if (score > 0.8) tags.add("top_score");
        return tags;
    }

    public OfflineRunner(DualBufferIndex forwardIndex,
                         OrderedInvertedIndex invertedIndex) {
        this.forwardIndex = forwardIndex;
        this.invertedIndex = invertedIndex;
    }
}
```

上述代码展示了离线 Runner 的基本执行流程：先确定一次运行对应的数据和版本，再完成全量数据写入和索引构建，最后生成可供后续发布的版本。在实际系统中，分片处理完成并不意味着结果可以立即被线上读取。业务处理器应先将结果写入带版本的暂存区域，待所有必要分片完成并通过数据校验后，再一次性切换线上读取的版本。

失败处理同样需要与正常的空值区分开来。某个分片在读取下游 RPC 或缓存时发生异常，应保留其输入范围、尝试次数和错误原因，并将分片标记为失败，而不是用空字段生成文档后继续发布。这样既可以通过重试恢复临时故障，也能让人工排查准确定位问题，避免错误结果随着一次全量发布扩散到线上索引。

16.6.2 批量正排索引更新服务

批量正排索引更新服务会周期性地重建完整的物品正排信息。服务按照地域、内容类型或物品 ID 范围枚举当前有效物品，批量调用内容、状态和统计特征服务，并读取 Redis 中的窗口计数、开关以及补充属性等，最终生成完整的物品正排信息。批量更新可以重新计算整个索引，因此能够处理规则调整、历史数据修复、实时消息遗漏以及长时间服务异常等情况。这些问题仅依靠实时更新很难发现和补齐。在构建完成并通过校验之后，新的索引才会被发布为线上使用的版本。

批量构建通常采用双 Buffer 的形式进行发布：线上请求始终读取当前版本，新的索引则在另一个 Buffer 中完整构建。在构建完成后，服务只需切换当前读取的 Buffer，读请求就能从新版本获取数据，而不会在构建过程中读到不完整的结果。代码16.36展示了这一过程。`DualBufferIndex` 使用 `AtomicReference` 保存当前读 Buffer，`reload` 先将完整数据写入新的 Map，再通过 `set` 一次性切换读指针，因此切换过程中不需要暂停读请求。

代码 16.36: 双 Buffer 全量索引构建与发布

```
// 对应文件：ItemIndexService.java (DualBufferIndex)
public static class ItemFeature {
    public final long itemId;
    public final double score;
    public final Map<String, String> attributes;
    public ItemFeature(long itemId, double score, Map<String, String> attributes) {
        this.itemId   = itemId;
        this.score    = score;
        this.attributes = Map.copyOf(attributes);
    }
}

public static class DualBufferIndex {
    /** 当前读 Buffer（原子引用实现零拷贝切换） */
    private final AtomicReference<Map<Long, ItemFeature>> readBuffer =
            new AtomicReference<>(new ConcurrentHashMap<>());
    private volatile Map<Long, ItemFeature> writeBuffer = new ConcurrentHashMap<>();
    /**
     * 全量加载新索引到写 Buffer，然后原子切换。
     * 切换期间读请求仍可访问旧 Buffer，无锁等待。
     */
    public void reload(List<ItemFeature> newData) {
        Map<Long, ItemFeature> newBuffer = new ConcurrentHashMap<>();
        for (ItemFeature f : newData) newBuffer.put(f.itemId, f);
        writeBuffer = newBuffer;
        // 原子切换：旧 Buffer 由 GC 回收，不影响正在读取的线程
        readBuffer.set(newBuffer);
    }

    public ItemFeature get(long itemId) { return readBuffer.get().get(itemId); }
    public int size() { return readBuffer.get().size(); }
}
```

上述代码的核心是先构建完整的新 Buffer，再切换线上读取的 Buffer。这样，即使全量构建过程中出现依赖异常，当前线上版本也不会受到影响。在实际构建时，内容特征 RPC、状态服务或者 Redis 都可能出现超时、部

分返回、数据损坏等问题。对于尚无互动数据的新物品，可以使用预设值并标记为冷启动。而当关键字段缺失、依赖调用失败或数据明显异常时，则应该让对应物品或分片构建失败，而不能简单填充零值后继续发布。

在生产环境中，全量构建通常会拆成多个分片并行处理，因此还需要避免多个任务同时修改或发布同一个版本。不同分片可以并行写入，但版本的创建、构建完成和最终切换需要受到统一控制，避免一个任务清空或覆盖另一个任务正在构建的数据。对于已经完成的版本，还应确保后续的实时变更能够正确衔接，避免全量快照发布后遗漏这段时间产生的新变化。双 Buffer 的作用正是将“构建新版本”和“线上读取旧版本”隔离开，使全量更新可以在不影响线上读请求的情况下完成，最终再通过一次版本切换让新数据生效。

16.6.3 批量倒排索引更新服务

倒排索引可以看作正排索引的另一种访问方式。正排索引根据物品 ID 查找完整属性，而倒排索引则根据类目、标签或地域等条件快速找到符合条件的物品。例如，正排记录“直播 42”的类目为美食、地域为北京，那么“类目 = 美食”和“地域 = 北京”对应的倒排列表中都应包含直播 42。召回服务可以先通过这些倒排列表获得候选物品，再由粗排和排序根据物品 ID 获取正排特征。因此，倒排索引的核心工作就是从正排数据中提取可检索属性，为每个属性维护一个按分数排序的候选列表。

代码16.37给出了一个简单的实现。`upsert` 更新物品时，会先根据 `forwardMap` 找到该物品原来的标签，并从对应倒排列表中删除旧记录，再写入新的标签和排序分数。这样可以处理物品修改类目、标签变化等情况，而不会让旧属性继续保留在倒排索引中。每个倒排列表最多保存 10,000 个物品，查询时再从中取出所需的 topK 结果。对于过于宽泛的检索条件，如果候选数量长期超过这一上限，则需要进一步增加检索条件，或者将候选交给后续粗排继续筛选。

代码 16.37: 批量构建有序倒排索引

```
// 对应文件：ItemIndexService.java (OrderedInvertedIndex)
public static class InvertedEntry implements Comparable<InvertedEntry> {
    public final long itemId;
    public final double sortScore;
    public InvertedEntry(long itemId, double sortScore) {
        this.itemId = itemId; this.sortScore = sortScore;
    }

    @Override
    public int compareTo(InvertedEntry o) { return Double.compare(o.sortScore, sortScore); }
}

public static class OrderedInvertedIndex {
    /** tag -> sorted set of InvertedEntry（降序） */
    private final Map<String, TreeSet<InvertedEntry>> invertedMap = new ConcurrentHashMap<>();
    /** itemId -> set of tags（用于更新时删除旧条目） */
    private final Map<Long, Set<String>> forwardMap = new ConcurrentHashMap<>();
    private static final int MAX_PER_TAG = 10_000;
    public void upsert(long itemId, double score, Set<String> tags) {
        Set<String> oldTags = forwardMap.get(itemId);
        if (oldTags != null) for (String tag : oldTags) removeFromInverted(tag, itemId);
        // removeFromInverted(tag, itemId): 从 invertedMap[tag] 的 TreeSet 中移除 itemId 对应条目
        for (String tag : tags) {
            TreeSet<InvertedEntry> bucket =
                    invertedMap.computeIfAbsent(tag, k -> new TreeSet<>());
```

```
            bucket.add(new InvertedEntry(itemId, score));
            while (bucket.size() > MAX_PER_TAG) bucket.pollLast();
        }

        forwardMap.put(itemId, new HashSet<>(tags));
    }

    public void remove(long itemId) {
        Set<String> tags = forwardMap.remove(itemId);
        if (tags != null) for (String tag : tags) removeFromInverted(tag, itemId);
    }

    /** 按分数降序返回 topK 结果 */
    public List<InvertedEntry> query(String tag, int topK) {
        TreeSet<InvertedEntry> bucket = invertedMap.get(tag);
        if (bucket == null || bucket.isEmpty()) return List.of();
        List<InvertedEntry> result = new ArrayList<>();
        int count = 0;
        for (InvertedEntry e : bucket) { if (count++ >= topK) break; result.add(e); }
        return result;
    }
}
```

从上述代码可以看出，倒排索引与正排索引来源于同一批数据，因此两者需要使用相同的输入截止时间和输出版本。只有通过正排校验的物品才能进入倒排构建，在发布时也应该对两边进行抽样检查，确认倒排得到的物品 ID 能够在正排中正常读取。否则，线上可能出现倒排已经返回候选物品，但在正排中没有对应的信息，或者两边读取到不同版本数据的情况。

16.6.4 实时索引更新服务

实时索引更新服务通常会消费实时索引更新的 Kafka Topic。当内容理解完成、互动计数发生变化、物品上线或下线时，上游会向 Kafka Topic 中写入物品 ID 以及变化原因，通知实时索引服务重新处理对应的物品。由于全量索引构建需要较长的时间，如果只等待下一轮全量任务，新上线的物品和最新的特征就无法及时进入线上索引。实时更新通常只处理发生变化的少量物品，因此可以在全量任务之间及时补充这些变化。代码16.38展示了实时索引更新的核心流程，如下所示。

代码 16.38: 实时重建物品索引

```
// 对应文件：ItemIndexService.java (RealtimeIndexUpdater)
// [MOCK] MockKafkaConsumer 为内存队列；工业落地时替换为真实 KafkaConsumer。
public static class RealtimeIndexUpdater {
    private final DualBufferIndex forwardIndex;
    private final OrderedInvertedIndex invertedIndex;

    public RealtimeIndexUpdater(DualBufferIndex forwardIndex,
                                OrderedInvertedIndex invertedIndex) {
        this.forwardIndex = forwardIndex;
        this.invertedIndex = invertedIndex;
    }
```

```
    /**
     * 消费一批 IndexUpdateTrigger 消息并同步更新正排与倒排索引。
     * 1. 提取并去重本批物品 ID
     * 2. 重新加载每个物品的当前特征文档（模拟 RPC 读取）
     * 3. 不可见/已下线物品 → 同时从正排和倒排删除
     * 4. 可见物品 → 写入正排，并更新倒排
     * 异常直接向上传播，调用方（Kafka 框架）负责提交或回退位点。
     */
    public void consume(List<IndexUpdateTrigger> triggers) {
        // Step 1: 提取并去重物品 ID
        Set<Long> itemIds = new LinkedHashSet<>();
        for (IndexUpdateTrigger t : triggers) {
            if (t.itemId > 0) itemIds.add(t.itemId);
        }

        if (itemIds.isEmpty()) return;
        // Step 2: 批量重新加载当前特征文档
        for (long itemId : itemIds) {
            ItemFeature doc = loadCurrentDoc(itemId);
            if (doc == null || isOffline(doc)) {
                // Step 3: 不可见或已下线 → 从正排和倒排同时删除
                forwardIndex.remove(itemId);
                invertedIndex.remove(itemId);
            } else {
                // Step 4: 写入正排，并更新倒排
                forwardIndex.upsert(doc);
                Set<String> tags = deriveTags(doc.score, doc.attributes);
                invertedIndex.upsert(doc.itemId, doc.score, tags);
            }
        }
    }

    private ItemFeature loadCurrentDoc(long itemId) { /* 调用内容/状态/统计 RPC */ }
    private boolean isOffline(ItemFeature doc) { /* 检查可见性字段 */ }
    private Set<String> deriveTags(double score, Map<String, String> attributes) { /* ... */ }
}
```

从上述代码可以看出，实时索引更新的核心是以变化事件触发索引重建过程，而不是直接使用事件中的旧数据来更新索引。实时索引更新消费者在收到事件后会重新读取物品当前的完整状态，因此即使短时间内连续产生多次变化，最终写入索引的仍然是最新结果。正排和倒排也会随着同一次重建一起更新，使线上候选与物品特征保持一致。对于下线或不可见的物品，索引可以及时删除。而其他物品则只需重新构建自身数据，无需等待下一轮全量更新过程。

16.7 图存储服务

图关系可以补充倒排和向量召回，为推荐系统提供基于共同消费、关注和内容关联的候选 Item。在构建图数据时，离线任务根据用户历史反馈行为以及内容数据来构建图的边，实时的任务可以补充新产生的关系。在

构建完图数据需要查询时，在线服务会按照限定的跳数和候选数量来查询图上的关系。本节首先介绍图数据的构建与查询方式，再说明实时写入和在线读取的基本流程。

16.7.1 图数据构建与在线查询

图存储服务将推荐系统中的用户、物品、作者、类目等实体及其关系保存为可查询的数据。例如，用户观看物品、关注作者、物品属于类目，以及两个物品被同一用户先后消费，都可以表示为图中的边。边通常还需要记录关系类型、发生时间、权重和来源版本。与提前为每种召回策略生成候选列表不同，图存储保存的是这些关系本身，因此可以根据不同的起点和关系类型查询一跳或两跳邻居。倒排索引按照属性查找候选，向量索引按照相似度查找近邻，而图存储更适合根据已有关系获取候选。

以 Online Swing 为例，离线或流式任务先在用户近期行为窗口中统计物品共现关系，再根据共同用户数和行为窗口计算 Item–Item 权重，形成 $i \xrightarrow{\text{SWING}} j$ 边。在线请求提供 Trigger 物品 i 和候选数 K，图服务返回权重较高的 K 个邻居 j，召回服务再结合内容、地域等条件进行过滤。此外，用户最近消费过的物品也可以作为起点，进而查询其 Swing 邻居并合并候选。

图的边通常同时通过离线和实时两条链路进行写入。离线任务根据历史用户反馈行为重新计算指定范围内的节点和边，适合处理历史数据补齐、规则调整等情况。而实时任务通过消费 Action Log，在新行为发生之后及时补充对应的关系。以实时路径为例，Action Log 中的一条反馈至少包含事件 ID、用户 ID、物品 ID 和事件时间。消费者从 Kafka 读取反馈后，维护每个用户最近的 50 个物品，并根据这段窗口中的历史物品生成新的 Swing 边。

代码16.39给出了这一过程的 C++ 简化实现。其中 `run_once` 方法拉取一批消息，成功写入图后再更新用户近期窗口。如果写入失败，则停止处理当前批次，等待下一次消费。示例代码使用内存图进行存储，在生产环境中可以将相同的节点和边写入持久化的数据库或图数据库中。

代码 16.39: 消费 Action Log 并写入 Online Swing 图边

```
// 对应文件：graph_kafka_consumer.h (ActionLogGraphConsumer)
// ActionLogGraphConsumer — 对应 graph_kafka_consumer.h
// [MOCK] MockKafkaConsumer 为内存队列；工业落地时替换为 librdkafka KafkaConsumer。

class ActionLogGraphConsumer {
public:
    static constexpr int MAX_WINDOW_SIZE = 50;

    explicit ActionLogGraphConsumer(
        MockKafkaConsumer& consumer,
        InMemorySwingGraphStore& graph_store)
        : consumer_(consumer), graph_store_(graph_store), running_(false) {}

    // 处理一批消息：拉取 → 写图边 → 更新用户近期窗口 → 提交位点
    void run_once() {
        auto records = consumer_.poll(/*timeout_ms=*/500);
        for (const auto& action : records) {
            auto& recent_items = recent_items_by_user_[action.user_id];
            try {
                // 写入节点 + 双向 SWING 边
                graph_store_.write(action, recent_items);
                // 追加物品到用户窗口，超长时移除头部
                recent_items.push_back(action.item_id);
                if (static_cast<int>(recent_items.size()) > MAX_WINDOW_SIZE) {
```

```
                recent_items.pop_front();
            }

            // commit_sync: 成功写库后才提交 Kafka 位点（[MOCK] 此处 no-op）
        } catch (const std::exception& e) {
            std::cerr << "[Consumer] write failed: " << e.what() << "\n";
            break; // 失败时停止本批处理，等待下次 poll 重试
        }
    }
}

// 便捷方法：处理所有消息直到队列为空
void drain() {
    while (!consumer_.empty()) { run_once(); }
}

private:
MockKafkaConsumer& consumer_;
InMemorySwingGraphStore& graph_store_;
std::unordered_map<int64_t, std::deque<int64_t>> recent_items_by_user_;
};
```

16.7.2 图数据库的技术选型

图存储的选型首先要看线上到底需要什么样的查询，而不是看到数据中存在关系就直接选择图数据库。如果业务主要是固定的一跳邻接查询，使用普通宽表或 KV 存储往往已经足够。只有当查询需要比较灵活的关系组合、属性过滤或者多跳路径时，专用图数据库的价值才会更加明显。因此，实际选型除了考虑图规模，还需要同时看查询方式、扩展能力以及故障恢复成本。

对于单机或中等规模、查询模式比较灵活的场景，Neo4j 的属性图和 Cypher 比较容易表达复杂的关系查询。其集群可以配置多个 Primary 来承担事务处理，也可以增加 Secondary 分担只读请求。Primary 参与写事务的故障容错，而 Secondary 通过异步复制扩展读能力。因此，在读 Secondary 时可能暂时看不到刚提交的数据 [**neo4j_cluster**]。这类架构比较适合写入规模可控、同时存在较多图查询需求的服务。需要注意的是，增加副本并不能自动解决跨地域部署带来的网络延迟，具体的读写路由和一致性要求仍然需要由服务层结合业务场景确定。

当图规模进一步扩大，需要将顶点和邻接关系分布到多台机器时，可以考虑 NebulaGraph 这类分布式图数据库。它将图数据分布在多个 Storage 节点上，由 Graph 服务负责执行查询，因此可以通过增加节点扩展容量和查询能力 [**nebula_manual**]。对于 Online Swing 这类推荐场景，真正需要关注的是查询是否跨越多个分区：一跳查询通常比较容易控制，而多跳查询会随着访问分区数量和返回边数增加而产生更多网络开销。热点用户和热门物品也可能形成高连接度顶点，因此实际部署时需要结合分区方式、热点分散和 P99 延迟进行压测，而不能只看单次查询的平均耗时。

JanusGraph 则采用了不同的思路。图模型和 Gremlin 查询由 JanusGraph 提供，而底层数据可以交给 Cassandra、HBase、ScyllaDB 等存储系统保存，并可配合外部索引处理属性查询 [**janusgraph_backends**]。这种方式的优势是可以复用已有的分布式存储、扩容和备份体系，比较适合团队已经具备成熟存储基础设施的场景。代价是系统由多个组件共同完成一次查询，图层、底层存储以及可能使用的外部索引都需要分别考虑容量、超时和恢复。Cassandra 按分区键组织数据，副本数和跨数据中心复制由 keyspace 配置 [**cassandra_overview**]；HBase 按行键将数据划分到不同 Region，适合按行键访问和批量处理 [**hbase_reference**]；ScyllaDB 与 Cassandra 保持查询协议兼

容，也采用类似的分区式数据组织方式 [scylladb_overview]。因此，这类方案可以充分利用已有的分布式存储能力，但一次跨分区的图查询可能涉及多次远端读取，实际延迟仍需要结合数据模型和访问模式进行控制。

如果线上只需要类似 Online Swing 的“一跳获取最近 K 个 Item–Item 邻居”，则没有必要为了图关系本身引入通用图遍历能力。此时可以直接使用按 `source_item_id` 分区、按 `weight` 排序的宽表或 KV 邻接表，将每个源物品的候选关系直接存下来。这样的实现更容易控制单次查询的数据量和延迟，也便于水平扩展。代价是查询模式一旦变化，例如增加新的边类型或需要多跳关系，就需要重新设计表结构和查询逻辑。因此，图数据库与普通邻接表并不存在绝对的优劣，更重要的是存储结构是否与线上查询保持一致。

表16.3从查询方式、延迟来源和服务侧需要承担的工作三个方面总结这些方案。

表 16.3: 图存储技术选型与服务责任

选择	适合的查询与扩展方式	延迟和陈旧读来源	高可用、备份与服务层责任
Neo4j 集群	灵活属性图、模式匹配；Primary 写入，Secondary 扩展只读	写入等待 Primary 复制确认；Secondary 异步复制，可能读到旧边	多 Primary 和备份策略需部署配置；服务选择读路由、超时和版本检查
NebulaGraph 集群	大图邻接查询；顶点和边按分区分布到 Storage 节点	跨分区多跳、热点顶点和返回边数增加耗时	副本数与故障域需显式配置；服务限制跳数、邻居数并验证恢复水位
JanusGraph 加后端存储	固定图模型、复用 Cassandra/HBase 等分布式后端	图层、外部索引和后端的多次远端读取叠加	底层存储负责副本和备份；服务协调图版本、索引版本与恢复校验
宽表或 KV 邻接表	固定一跳 Item–Item 查询；按源点分区横向扩展	单分区读快；热点源点和候选过长仍会拖慢读取	由表或 KV 集群提供副本与备份；服务自行维护边类型、排序和删除语义

16.7.3 图数据库封装为在线服务

无论底层使用哪种图存储，在线召回服务都不应直接依赖具体的查询语言和表结构，而应通过统一的 API 获取邻居。图存储服务在这一层控制查询规模，同时完成物品可见性过滤和异常降级，使上层只需要关心“给定物品需要哪些候选”。代码16.40给出了一跳邻居查询的实现：请求携带 Trigger 物品、边类型、候选数和最低图版本，服务读取有限数量的邻边，过滤当前不可见的物品，并在查询失败或读取到过旧版本时返回降级结果。当召回服务收到降级结果之后，可以继续使用其他召回通道，而不必让一次图查询故障影响整个推荐请求。

代码 16.40: 面向在线召回的一跳图邻居服务

```
// 对应文件：graph_neighbor_service.h (GraphNeighborService)
// GraphNeighborService — 对应 graph_neighbor_service.h
// ItemVisibility 为可替换接口，封装物品可见性判断（下线、审核等）。

class GraphNeighborService {
public:
    static constexpr int MAX_RESULT_SIZE = 200;
    static constexpr int MAX_SCAN_SIZE = 1000;
    static constexpr int READ_TIMEOUT_MS = 20;
        InMemorySwingGraphStore& store,
        const ItemVisibility& visibility)
        : store_(store), visibility_(visibility) {}

    // 一跳邻居查询：根据种子物品、边类型和候选数返回可见邻居
```

```
    NeighborResult neighbors(const NeighborQuery& query) const {
        if (query.limit > MAX_RESULT_SIZE) {
            return NeighborResult::degraded_result();
        }

        try {
            // 读取最多 scan_limit 条边（候选数的 5 倍，以便过滤后仍有足够结果）
            int scan_limit = std::min(MAX_SCAN_SIZE, query.limit * 5);
            GraphRead graph_read = store_.read_neighbors(
                query.source_item_id, query.edge_type, scan_limit);

            // 版本检查：副本版本低于请求最低版本时降级
            if (graph_read.served_graph_version < query.min_graph_version) {
                return NeighborResult({}, /*degraded=*/true,
                                      graph_read.served_graph_version);
            }

            // 过滤不可见物品，累积 limit 个候选后截止
            std::vector<int64_t> item_ids;
            item_ids.reserve(query.limit);
            for (const auto& edge : graph_read.edges) {
                if (visibility_.is_visible(edge.target_item_id)) {
                    item_ids.push_back(edge.target_item_id);
                    if (static_cast<int>(item_ids.size()) == query.limit) break;
                }
            }

            return NeighborResult(std::move(item_ids), /*degraded=*/false,
                                  graph_read.served_graph_version);
        } catch (const std::exception&) {
            // 超时、数据库故障：返回空候选 + 降级标记
            return NeighborResult::degraded_result();
        }
    }

private:
    InMemorySwingGraphStore& store_;
    const ItemVisibility& visibility_;
};
```

上述代码的关键是把图查询限制在可控范围内，先限制候选数和扫描数量，再过滤不可见物品，最终返回满足要求的邻居。如果副本版本过旧，或者查询过程中发生超时、数据库故障，则直接返回降级结果，由其他召回通道继续提供候选。这样既能控制图查询的时延，也不会让单个图存储故障影响整次推荐请求。

16.8 离线训练任务

离线训练任务以离线积累或持续到达的带标签推荐样本为输入，在一段时间内持续计算梯度、更新模型参数，并最终产出能够被线上排序或召回服务加载的模型版本。它关注的并不是一次请求如何立即返回结果，而

是模型如何从大量历史样本中不断学习，以及新参数如何经过验证后安全地替换线上旧版本。要实现这一过程，样本先要从 Kafka 经由特征处理稳定地组成 batch；这些 batch 再由多台机器协同用于参数更新，稠密参数和 Embedding 表也在这一阶段分别保存；形成阶段性训练状态后，checkpoint、模型评估与线上发布决定哪个版本可以投入服务；对混合精度数值问题以及机器或训练进程故障的处理和恢复，则保证这条链路能够持续运行。

16.8.1 Kafka 样本流与数据处理

离线训练的第一步，是让训练进程持续、稳定地获得可直接计算的 batch。模型计算需要连续输入；如果训练线程同时负责从 Kafka 拉取消息、进行 protobuf 反序列化和特征处理，Kafka 的瞬时抖动或单条消息处理时间过长，都会直接传导到 GPU，表现为计算设备等待数据。工业训练系统通常把这些准备工作拆到独立线程中，让训练线程尽量只面对已经组装完成的 batch。对于持续到达的训练样本，这条数据通道通常以 Kafka 流为起点。

Kafka 消费线程从分区中读取消息后，protobuf 解码线程按照样本流定义的字段，将字节内容恢复为标签、类别特征、数值特征、序列特征以及特征版本等信息。消息格式由上游样本流决定，因此训练数据通道需要明确字段的类型、是否必填以及对应的版本。标签通常已经在上游按照统一业务口径生成，训练进程只负责读取和使用，不再重新关联曝光、点击或播放日志，更不会在训练阶段重新推导标签。

在完成字段还原后，特征处理线程进一步把样本转换成模型可以直接消费的表示。稀疏特征通常表示为 slot 和 sign 两个整数，其中 slot 用来区分用户、物品、类目等不同类型的特征，sign 则表示该 slot 下具体的 ID 或离散取值。对于已经写出 slot 和 sign 的样本，读取器主要校验字段类型、特征版本以及编号范围，然后直接使用。如果上游提供的仍是类别、数值或序列字段，则由训练侧根据配置将其转换为对应的 slot 和 sign。前一种方式把更多 CPU 工作前移到样本侧，后一种方式更方便训练阶段调整编码规则。无论采用哪种方案，训练进程处理的都应该是已经定义好的样本，而不是重新拼接原始行为数据。

经过特征处理的样本会进入一个具有容量上限的内存队列，即 channel。组 batch 的线程不断从 channel 中取出样本，将属于同一 batch 的稀疏特征按 slot 整理，并拼接连续的稠密字段，最终形成可直接交给 TensorFlow 或 PyTorch C++ 运行时的数据结构。channel 的容量上限也形成了数据通道中的背压机制：当队列写满时，特征处理线程会自然放慢，Kafka 的消费速度也随之下降，从而避免内存持续增长。当训练结束时，写入线程关闭 channel，组 batch 的线程继续处理剩余样本后再退出。

这条链路上的每个环节都可能成为 GPU 利用率的瓶颈，因此训练任务通常会分别记录 Kafka 拉取、protobuf 解码、特征处理、channel 积压以及 batch 等待所花费的时间。这样才能判断问题究竟出在消息读取、CPU 数据准备，还是模型计算本身，而不是简单地把 GPU 利用率下降归因于模型性能。

为了让任务在中断后仍能沿着原来的数据路径恢复，checkpoint 还需要记录 Kafka 的消费位置。这个位置对应训练循环已经处理到的消息范围，用于恢复时确定需要重新读取和回放的数据，而不是用来描述样本的时间语义、标签定义或特征截止时间。对于持续产生消息的流式训练任务，也通常不会把整个 Kafka 流直接划分成静态的训练集和测试集；需要评估时，可以由独立的样本服务按照固定时间窗口提供评估数据。至此，样本已经能够稳定地从 Kafka 流进入 batch，下一步则是让多个训练进程围绕这些 batch 协同更新模型参数。

代码16.41给出了单机训练循环的 Python 实现骨架，如下所示。

代码 16.41: 单机 DNN 训练循环（PyTorch）

```
# 对应文件: single_machine_train.py
@dataclass
class TrainConfig:
    feature_dim: int    = 64
    max_steps: int      = 100
    batch_size: int     = 256
    hidden_dims: list[int] = field(default_factory=lambda: [256, 128, 64])
    dropout: float      = 0.1
    lr: float           = 1e-3
```

```
    weight_decay: float = 1e-5
    use_amp: bool       = True
    checkpoint_dir: str = "checkpoints/single"
    checkpoint_every: int = 20
    device: str         = "cuda" if torch.cuda.is_available() else "cpu"


def train(config: TrainConfig = None) -> dict:
    """单机训练入口：样本流 → batch → 前向/反向 → checkpoint。"""
    config = config or TrainConfig()
    stream = MockSampleStream(feature_dim=config.feature_dim)
    device = torch.device(config.device)
    model = build_model(config).to(device)
    optimizer = optim.Adam(model.parameters(),
                           lr=config.lr, weight_decay=config.weight_decay)
    criterion = nn.BCEWithLogitsLoss()
    # 混合精度：仅在 CUDA 设备上启用 GradScaler
    scaler = GradScaler() if (config.use_amp and device.type == "cuda") else None

    # 恢复 checkpoint（存在时从上次保存点继续）
    start_step, _ = load_latest_checkpoint(model, optimizer, config.checkpoint_dir)

    last_loss = float("inf")
    for step in range(start_step, config.max_steps):
        batch = stream.next_batch(config.batch_size)
        x = torch.tensor([[s.features] for s in batch],
                         dtype=torch.float32, device=device).squeeze(1)
        y = torch.tensor([[s.label] for s in batch],
                         dtype=torch.float32, device=device).squeeze(1)

        optimizer.zero_grad()
        if scaler is not None:      # AMP 路径
            with autocast():
                logits = model(x).squeeze(1)
                loss   = criterion(logits, y)
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
        else:                       # FP32 路径
            logits = model(x).squeeze(1)
            loss   = criterion(logits, y)
            loss.backward()
            optimizer.step()

        if (step + 1) % config.checkpoint_every == 0:
            save_checkpoint(model, optimizer, step + 1,
                            loss.item(), config.checkpoint_dir)

    return {"final_loss": loss.item(), "steps": config.max_steps}
```

上述代码将从 MockSampleStream（或真实 Kafka 消费者）拉取的样本整理成张量，通过 BCEWithLogitsLoss 计算二分类损失，再分别走混合精度（AMP）或 FP32 两条更新路径。AMP 路径在 autocast() 上下文中以低精度执行前向和损失计算，用 GradScaler 缩放梯度以避免 FP16 下溢，更新后再恢复到 FP32；FP32 路径则直接调用 backward() 和 optimizer.step()。无论哪条路径，每隔 checkpoint_every 步都会把模型权重和优化器状态一起保存，以便从任意中断点恢复。这一单机训练循环构成了分布式训练的基础，后者在此基础上增加进程间梯度同步和多 rank 协调。我们将在下一个小节对分布式训练进行介绍。

16.8.2 分布式训练框架与任务执行

多个训练进程共同训练一个模型时，首先需要解决参数如何同步更新的问题。不同机器上的进程即使都从 Kafka 读取样本并完成反向计算，如果缺少统一的参数更新机制，各 GPU 上的模型权重很快就会出现差异。工业训练通常以 TensorFlow 或 PyTorch 作为计算底座，C++ 负责训练进程的启动、样本读取、batch 准备、模型计算和 checkpoint 保存，Python 则更多用于定义模型结构和训练配置。

训练平台为一次任务分配机器和 GPU，并在每张 GPU 上启动一个训练进程。启动器为进程分配编号，同时提供参与训练的进程地址，使它们能够建立通信连接。TensorFlow 可以将任务组织为训练协调进程、训练进程和参数节点；PyTorch 则可以通过 **torchrun** 启动多个训练进程并完成地址发现 [**pytorch_torchrun**]。训练进程负责 batch 的前向和反向计算，协调进程则负责维护任务状态、训练进度和 checkpoint 信息。在使用参数服务器时，模型参数和优化器状态通常由参数节点统一保存。当训练进程异常退出后，任务可以从最近一次完整 checkpoint 恢复。

对于持续从 Kafka 消费样本的流式训练，不存在一个天然的“训练集遍历完成”时刻，因此通常需要人为划分训练周期，这就是 pass。一个 pass 可以定义为固定数量的 batch，完成后统一进行指标评估、状态记录和 checkpoint 保存，然后继续消费后续样本。pass 过大，异常发现和状态保存会滞后；而 pass 过小，则会增加评估和 checkpoint 带来的额外开销。因此，pass 更适合作为流式训练中组织评估和状态管理的时间尺度，而不是数据集遍历的概念。Kafka 数据流也不会因为 pass 结束而停止。

在确定训练任务如何运行之后，还需要决定参数更新采用同步还是异步方式。在同步训练中，每个进程独立计算自己的 batch 和梯度，随后通过通信操作聚合所有进程的梯度，再使用相同的结果更新模型。这样每轮计算开始时各进程的模型仍保持一致，但整体速度会受到最慢进程和通信延迟的影响。

异步训练则允许进程独立完成 batch 后立即提交梯度，由参数节点更新模型，不需要等待其他进程。这样可以减少同步等待，但不同进程计算梯度时可能使用不同版本的模型。当参数更新速度较快时，某个进程提交的梯度可能已经对应较旧的参数，从而产生 stale gradient。因此，异步训练除了关注吞吐，还需要监控参数版本差以及过旧梯度等情况。

同步训练解决了参数一致性问题，但在 GPU 数量增加后，梯度聚合本身也可能成为新的瓶颈。下一节进一步介绍数据并行训练中常用的梯度通信方式，以及通信开销为什么会限制分布式训练的扩展能力。

16.8.3 同步数据并行与梯度通信

同步数据并行让不同 GPU 处理不同 batch，同时保持相同的模型参数。每个训练进程先独立完成前向和反向计算，再通过 AllReduce 聚合各 GPU 上对应位置的梯度，并将结果同步到所有 GPU。各进程使用相同的平均梯度更新本地模型，因此下一轮计算仍从一致的参数开始。

PyTorch 的 DistributedDataParallel（DDP）将这一过程集成到了训练流程中。每张 GPU 对应一个 DDP 进程，并在启动时获得模型副本。执行 loss.backward() 时，DDP 会随着反向传播逐步产生梯度，并将多个参数的梯度组织成 bucket 进行通信。在 AllReduce 完成后，各进程获得一致的梯度，再由 optimizer.step() 更新本地参数 [**pytorch_ddp**]。DDP 还可以让部分梯度计算与通信重叠：前面的梯度准备好后即可开始传输，GPU 同时继续

计算其他层的梯度。不过最终仍需要等待所有进程完成，因此数据读取慢或计算量较大的进程会拖慢整个训练任务。

随着 GPU 数量的增加，通信开销会逐渐成为训练扩展的限制因素。一次更新除了模型计算，还包含数据读取、主机到 GPU 的传输和梯度同步，因此扩容前需要先确认时间主要消耗在哪个环节。如果瓶颈在数据读取或者 CPU 处理环节，则增加 GPU 的收益会相当有限。而如果梯度通信占据主要时间，则需要进一步检查网络拓扑、batch 大小和通信粒度，并通过梯度累积等方式减少同步频率。

代码16.42给出了一个基于 DDP 的训练 worker。`setup_dist` 初始化进程组，`DistributedDataParallel` 将模型包装为 DDP 模型，使 `loss.backward()` 自动完成梯度同步。训练进程分别从 `DistributedSampleStream` 获取自己的 batch，并在更新前进行梯度裁剪。训练 loss 通过一次额外的 AllReduce 汇总，便于 rank-0 统一打印。checkpoint 只由 rank-0 写入，避免多个进程同时操作同一个文件。

代码 16.42: 分布式 DDP 训练 worker（AllReduce + AMP + Checkpoint）

```
# 对应文件: distributed_train.py (train_worker)
def train_worker(rank: int, world_size: int,
                 config: DistributedTrainConfig) -> None:
    """DDP 训练 worker — 每个进程（rank）运行此函数。"""
    # 1. 初始化进程组（NCCL for GPU, GLOO for CPU mock）
    setup_dist(rank, world_size, config.backend, config.init_method)

    # 2. 设备分配 + 模型 DDP 包装
    device = (torch.device(f"cuda:{rank}")
              if torch.cuda.is_available() and config.backend == "nccl"
              else torch.device("cpu"))
    model = DDP(build_model(config).to(device),
                device_ids=[rank] if device.type == "cuda" else None)
    optimizer = optim.Adam(model.parameters(),
                           lr=config.lr, weight_decay=config.weight_decay)
    criterion = nn.BCEWithLogitsLoss()
    use_amp   = config.use_amp and device.type == "cuda"
    scaler    = GradScaler() if use_amp else None
    stream    = DistributedSampleStream(config.feature_dim, rank)

    for step in range(config.max_steps):
        batch = stream.next_batch(config.batch_size)
        x = torch.tensor([s.features for s in batch],
                         dtype=torch.float32, device=device)
        y = torch.tensor([s.label for s in batch],
                         dtype=torch.float32, device=device)

        optimizer.zero_grad()
        if scaler is not None:
            with autocast():
                loss = criterion(model(x).squeeze(1), y)
            scaler.scale(loss).backward()     # backward 触发 AllReduce
            scaler.unscale_(optimizer)
            # 梯度裁剪: 限制梯度范数，防止异常样本扰动模型
            torch.nn.utils.clip_grad_norm_(model.parameters(),
                                           config.max_grad_norm)
```

```
            scaler.step(optimizer)
            scaler.update()
        else:
            loss = criterion(model(x).squeeze(1), y)
            loss.backward()                    # backward 触发 AllReduce
            torch.nn.utils.clip_grad_norm_(model.parameters(),
                                           config.max_grad_norm)
            optimizer.step()

        # AllReduce loss: 将各 rank 的 loss 求和再取均值，供监控打印
        loss_t   = torch.tensor(loss.item(), dtype=torch.float32, device=device)
        last_loss = all_reduce_mean(loss_t)
        if step % 10 == 0 and rank == 0:
            print(f" step={step:4d} global_loss={last_loss:.4f}")

        # Checkpoint 仅由 rank-0 写入
        if (step + 1) % config.checkpoint_every == 0:
            save_checkpoint_rank0(model.module, optimizer,
                                  step + 1, last_loss, config, rank)

    cleanup_dist()
```

在上述实现中，DDP 将梯度同步隐藏在反向传播的过程中，而训练代码本身不需要显式调用 AllReduce。每个 rank 只处理自己的 batch，AllReduce 后获得相同的平均梯度，再独立更新模型，因此各 GPU 的参数可以保持一致。梯度裁剪用于限制异常 batch 对参数更新的影响，checkpoint 则由 rank-0 统一保存。

同步训练的有效 batch size 等于单卡 batch size 乘以训练进程数。在增加 GPU 后，如果单卡 batch 不变，每次参数更新覆盖的样本数也会增加，学习率、训练步数和 warmup 等配置通常需要重新验证。梯度累积可以在不增加显存的情况下进一步扩大有效 batch，但也会减少参数更新频率，因此需要和训练步数一起调整。

DDP 适合能够完整复制到每张 GPU 的稠密模型，但大规模 Embedding 表通常无法采用相同方式复制。Embedding 的参数量很大，而一次 batch 只会访问其中少量行，将完整表复制到每张 GPU 会浪费大量显存和同步带宽。因此，工程上通常按行号或哈希将 Embedding 表切分到不同参数节点，训练进程只访问当前 batch 需要的参数，并将更新写回对应分片。下一节将进一步介绍这种参数服务器模式，以及稀疏参数为什么需要与稠密网络不同的更新方式。

16.8.4 参数服务器与稀疏参数

参数服务器（Parameter Server，PS）主要用于保存单张 GPU 或单台机器难以容纳的大规模稀疏参数，例如用户和物品的 Embedding 表。工程上通常按照行号范围或哈希结果将 Embedding 表切分成多个分片，每个参数节点负责其中一部分。当训练进程处理 batch 时，通常只需要读取当前 batch 实际访问的 Embedding 行。在反向计算完成后，训练任务再将对应更新写回负责这些行的参数节点。这样既避免了完整复制大表，也减少了不必要的参数传输。

在这种架构中，训练进程负责样本读取和模型计算，参数节点负责 Embedding 及其优化器状态，协调进程则负责训练状态和 checkpoint 管理。TensorFlow 的 `ParameterServerStrategy` 就采用了类似的组织方式，将训练进程与参数节点作为独立服务运行，再由协调机制管理整个训练任务 [**tensorflow_parameter_server**]。与上一节的 DDP 不同，稠密参数通常直接复制到每张 GPU 并通过 AllReduce 同步，稀疏参数则分布在参数节点上，训练时按需远程读取和更新。

Embedding 的分片还需要考虑访问是否均衡。热门物品可能集中在少数分片，使对应参数节点承担更多请

求，最终成为训练瓶颈。此时继续增加训练进程并不能解决问题，反而可能进一步增加热点节点的请求压力。因此，在实际部署中除了关注各分片的内存使用，还需要观察请求吞吐以及尾部延迟，并通过重新划分热点、合并同一 batch 的访问请求等方式降低负载。

总的来说，PS 带来了更大的参数容量，但也增加了网络访问和一致性管理的成本。在训练完成后，稠密参数、Embedding 参数以及对应的优化器状态需要保存为同一个模型版本，恢复训练时才能保持一致。由此可见，稠密参数适合通过 GPU 本地复制和 AllReduce 更新，超大规模稀疏参数更适合采用参数分片和按需访问的方式。

16.8.5 流式模型保存、评估与线上更新

对于持续从 Kafka 消费样本的训练任务，不存在天然的训练结束时刻。如果等任务结束后再导出模型，线上服务可能长期使用旧参数。如果每次参数变化都直接覆盖线上版本，又容易把未经验证的异常带入真实流量。因此，流式训练通常以 pass 为周期生成模型版本：先保存一份完整、可恢复的训练状态，再对该版本进行评估，确认满足发布条件后才进入线上更新流程。

在一个 pass 完成后，训练协调进程需要记录当前的训练状态，包括 pass 编号、全局步数、Kafka 消费位置、模型参数、Embedding 分片和优化器状态等信息。模型的 checkpoint 可以先写入 HDFS 的临时目录，等到所有文件写入并通过完整性检查之后，再切换为正式版本，同时生成对应的文件清单。这样即使保存过程中某个 Embedding 分片失败，也不会被线上服务误认为是一个完整模型。checkpoint 保存也不必阻塞训练主循环，可以由后台任务通过异步保存的方式来完成。在新版本确认可用之前，训练任务始终保留上一份完整 checkpoint 作为恢复点。

模型评估需要使用固定的数据窗口或独立生成的评估集，并将评估结果与对应的模型版本绑定。除了整体 AUC、LogLoss、NDCG 等指标，还应关注不同用户、物品和内容类型上的变化，避免整体指标掩盖局部异常。例如，当 AUC 接近 0.5 时，应先检查标签、特征和数据版本是否发生异常，再与最近稳定版本比较预测分布，而不是直接继续调整训练参数。对于召回模型，还需要确认用户和物品 Embedding 已完整导出，并验证对应索引的 Recall@K。只有当评估数据、指标和模型版本能够对应起来，历史版本之间的结果才具有可比性。

在通过评估后，线上模型推理服务会读取完整模型版本，并根据模型类型加载相应的参数和索引。排序模型主要加载网络参数，召回模型还需要加载用户、物品 Embedding 以及对应的检索索引。具体来说，训练任务将模型版本、特征版本等信息写入消息队列，线上服务在消费消息之后先在备用实例中加载新版本的模型参数，并使用真实请求格式进行验证。在确认模型、特征以及索引等能够正常工作之后，再将流量切换到新实例。如果模型版本无法匹配或者出现加载失败等问题，则继续使用当前稳定版本的模型参数进行线上推理。

16.8.6 混合精度、异常指标与故障回滚

随着模型规模和训练数据量增加，显存容量和 GPU 计算吞吐都会成为训练效率的限制。混合精度训练让大部分矩阵运算使用 FP16 或 BF16，从而减少显存占用，并在支持的硬件上提高计算吞吐。模型参数、优化器状态以及部分对数值精度敏感的计算仍可以保留 FP32。这样既可以在相同显存下使用更大的 batch，也能缩短单步训练时间。

混合精度带来的主要问题是数值范围变小。过小的梯度可能下溢为零，过大的中间结果则可能产生 Inf，并进一步传播为 NaN。对于 FP16，PyTorch AMP 会通过 loss scaling 放大反向传播中的梯度，参数更新前再恢复到原始尺度，并检查是否出现 Inf 或 NaN[**pytorch_amp**]。当发现异常时，可以跳过当前 batch 的参数更新，并调整后续的 loss scale。梯度裁剪则用于处理另一类问题：当少量样本产生异常大的梯度时，在参数更新前限制梯度范数，避免一次 batch 对模型造成过大的扰动。它并不能解决输入中的缺失值、除零或异常特征，因此两者需要分别处理。

训练监控也不能只看 loss。除了 AUC、LogLoss 等模型指标，还应关注梯度范数、参数范数、Inf/NaN 次数以及数据读取和梯度通信的等待时间。不同指标需要结合起来看，才能判断异常究竟来自数据、数值计算还是分布式训练链路。例如 AUC 接近 0.5 或者 LogLoss 出现明显恶化时，应该先比较当前版本与稳定版本的标签比例、特征映射、slot、sign、特征版本以及预测分布，再检查学习率、全局 batch 和负采样等训练配置，而不是直接继续训练或发布新模型。

一旦确认训练结果异常，当前版本应立即停止进入线上发布流程，保证线上服务继续使用已有的稳定版本。对于 NaN、Inf 等数值异常，我们还需要记录发生异常的 pass、训练步数、rank 和 Kafka 消费位置，并根据输入数据、loss scale、学习率和 Embedding 初始化等环节逐步定位原因。偶发异常可以跳过当前 batch 后继续观察。如果异常持续出现，或者多个评估窗口中的指标持续恶化，则应停止当前训练任务。

当训练任务发生机器或进程故障时，模型恢复必须从一份完整的 checkpoint 开始，而不能直接使用最新目录中的部分文件。需要恢复的模型状态至少应该包含模型参数、Embedding 分片、优化器状态、pass 编号以及 Kafka 消费位置。当模型恢复时，这些状态必须来自同一个 checkpoint。如果只恢复模型权重而缺少优化器状态，则会丢失已有的动量等训练信息。如果 Kafka 消费的位置不一致，则可能重复或跳过训练样本。

因此，checkpoint 发布与恢复都需要以完整性校验作为前提。最新的模型版本在缺少文件、清单不完整或者校验失败时，离线训练任务应该直接回退到上一个完整版本，而不是拼接不同版本的模型参数和 Embedding 参数。此外，在恢复后，我们还要重新检查 AUC、LogLoss、梯度范数和数据延迟，在确认训练状态正常后才能再继续消费 Kafka 数据和生成新的模型版本。

从整个训练链路来看，模型训练并不只是完成一次梯度更新。数据消费、分布式计算、稀疏参数、模型保存和线上发布都需要围绕同一个版本协同工作。只有训练状态和数据位置能够一起保存、一起恢复，持续训练任务才能真正做到可恢复，并稳定地产出可交付的模型版本。

16.9 本章小结

本章围绕推荐系统的离线服务展开，讨论如何将持续到达的用户行为和内容数据加工为训练样本、行为序列、统计特征、内容表示、检索索引、图关系和模型版本，并最终提供给线上推荐链路使用。整章按照数据从原始事件到服务化产物的流向展开，覆盖了样本构造、数据加工、索引更新和模型训练等主要环节。

样本流服务是离线链路的起点。常规样本流通过曝光与点击、播放、点赞等反馈进行拼接，并利用等待窗口处理延迟行为；级联样本流进一步保证粗排、精排等阶段的标注一致性；Session 样本流将同一 Session 内的多次请求组织为行为序列；非实时样本流则通过更长的等待窗口换取更加完整的反馈。针对强化学习场景，本章进一步介绍了基于 Redis 和 Flink 的状态转移样本构造方式，将相邻请求组织为 (s_t, a_t, r_t, s_{t+1}) 的格式，并扩展到长序列模型所需的历史行为。

在样本之外，本章还讨论了行为序列和统计特征的持续生产。行为日志解析服务将分散在不同 Topic 中的事件按用户或设备聚合为有序序列，并通过时间边界避免未来行为泄漏。统计特征生产服务则按照用户、物品、作者、类目等维度进行窗口聚合，通过事件去重和先验平滑保证计数与比率的稳定性，并触发相关索引更新。

内容理解和索引服务进一步将原始内容转换为可检索的线上数据。内容理解服务从图片、视频帧和文本中生成标签与 Embedding，多模态推理服务负责统一封装 GPU 模型调用；物品索引服务则将内容属性、统计特征和运营状态组织为正排、倒排和向量检索所需的数据，并通过全量重建、批量更新和实时事件驱动保持索引与物品状态同步。双 Buffer 机制保证索引切换期间产生的变更不会丢失。

图存储服务在传统索引之外提供关系型候选生成能力，将用户、物品、作者和类目之间的关系组织为可查询的图结构，并通过离线构建和实时更新持续维护。在线查询则通过 RPC 封装，并配合超时、限流和降级策略控制图服务对推荐链路的影响。

最后，本章介绍了离线训练任务如何消费前述服务产生的数据。Kafka 中的样本经过解码、特征处理和 batch 组装后进入分布式训练；稠密参数通过同步数据并行和 AllReduce 更新，大规模稀疏 Embedding 则通过参数服务器进行分片管理。训练过程以 checkpoint 保存模型参数、Embedding、优化器状态、训练进度和 Kafka 消费位置，并结合模型评估、混合精度、异常监控和故障回滚，持续产出可恢复、可验证和可发布的模型版本。

将这些服务串联起来，本章关注的核心问题可以归结为：如何让行为、内容、统计、关系和模型产物在统一的时间、特征和版本口径下持续演进。只有这样，在线推荐拿到的各类输入才能相互对应、可以重放，并能够在发生异常时恢复到一致的状态。